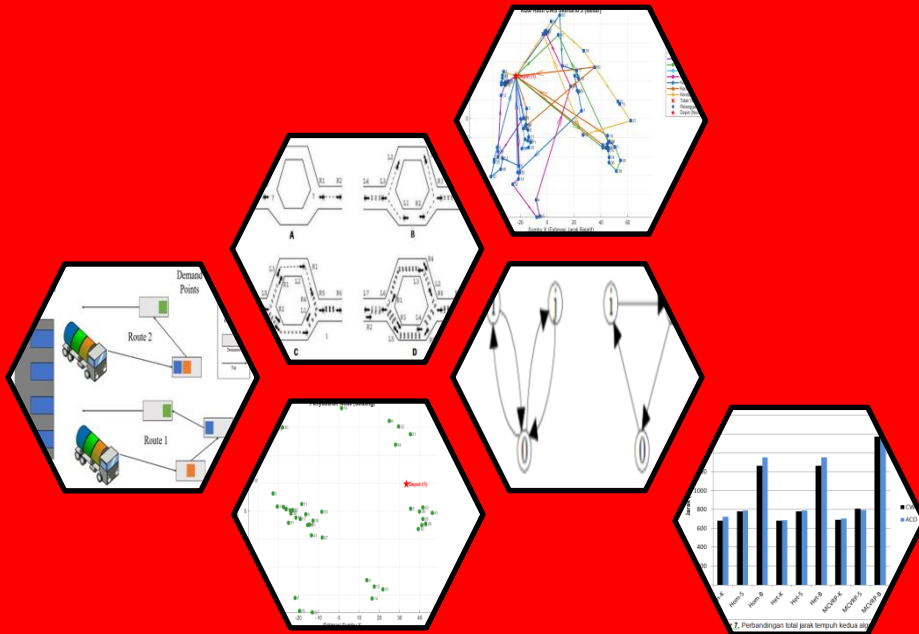


**PERBANDINGAN CLARKE AND WRIGHT SAVINGS DAN ANT
COLONY OPTIMIZATION PADA CAPACITATED VEHICLE ROUTING
PROBLEM DAN MULTI-COMPARTMENT VEHICLE ROUTING
PROBLEM**



HELMY ZAKINA

H011221054



**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

**PERBANDINGAN CLARKE AND WRIGHT SAVINGS DAN ANT COLONY
OPTIMIZATION PADA CAPACITATED VEHICLE ROUTING PROBLEM
DAN MULTI-COMPARTMENT VEHICLE ROUTING PROBLEM**

HELMY ZAKINA

H011221054



**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

**PERBANDINGAN CLARKE AND WRIGHT SAVINGS DAN ANT COLONY
OPTIMIZATION PADA CAPACITATED VEHICLE ROUTING PROBLEM
DAN MULTI-COMPARTMENT VEHICLE ROUTING PROBLEM**

HELMY ZAKINA

H011221054

Skripsi

Sebagai salah satu syarat untuk mencapai gelar sarjana

Program Studi Matematika

**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

SKRIPSI
PERBANDINGAN CLARKE AND WRIGHT SAVINGS DAN ANT COLONY
OPTIMIZATION PADA CAPACITATED VEHICLE ROUTING PROBLEM
DAN MULTI-COMPARTMENT VEHICLE ROUTING PROBLEM

HELMY ZAKINA

H011221054

Skripsi

Telah dipertahankan di depan Panitia Ujian Sarjana Matematika pada 11 Agustus
2026 dan dinyatakan telah memenuhi syarat kelulusan

Pada

Program Studi Sarjana Matematika
Departemen Matematika
Fakultas Matematika Dan Ilmu Pengetahuan Alam
Universitas Hasanuddin
Makassar

Mengesahkan:
Pembimbing tugas akhir,

Dr. Khaeruddin, M.Sc.

NIP. 19650914 199103 1 003

Mengetahui:
Ketua Program Studi,

Dr. Firmansyah, S.Si., M.Si.

NIP. 19680429 200212 1 001

PERNYATAAN KEASLIAN SKRIPSI DAN PELIMPAHAN HAK CIPTA

Dengan ini saya menyatakan bahwa, skripsi berjudul "Perbandingan Algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* dalam Penyelesaian CVRP dan MCVRP pada Optimasi Rute Distribusi BBM" adalah benar karya saya dengan arahan dari pembimbing Dr. Khaeruddin, M.Sc. sebagai Pembimbing Utama. Karya ilmiah ini belum diajukan dan tidak sedang diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka skripsi ini. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau seluruh skripsi ini adalah karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut berdasarkan aturan yang berlaku.

Dengan ini saya melimpahkan hak cipta (hak ekonomis) dari karya tertulis saya berupa skripsi ini kepada Universitas Hasanuddin.

Makassar, 18 Agustus 2026



UCAPAN TERIMA KASIH

Ucapan puji dan syukur kepada Allah SWT. yang telah melimpahkan Rahmat dan Hidayah-Nya kepada penulis sehingga tugas akhir berjudul “Perbandingan Algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* dalam Penyelesaian CVRP dan MCVRP pada Optimasi Rute Distribusi BBM” dapat terselesaikan. Penyelesaian tugas akhir ini tidak terlepas dari dukungan, bimbingan, dan bantuan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis menyampaikan terima kasih yang sebesar-besarnya dan penghargaan kepada orang tua penulis, Bapak Hamzah dan Ibu Hernayanti yang telah membesarkan dan mendidik penulis, dengan penuh kesabaran memberikan banyak kasih sayang serta senantiasa memberikan doa, dukungan, motivasi dan materi sehingga penulis bisa menyelesaikan perkuliahan ini. Begitu pula kepada kakak penulis Hesti Sugesti dan adik penulis Kherunnisa yang senantiasa mendukung dan membantu penulis dalam segala hal, serta kepada seluruh keluarga yang telah memberikan doa dan dukungan dalam menyelesaikan tugas akhir ini. Pada kesempatan ini pula, dengan segala kerendahan hati penulis ingin menyampaikan terima kasih kepada:

1. Bapak Prof. Dr. Ir. Jamaluddin Jompa, M.Sc., selaku Rektor Universitas Hasanuddin beserta seluruh jajarannya, serta Bapak Dr.Sci. Muhammad Zakir, M. Si. selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Hasanuddin beserta jajarannya.
2. Bapak Dr. Firman, S.Si., M.Sc., selaku ketua Departemen Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Hasanuddin. Bapak dan Ibu Dosen Departemen Matematika yang telah memberikan banyak ilmu dan pengetahuan kepada penulis selama menjadi mahasiswa di Program Studi Matematika, serta bapak dan ibu staf Departemen Matematika yang telah membantu penulis dalam hal administrasi.
3. Bapak Dr. Khaeruddin, M.Sc., selaku pembimbing dan penasihat akademik untuk segala ilmu, nasihat, dan kesabaran dalam membimbing dan mengarahkan penulis, serta bersedia meluangkan waktunya untuk mendampingi penulis sehingga skripsi ini dapat diselesaikan.
4. Bapak Prof. Agustinus Ribal, S.Si., M.Sc., PhD dan Ibu Naimah Aris, S.Si., M.Math., selaku penguji yang telah bersedia meluangkan waktunya untuk memberikan saran dan arahan kepada penulis dalam penulisan skripsi ini.
5. Sahabat-sahabat penulis Rita Haerani, Bulgis Dirtha Rahman, Reza Fadrianti, Sulpa Reski, Fira Safitri dan Nur Reski yang telah menemani, menyemangati, dan menjadi tempat berbagi keluh-kesah penulis selama perkuliahan.
6. Teman-teman tersayang penulis Fatimah Az-zahra dan Lutfiah Zahrani yang telah membantu dan menjadi teman diskusi penulis selama perkuliahan dan proses penulisan skripsi.

7. Teman-teman Matematika 2022 atas segala dukungan, kebersamaan, dan kerja samanya selama ini.
8. Apresiasi yang sebesar-sebesarnya kepada Helmy Zakina, diri saya sendiri yang telah kuat, tekun, sabar, dan kerja keras selama menyelesaikan skripsi ini.
9. Serta segala pihak yang tidak dapat disebutkan satu per satu yang telah membantu penulis dalam menyelesaikan skripsi ini.

Penulis menyadari bahwa skripsi ini masih jauh dari kata sempurna, sehingga kritik dan saran yang membangun akan selalu diterima dengan baik. Akhir kata, semoga skripsi ini dapat bermanfaat bagi pembaca dan dapat memberikan kontribusi yang berarti dalam bidang ilmu yang saya tekuni.

ABSTRAK

HELMY ZAKINA. **Perbandingan *Clarke and Wright Savings* dan *Ant Colony Optimization* pada *Capacitated Vehicle Routing Problem* dan *Multi-Compartment Vehicle Routing Problem*.** (dibimbing oleh Khaeruddin).

Latar Belakang: Optimasi rute distribusi merupakan salah satu upaya untuk meminimalkan total jarak tempuh sehingga biaya operasional distribusi dapat ditekan. **Tujuan:** Penelitian ini bertujuan membandingkan kinerja algoritma Clarke and Wright Savings dan Ant Colony Optimization (ACO) dalam menyelesaikan permasalahan *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP). **Metode:** Penelitian menggunakan dataset benchmark CVRPLIB yang dipraproses melalui perhitungan matriks jarak, konversi *demand* ke satuan liter, serta pembentukan tiga skenario penelitian, yaitu CVRP dengan kapasitas kendaraan homogen, CVRP dengan kapasitas kendaraan heterogen, dan MCVRP menggunakan kendaraan dua kompartemen. Kinerja kedua algoritma kemudian dibandingkan berdasarkan total jarak tempuh dan waktu komputasi. **Hasil:** Hasil penelitian menunjukkan bahwa algoritma Clarke and Wright Savings menghasilkan total jarak tempuh yang lebih rendah pada sebagian besar skenario penelitian dibandingkan algoritma ACO. Selain itu, Clarke and Wright Savings juga memerlukan waktu komputasi yang relatif lebih singkat, sedangkan ACO hanya menghasilkan total jarak tempuh yang lebih rendah pada skenario MCVRP dengan dataset sedang. **Kesimpulan:** Pada dataset benchmark CVRPLIB yang digunakan, algoritma Clarke and Wright Savings menghasilkan total jarak tempuh yang lebih rendah dengan waktu komputasi yang relatif lebih singkat dibandingkan algoritma ACO.

Kata kunci: benchmark CVRPLIB; distribusi bahan bakar; heuristik konstruktif; metaheuristik; optimasi kombinatorial; perencanaan rute kendaraan.

ABSTRACT

HELMY ZAKINA. **Performance Comparison of *Clarke and Wright Savings* and *Ant Colony Optimization* for the *Capacitated Vehicle Routing Problem* and the *Multi-Compartment Vehicle Routing Problem*.** (supervised by Khaeruddin).

Background: Distribution route optimization is an important approach to minimizing total travel distance and reducing distribution operational costs. **Objective:** This study aims to compare the performance of the Clarke and Wright Savings algorithm and Ant Colony Optimization (ACO) in solving the *Capacitated Vehicle Routing Problem* (CVRP) and *Multi-Compartment Vehicle Routing Problem* (MCVRP). **Methods:** This study employed benchmark datasets from CVRPLIB, which were preprocessed by computing the distance matrix, converting customer demand into liters, and constructing three research scenarios: CVRP with a homogeneous fleet, CVRP with a heterogeneous fleet, and MCVRP using two-compartment vehicles. The performance of both algorithms was evaluated based on total travel distance and computational time. **Results:** The results showed that the Clarke and Wright Savings algorithm produced shorter total travel distances in most research scenarios than ACO. In addition, Clarke and Wright Savings required relatively shorter computational time, while ACO produced a shorter total travel distance only in the medium-sized MCVRP scenario. **Conclusion:** For the benchmark CVRPLIB datasets used in this study, the Clarke and Wright Savings algorithm produced shorter total travel distances with relatively lower computational time than ACO.

Keywords: benchmark CVRPLIB; fuel distribution; constructive heuristic; metaheuristic; combinatorial optimization; vehicle routing.

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN PENGANTAR	ii
HALAMAN PENGESAHAN	Error! Bookmark not defined.
PERNYATAAN KEASLIAN SKRIPSI	Error! Bookmark not defined.
UCAPAN TERIMA KASIH	v
ABSTRAK	vii
ABSTRACT	viii
DAFTAR ISI	ix
DAFTAR GAMBAR	xi
DAFTAR TABEL	xii
DAFTAR LAMPIRAN	xiii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batas Masalah	3
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	3
1.6 Landasan Teori	3
1.6.1 Optimasi	3
1.6.2 Vehicle Routing Problem (VRP)	4
1.6.3 Capacitated Vehicle Routing Problem (CVRP)	5
1.6.4 Multi-Compartment Vehicle Routing Problem (MCVRP)	6
1.6.5 Clarke and Wright Savings	8
1.6.6 Ant Colony Optimization (ACO)	12
BAB II METODE PENELITIAN	20
2.1 Jenis Penelitian	20
2.2 Data Penelitian	20
2.3 Prapemrosesan Data	20
2.4 Pembentukan Skenario Penelitian	22
2.5 Implementasi Algoritma Clarke and Wright Savings	23
2.5.1 Implementasi pada CVRP dengan Armada Homogen	23

2.5.2	Implementasi pada CVRP dengan Armada Heterogen	24
2.5.3	Implementasi pada MCVRP dengan Dua Kompartemen	25
2.6	Implementasi Algoritma Ant Colony Optimization (ACO)	26
2.6.1	Implementasi pada CVRP dengan Armada Homogen	26
2.6.2	Implementasi pada CVRP dengan Armada Heterogen	26
2.6.3	Implementasi pada MCVRP dengan Dua Kompartemen	27
2.7	Prosedur Penelitian	28
2.8	Diagram Alir Penelitian.....	29
BAB III HASIL DAN PEMBAHASAN		29
3.1	Hasil	29
3.1.1	Deskripsi Data	29
3.1.2	Evaluasi Gap terhadap <i>Best Known Solution</i> (BKS)	30
3.2	Pembahasan	30
BAB IV PENUTUP		37
4.1	Kesimpulan	37
4.2	Saran.....	37
DAFTAR PUSTAKA		38
LAMPIRAN		40

DAFTAR GAMBAR

Gambar 1. Proses distribusi dengan MCVRP	7
Gambar 2. Penghematan rute.....	9
Gambar 3. Perjalanan semut menemukan sumber makanan	13
Gambar 4. Visualisasi penyebaran node (kecil).....	21
Gambar 5. Visualisasi penyebaran node (sedang)	21
Gambar 6. Visualisasi penyebaran node (besar).....	22
Gambar 7. Diagram alir penelitian	29
Gambar 8. Perbandingan total jarak tempuh kedua algoritma	34
Gambar 9. Perbandingan waktu komputasi kedua algoritma.....	36

DAFTAR TABEL

Tabel 1. Bentuk umum matriks jarak.....	9
Tabel 2. Bentuk umum matriks <i>savings</i>	10
Tabel 3. Matriks jarak.....	10
Tabel 4. Permintaan pelanggan CVRP	10
Tabel 5. Matriks <i>savings</i>	11
Tabel 6. Permintaan pelanggan MCVRP	12
Tabel 7. Parameter <i>Ant Colony Optimization</i> (ACO).....	29
Tabel 8. Hasil validasi implementasi algoritma	30
Tabel 9. Hasil Algoritma <i>Clarke and Wright Savings</i> Skenario 1	31
Tabel 10. Hasil Algoritma <i>Clarke and Wright Savings</i> Skenario 2.....	31
Tabel 11. Hasil Algoritma <i>Clarke and Wright Savings</i> Skenario 3.....	32
Tabel 12. Hasil pengujian parameter algoritma ACO	33
Tabel 13. Ringkasan statistik hasil pengujian algoritma ACO	33
Tabel 14. Persentase perubahan total jarak tempuh	35

DAFTAR LAMPIRAN

Lampiran 1. Rute hasil <i>Clarke and Wright Savings</i> skenario 1	40
Lampiran 2. Rute hasil <i>Clarke and Wright Savings</i> skenario 2	41
Lampiran 3. Rute hasil <i>Clarke and Wright Savings</i> skenario 3	43
Lampiran 4. Rute hasil <i>Ant Colony Optimization</i> skenario 1	44
Lampiran 5. Rute hasil <i>Ant Colony Optimization</i> skenario 2	46
Lampiran 6. Rute hasil <i>Ant Colony Optimization</i> skenario 3	47
Lampiran 7. Program Matlab pembagian <i>demand</i> pada MCVRP	48
Lampiran 8. Program Matlab visualisasi penyebaran node	50
Lampiran 9. Program Matlab <i>Clarke and Wright Savings</i> skenario 1	51
Lampiran 10. Program Matlab <i>Clarke and Wright Savings</i> skenario 2	55
Lampiran 11. Program Matlab <i>Clarke and Wright Savings</i> skenario 3	61
Lampiran 12. Program Matlab <i>Ant Colony Optimization</i> skenario 1	67
Lampiran 13. Program Matlab <i>Ant Colony Optimization</i> skenario 2	76
Lampiran 14. Program Matlab <i>Ant Colony Optimization</i> skenario 3	85

BAB I

PENDAHULUAN

1.1 Latar Belakang

Bahan Bakar Minyak (BBM) merupakan komoditas strategis yang memiliki peran penting dalam menunjang aktivitas ekonomi, transportasi, dan kehidupan masyarakat sehari-hari. Kebutuhan BBM terus mengalami peningkatan seiring dengan pertumbuhan jumlah kendaraan dan aktivitas ekonomi masyarakat (Gustova, 2025). Hal ini menjadikan distribusi BBM sebagai salah satu aspek penting dalam menjaga stabilitas energi nasional.

Distribusi merupakan salah satu komponen penting dalam manajemen rantai pasok yang bertujuan menyalurkan barang dari sumber menuju pelanggan. Dalam proses distribusi, penentuan rute kendaraan menjadi faktor yang sangat berpengaruh terhadap efisiensi perjalanan. Rute yang kurang optimal dapat menyebabkan kendaraan menempuh jarak yang lebih panjang dari yang seharusnya sehingga proses distribusi menjadi kurang efisien (Hulu et al., 2026). Oleh karena itu, diperlukan suatu metode yang mampu menentukan rute distribusi dengan total jarak tempuh minimum. Permasalahan tersebut menjadi salah satu kajian dalam bidang riset operasi yang memanfaatkan pendekatan analisis dan pemodelan matematis untuk memperoleh solusi yang optimal.

Permasalahan penentuan rute distribusi secara umum dimodelkan sebagai *Vehicle Routing Problem* (VRP), yaitu permasalahan optimasi yang bertujuan menentukan rute sejumlah kendaraan untuk melayani seluruh pelanggan dengan total jarak minimum. Seiring berkembangnya kebutuhan distribusi, VRP mengalami berbagai pengembangan untuk menyesuaikan karakteristik permasalahan nyata. Salah satu varian yang paling banyak digunakan adalah *Capacitated Vehicle Routing Problem* (CVRP) (Hulu et al., 2026). Model CVRP bertujuan untuk mempertimbangkan keterbatasan kapasitas kendaraan sehingga total permintaan pelanggan pada setiap rute tidak boleh melebihi kapasitas kendaraan yang digunakan (Soimun et al., 2025). Dalam kasus distribusi BBM, setiap kendaraan memiliki kapasitas maksimum yang tidak boleh dilampaui, sehingga perencanaan rute harus dilakukan secara optimal agar biaya, jarak atau waktu dapat diminimalkan.

Dalam praktik distribusi, karakteristik armada tidak selalu sama. Sebagian sistem distribusi menggunakan kendaraan dengan kapasitas yang berbeda-beda sesuai kebutuhan pengangkutan. Selain itu, terdapat pula sistem distribusi yang melibatkan lebih dari satu jenis produk sehingga kendaraan dilengkapi dengan beberapa kompartemen untuk mengangkut produk yang berbeda dalam satu perjalanan. Kondisi tersebut dimodelkan sebagai *Multi-Compartment Vehicle Routing Problem* (MCVRP), yaitu pengembangan CVRP yang mempertimbangkan pembagian kapasitas kendaraan ke dalam beberapa kompartemen sehingga lebih

sesuai untuk menggambarkan proses distribusi multi-produk (Bouleft dan Alaoui, 2023).

Untuk menyelesaikan permasalahan tersebut, diperlukan metode optimasi yang mampu menghasilkan solusi rute distribusi yang efisien. Salah satu metode klasik yang sering digunakan adalah algoritma *Clarke and Wright Savings*. Algoritma *Clarke and Wright* merupakan suatu algoritma yang berfungsi mencari lokasi pelanggan selanjutnya dengan memperhitungkan penghematan jarak yang muncul dari penggabungan dua pelanggan atau lebih ke dalam sebuah rute. Algoritma ini juga digunakan untuk menentukan rute yang harus ditempuh dengan mempertimbangkan kapasitas kendaraan (Kusuma & Sumianti, 2020)

Untuk meningkatkan kualitas solusi, diperlukan metode metaheuristik seperti *Ant Colony Optimization* (ACO). Algoritma ACO terinspirasi dari koloni semut dalam mencari makanan. Eksperimen yang dilakukan oleh Deneubourg, Goss Aron, dan Pasteels pada tahun 1989 menemukan bahwa ketika semut berjalan, semut melepaskan jejak feromon untuk menandai jalur yang telah ditempuh. Algoritma ini memiliki kemampuan yang relevan dalam menyelesaikan masalah optimasi yang kompleks (Vania & Rochmoeljati, 2025)

Berdasarkan uraian tersebut, pendekatan ini bertujuan untuk membandingkan kinerja algoritma *Clarke and Wright* dengan *Ant Colony Optimization* (ACO) dalam pengoptimalan sistem pendistribusian BBM menggunakan pendekatan *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP). Perbandingan dilakukan pada tiga skenario, yaitu CVRP dengan kapasitas kendaraan homogen, CVRP dengan kapasitas kendaraan heterogen, dan MCVRP dengan kendaraan dua kompartemen.

Dengan demikian, penulis tertarik untuk meneliti dengan judul: **“Perbandingan *Clarke and Wright Savings* dan *Ant Colony Optimization* pada *Capacitated Vehicle Routing Problem* dan *Multi-Compartment Vehicle Routing Problem*”**.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka dapat dirumuskan beberapa permasalahan sebagai berikut:

1. Bagaimana solusi rute distribusi yang dihasilkan oleh algoritma *Clarke and Wright Savings* pada *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP)?
2. Bagaimana solusi rute distribusi yang dihasilkan oleh algoritma *Ant Colony Optimization* (ACO) pada *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP)?
3. Bagaimana perbandingan kinerja antara algoritma *Clarke and Wright* dan *Ant Colony Optimization* (ACO) berdasarkan total jarak tempuh yang dihasilkan pada tiga skenario?

1.3 Batas Masalah

Adapun batas masalah dalam penelitian ini adalah:

1. Data yang digunakan merupakan tiga dataset benchmark CVRPLIB yang telah disesuaikan dengan kebutuhan penelitian, yaitu dataset berukuran kecil (31 node), sedang (43 node), dan besar (78 node).
2. Perhitungan jarak antar titik menggunakan jarak Euclidean sehingga kondisi jalan, kemacetan, cuaca, maupun hambatan operasional lainnya tidak dipertimbangkan.
3. Penelitian ini menggunakan tiga skenario, yaitu CVRP dengan kapasitas kendaraan homogen, CVRP dengan kapasitas kendaraan heterogen, dan MCVRP dengan kendaraan dua kompartemen.
4. *Demand* dataset benchmark dikonversi ke dalam satuan liter dan pada skenario MCVRP dibagi menjadi dua jenis BBM dengan tetap mempertahankan total *demand* setiap pelanggan.
5. Kapasitas kendaraan diasumsikan tetap sesuai skenario penelitian.
6. Penentuan rute terpendek ditentukan dengan memperhatikan jarak dan kapasitas.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini yaitu sebagai berikut:

1. Mengetahui penerapan algoritma *Clarke and Wright* dan *Ant Colony Optimization* (ACO) dalam menyelesaikan *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP).
2. Membandingkan kinerja kedua algoritma dalam menyelesaikan rute distribusi dengan total jarak tempuh minimum pada ketiga skenario penelitian.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberi manfaat sebagai berikut:

1. Menambah literatur dan pemahaman terkait penerapan *Clarke and Wright* dan *Ant Colony Optimization* (ACO) pada *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP).
2. Mengetahui perbandingan kinerja antara kedua algoritma sehingga dapat menjadi potensi dalam pengembangan penelitian.
3. Sebagai bahan pustaka di Universitas Hasanuddin.

1.6 Landasan Teori

1.6.1 Optimasi

Optimasi adalah proses menjadikan sesuatu menjadi yang terbaik atau efisien, seperti memaksimalkan keuntungan atau meminimalkan kerugian, dengan mempertimbangkan berbagai kendala. Oleh karena itu, optimasi dapat diterapkan untuk menyelesaikan masalah dengan batasan tertentu, seperti kendala waktu, kapasitas ruang, atau jumlah sumber daya (Winston, 2004). Kendala tersebut dapat

dinyatakan dalam bentuk himpunan persamaan linear atau pertidaksamaan variabel. Dengan demikian, fungsi yang dioptimalkan adalah solusi dengan nilai fungsi tujuan yang diinginkan. Nilai yang diinginkan adalah nilai maksimum yaitu nilai maksimum fungsi tujuan, dan nilai minimum yaitu nilai minimum fungsi tujuan.

Ada beberapa model dalam penyelesaian permasalahan optimasi, diantaranya adalah *Mathematical Programming*, merupakan teknik optimasi yang menggunakan model matematika untuk menentukan solusi terbaik. *Integer Programming* berfokus pada optimasi dengan variabel keputusan yang harus bernilai bilangan bulat. Kemudian *Mixed-Integer Programming*, menggabungkan variabel integer dan kontinu dalam satu model. Dan Optimasi Kombinatorial adalah pendekatan optimasi yang bekerja pada himpunan solusi yang bersifat diskrit.

1.6.2 Vehicle Routing Problem (VRP)

Vehicle Routing Problem (VRP) menurut Zahra dalam (Kurniawan et al., 2014) adalah masalah dalam menentukan rute pengiriman yang melibatkan serangkaian rute kendaraan-kendaraan yang berpusat pada satu depot untuk melayani pelanggan yang tersebar di berbagai area pengiriman dengan permintaannya masing-masing.

Vehicle Routing Problem (VRP) merupakan perkembangan dari masalah klasik yaitu *Traveling Salesman Problem* (TSP). VRP memiliki tujuan meminimumkan biaya transportasi, meminimumkan jumlah kendaraan, menyeimbangkan rute serta muatan kendaraan, dan meminimumkan penalti yang mengakibatkan terlambatnya pengiriman ke pelanggan. Berdasarkan tujuan tersebut, ada beberapa karakteristik dalam VRP yaitu (Kusuma dan Sumiati, 2020):

- a. Perjalanan kendaraan berawal dan berakhir dari dan ke depot yang sama.
- b. Ada sejumlah tempat yang semuanya harus dikunjungi dan dipenuhi permintaannya tepat satu kali.
- c. Jika kapasitas kendaraan sudah terpakai dan tidak dapat melayani pelanggan berikutnya, kendaraan dapat kembali ke depot untuk memenuhi kapasitas kendaraan dan melayani tempat berikutnya.

Berdasarkan Toth dan Vigo (2002), suatu permasalahan dapat dikategorikan sebagai VRP jika memenuhi syarat-syarat berikut (Rupawan, 2026):

- Adanya set pelanggan yang harus dijangkau.
Permasalahan VRP melibatkan sekumpulan pelanggan yang tersebar di lokasi tertentu. Setiap pelanggan memiliki permintaan (*demand*) yang harus dipenuhi.
- Keberadaan depot atau pusat distribusi
Terdapat satu atau lebih depot sebagai titik awal dan akhir perjalanan kendaraan.
- Kendala kapasitas kendaraan
Setiap kendaraan memiliki kapasitas maksimum yang tidak boleh dilampaui. Total permintaan pelanggan dalam satu rute tidak boleh melebihi kapasitas kendaraan.

- Tujuan minimisasi biaya atau jarak
Tujuan utama VRP adalah meminimalkan total biaya atau jarak tempuh kendaraan. Termasuk biaya transportasi, waktu perjalanan, atau jumlah kendaraan yang digunakan.
- Jumlah kendaraan yang terbatas
Jumlah kendaraan yang tersedia untuk melayani pelanggan terbatas. Ini menambah kompleksitas dalam merencanakan rute.
- Rute yang layak (*Feasible routes*)
Setiap rute yang dibuat harus memenuhi semua kendala, seperti kapasitas kendaraan, waktu layanan, dan batasan lainnya.
- Permasalahan kombinatorial
VRP adalah masalah kombinatorial yang kompleks karena melibatkan penentuan rute optimal dari sejumlah besar kemungkinan kombinasi rute.

1.6.3 Capacitated Vehicle Routing Problem (CVRP)

Capacitated Vehicle Routing Problem (CVRP) adalah bentuk yang paling umum dari permasalahan VRP dengan adanya pembatas kendala pada setiap kendaraan yang bersifat homogen berdasarkan keterbatasan kapasitas kendaraan yang digunakan untuk mengunjungi setiap pelanggan sesuai dengan permintaan masing-masing pelanggan tersebut. Banyaknya permintaan yang harus dikirimkan ke pelanggan $v_i \in N$ dinotasikan dengan $q_i \geq 0$. Semua kendaraan diasumsikan homogen dengan kapasitas yaitu $Q > 0$. Kendaraan yang melayani pelanggan $S \subseteq N$ dimulai dari depot, melangkah sekali ke setiap pelanggan di S , kembali lagi ke depot. Perjalanan yang dilakukan dari i ke j dikenakan biaya/jarak perjalanan C_{ij} (Pratama et al., 2022).

Menurut El Hassani, et al., *Capacitated Vehicle Routing Problem* (CVRP) dapat direpresentasikan sebagai graf terarah berbobot $G = (V, A)$ dimana $V = \{v_0, v_1, v_2, \dots, v_n\}$ yang merepresentasikan himpunan titik dan $A = \{(v_i, v_j) : i \neq j\}$ merepresentasikan himpunan sisi. Titik v_0 merepresentasikan depot/gudang dan titik $V = \{v_1, v_2, \dots, v_n\}$ menunjukkan pelanggan. Untuk setiap sisi (v_i, v_j) , nilai *non-negatif* d_{ij} menunjukkan jarak antara titik v_i dan titik v_j pada biaya atau waktu tempuh. Setiap pelanggan v_i memiliki permintaan q_i , dan waktu pelayanan δ_i dengan $\delta_i = 0, S_i = 0$. Untuk menentukan urutan kunjungan pelanggan, didefinisikan variabel keputusan :

$$x_{ij}^k = \begin{cases} 1, & \text{jika kendaraan } k \text{ bergerak dari } i \text{ ke } j \\ 0, & \text{lainnya} \end{cases} \quad (1)$$

$$y_i^k = \begin{cases} 1, & \text{jika pelanggan } i \text{ dilayani kendaraan } k \\ 0, & \text{lainnya} \end{cases} \quad (2)$$

dengan fungsi tujuan

$$\min \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} d_{ij} x_{ij}^k \quad (3)$$

Dengan kendala sebagai berikut:

1. Setiap pelanggan dikunjungi tepat satu kali.

$$\sum_{k \in V} \sum_{j \in N} x_{ij}^k = 1, \forall i \in V \quad (4)$$

2. Total permintaan semua titik dalam satu rute tidak melebihi kapasitas kendaraan

$$\sum_{i \in V} \sum_{j \in V} d_i x_{ij}^k \leq Q, \forall i \in V, k \in K \quad (5)$$

3. Semua kendaraan k berangkat dari depot.

$$\sum_{j \in V} x_{0j}^k = 1, \forall k \in K \quad (6)$$

4. Setiap kendaraan yang mengunjungi satu titik pasti akan meninggalkan titik tersebut

$$\sum_{i \in V} x_{ij}^k - \sum_{j \in V} x_{ji}^k = 0, \forall i \in V, k \in K \quad (7)$$

5. Semua kendaraan harus kembali ke depot $n + 1$.

$$\sum_{j \in V} x_{jn+1}^k = 1, \forall k \in K \quad (8)$$

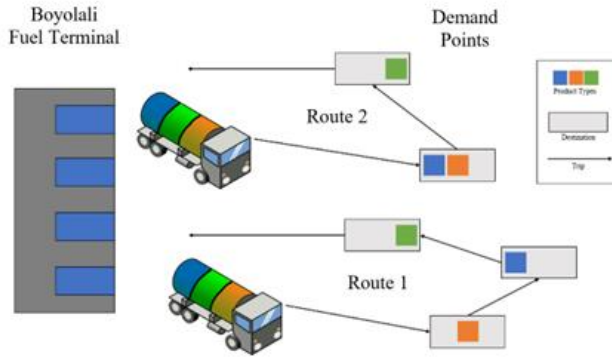
6. Variabel x_{ij}^k merupakan variabel biner

$$x_{ij}^k \in \{0,1\}, \quad \forall i, j \in V, \forall k \in K \quad (9)$$

(Patmawati & Nugroho, 2022)

1.6.4 Multi-Compartment Vehicle Routing Problem (MCVRP)

Multi-Compartment Vehicle Routing Problem (MCVRP) merupakan perkembangan dari CVRP, di mana dalam satu kendaraan terdapat beberapa kompartemen untuk mengangkut lebih dari satu jenis produk. (Anggraeni et al., 2023). Permasalahan ini bertujuan untuk mencari rute optimal dengan meminimalkan total jarak tempuh, dengan tetap memenuhi seluruh permintaan pelanggan serta memperhatikan kapasitas tiap kompartemen kendaraan. Untuk pemahaman lebih sederhana dapat dilihat pada Gambar 1 yang merupakan gambaran proses distribusi dengan MCVRP, dimana setiap kompartemen kendaraan memuat jenis produk yang berbeda (Anggraeni et al., 2023).



Gambar 1. Proses distribusi dengan MCVRP

Dalam model ini, himpunan pelanggan dinotasikan sebagai $N = \{1, 2, \dots, n\}$, sedangkan himpunan node yang mencakup depot dan pelanggan dinotasikan sebagai $V = \{0, 1, 2, \dots, n\}$ dengan node 0 sebagai depot. Himpunan kendaraan dinotasikan $K = \{1, 2, \dots, k\}$, dan himpunan produk atau kompartemen dinotasikan sebagai $M = \{1, 2, \dots, m\}$. Parameter yang digunakan meliputi d_{ij} sebagai jarak dari node i ke node j , q_{im} sebagai permintaan pelanggan i terhadap produk m , serta Q_{km} sebagai kapasitas kendaraan k pada kompartemen m , u_i sebagai variabel untuk eliminasi subtour. Untuk menentukan kombinasi rute dan distribusi yang optimal, didefinisikan variabel keputusan:

$$x_{ij}^k = \begin{cases} 1, & \text{jika kendaraan } k \text{ bergerak dari } i \text{ ke } j \\ 0, & \text{lainnya} \end{cases} \quad (10)$$

$$y_{ikm} = \begin{cases} 1, & \text{jika pelanggan } i \text{ dilayani kendaraan } k \text{ untuk produk } m \\ 0, & \text{lainnya} \end{cases} \quad (11)$$

dengan fungsi tujuan

$$\min z \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} d_{ij} x_{ij}^k \quad (12)$$

Dengan kendala sebagai berikut:

1. Setiap pelanggan dikunjungi tepat satu kali

$$\sum_{k \in K} \sum_{j \in V} x_{ij}^k = 1, \forall i \in N \quad (13)$$

2. Kendaraan yang masuk harus keluar

$$\sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = 0, \forall j \in N, \forall k \in K \quad (14)$$

3. Setiap kendaraan berangkat dari depot

$$\sum_{j \in V} x_{0j}^k \leq 1, \quad \forall k \in K \quad (15)$$

4. Setiap kendaraan kembali ke depot

$$\sum_{i \in V} x_{i0}^k \leq 1, \quad \forall k \in K \quad (16)$$

5. Kapasitas kompartemen Peralite

$$\sum_{i \in N} q_{i1} y_{i1}^k \leq Q_{k1}, \quad \forall i \in N, \quad 1, 2 \in M \quad (17)$$

6. Kapasitas kompartemen Solar

$$\sum_{i \in N} q_{i2} y_{i2}^k \leq Q_{k2}, \quad \forall i \in N, \quad 1, 2 \in M \quad (18)$$

7. Pemenuhan permintaan pelanggan

$$\sum_{k \in K} y_{im}^k = q_{im}, \quad \forall i \in N, m \in M \quad (19)$$

8. Hubungan alokasi produk dan rute

$$y_{im}^k \leq \sum_{j \in V} x_{ij}^k, \quad \forall i \in N, k \in K, m \in M \quad (20)$$

9. Eliminasi subtour (MTZ constraint)

$$u_i - u_j + n x_{ij}^k \leq n - 1, \quad \forall i \neq j \quad (21)$$

10. Variabel biner

$$x_{ij}^k \in \{0, 1\} \quad (22)$$

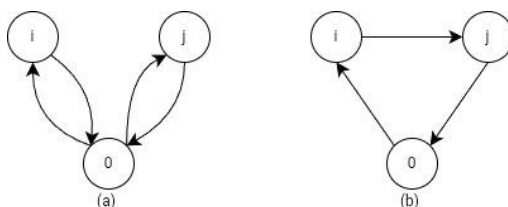
$$y_{im}^k \in \{0, 1\} \quad (23)$$

(Maulana, 2026)

1.6.5 Clarke and Wright Savings

Clarke and Wright Savings merupakan suatu metode yang ditemukan oleh *Clarke* dan *Wright* dalam paper berjudul *Scheduling of Vehicles from a Central Depot to a Number of Delivery Points* pada tahun 1964 yang memiliki tujuan untuk meminimumkan total jarak tempuh waktu dan/atau biaya dengan mempertimbangkan armada yang digunakan. Algoritma ini memiliki heuristik tersendiri dimana tidak menyediakan suatu penyelesaian yang optimal, namun juga sering menghasilkan penyelesaian yang baik, dan merupakan suatu penyelesaiannya yang sedikit berbeda dari penyelesaian dasar optimal (Anshoriy, 2023).

Algoritma *Clarke and Wright Saving* merupakan pengembangan dari algoritma pendekatan Dantzig Ramser. Perbedaan dari kedua algoritma tersebut adalah *Clarke* dan *Wright* memutuskan untuk menghubungkan kedua pelanggan dalam satu rute, daripada melayani tiap pelanggan dalam rute yang berbeda seperti pada Gambar 2 dengan titik nol adalah depot.



Gambar 2. Penghematan rute

Berdasarkan gambar di atas (a) menunjukkan pelanggan i dan j dikunjungi dengan rute yang berbeda sehingga memiliki jarak tempuh yang kurang optimal. Agar memiliki jarak tempuh yang minimal dan penghematan, maka dapat dilakukan penggabungan dua titik ke dalam satu rute seperti ada gambar (b), sehingga diperoleh jarak tempuh yang lebih optimal.

Secara umum, algoritma *Clarke and Wright Savings* dapat dituliskan dengan langkah-langkah sebagai berikut (Fadhilah, 2019):

1. Membuat matriks jarak pada setiap titik yang akan dikunjungi. Dengan bentuk umum matriks jarak sebagai berikut:

Tabel 1. Bentuk umum matriks jarak

	d_0	d_i	d_j	...	d_n
d_0	0	d_{0i}	d_{0j}		d_{0n}
d_i		0			
d_j			0		
...				0	
d_n					0

dimana:

d_0 = depot

d_i = pelanggan ke- i

d_j = pelanggan ke- j

d_{0i} = jarak antara depot dengan pelanggan ke- i

2. Menghitung nilai *savings*. Nilai *savings* merepresentasikan penghematan yang didapat dari menyatukan pesanan dua pelanggan ke dalam satu armada dengan menggunakan persamaan:

$$S_{ij} = d_{0i} + d_{0j} - d_{ij} \quad (24)$$

dimana:

S_{ij} = nilai penghematan jarak pelanggan ke- i ke pelanggan ke- j

d_{0i} = jarak antar depot dengan pelanggan ke- i

d_{0j} = jarak antar depot dengan pelanggan ke- j

d_{ij} = jarak antara pelanggan ke- i dengan pelanggan ke- j

Dengan bentuk umum matriks *savings* sebagai berikut:

Tabel 2. Bentuk umum matriks *savings*

	d_i	...	d_j	...	d_n
d_i	0		d_{ij}		d_{in}
...		0			
d_j			0		d_{jn}
...				0	
d_n					0

3. Mengalokasikan pelanggan ke dalam rute. Proses dimulai dengan membentuk rute baru. Berikut cara mengalokasikan pelanggan ke dalam rute baru:
- Langkah 1. Pilih kombinasi pelanggan dengan nilai *savings* terbesar dan masukkan ke dalam rute baru.
 - Langkah 2. Pilih pelanggan dengan total *savings* terbesar. Jika terdapat nilai total *savings* yang sama maka dipilih pelanggan yang memiliki jarak terdekat dari rute yang terbentuk.
 - Langkah 3. Mengulangi langkah 2 hingga kendaraan tidak bisa menampung permintaan pelanggan kembali dan rute selesai. Jika kendaraan masih mampu menampung permintaan tetapi sudah tidak ada pelanggan tersisa, maka rute selesai.
 - Langkah 4. Kembali ke langkah 1 jika terdapat pelanggan yang belum dialokasikan ke dalam rute.

Berikut contoh penerapan sederhana algoritma *Clarke and Wright* pada *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP).

Tabel 3. Matriks jarak

Node	0	1	2	3	4	5
0	0	10	12	15	20	18
1	10	0	8	9	14	16
2	12	8	0	6	10	11
3	15	9	6	0	8	9
4	20	14	10	8	0	7
5	18	16	11	9	7	0

Tabel 4. Permintaan pelanggan CVRP

Pelanggan	1	2	3	4	5
Demand	10	15	20	25	30

Kapasitas kendaraan (Q) = 60

Contoh 1. Penerapan *Clarke and Wright Savings* pada CVRP

Langkah 1: Rute Awal

- Rute 1 : 0 → 1 → 0 Jarak = 10 + 10 = 20
- Rute 2 : 0 → 2 → 0 Jarak = 12 + 12 = 24
- Rute 3 : 0 → 3 → 0 Jarak = 15 + 15 = 30
- Rute 4 : 0 → 4 → 0 Jarak = 20 + 20 = 40
- Rute 5 : 0 → 5 → 0 Jarak = 18 + 18 = 36

Total jarak awal : $20 + 24 + 30 + 40 + 36 = 150$ km

Jumlah kendaraan awal = 5

Langkah 2: Hitung Nilai *Savings*

Hitung semua pasangan

Tabel 5. Matriks *savings*

	1	2	3	4	5
1	0	14	16	16	12
2	14	0	21	22	19
3	16	21	0	27	24
4	16	22	27	0	31
5	12	19	24	31	0

Langkah 3: Mengalokasikan pelanggan ke dalam rute

➤ Urutkan *savings* terbesar

- $S_{45} = 31$
- $S_{34} = 27$
- $S_{35} = 24$
- $S_{24} = 22$
- $S_{23} = 21$
- $S_{25} = 19$
- $S_{13} = 16$
- $S_{14} = 16$
- $S_{12} = 14$
- $S_{15} = 12$

➤ Penggabungan Rute

$$\text{Total demand} \leq Q$$

- Penggabungan (4,5)

$$\text{demand total} = 25 + 30 = 55 \leq 60$$

$$\text{rute baru: } 0 - 4 - 5 - 0$$

- Penggabungan (3,4)

$$\text{demand total} = 20 + 25 + 30 = 75 > 60$$

melebihi kapasitas kendaraan, pelanggan 3 tidak masuk list rute 1

- Penggabungan (2,4)

$$\text{demand total} = 15 + 25 + 30 = 70 > 60$$

Melebihi kapasitas kendaraan, pelanggan 2 tidak masuk list rute 1

- Penggabungan (2,3)

$$\text{demand total} = 15 + 20 = 35 \leq 60$$

$$\text{rute baru: } 0 - 2 - 3 - 0$$

- Penggabungan (1,2)

$$\text{demand total} = 10 + 15 + 20 = 45 \leq 60$$

$$\text{rute baru: } 0 - 1 - 2 - 3 - 0$$

Langkah 4: Rute Akhir

- Rute 1 : $0 - 4 - 5 - 0$

$$L_1 = 20 + 7 + 18 = 45$$

- Rute 2 : $0 - 1 - 2 - 3 - 0$

$$L_2 = 10 + 8 + 6 + 15 = 39$$

Jumlah kendaraan yang digunakan : 2

$$L_{\text{total}} = 45 + 39 = 84$$

Diperoleh total jarak akhir sebesar 84 km dari jarak awal sebesar 150 km maka penghematan yang diperoleh sebanyak 44%.

Contoh 2. Penerapan *Clarke and Wright Savings* pada MCVRP

Berdasarkan Tabel 3 akan dihitung total jarak tempuh dengan MCVRP.

Tabel 6. Permintaan pelanggan MCVRP

Pelanggan	1	2	3	4	5
<i>Demand</i>	P1	6	5	4	7
	P2	4	3	6	2

Jumlah kompartemen kendaraan: 2

Kapasitas: Kompartemen 1 = 15

Kompartemen 2 = 15

Berdasarkan Langkah 1 hingga Langkah 2 pada Contoh 1.1 akan dilanjutkan Langkah 3.

Langkah 3: Mengalokasikan pelanggan ke dalam rute

➤ Urutkan *savings* terbesar

- $S_{45} = 31$
- $S_{34} = 27$
- $S_{35} = 24$
- $S_{24} = 22$
- $S_{23} = 21$
- $S_{25} = 19$
- $S_{13} = 16$
- $S_{14} = 16$
- $S_{12} = 14$
- $S_{15} = 12$

➤ Penggabungan Rute

- Penggabungan (4, 5)

$$P1 = 7 + 3 = 10$$

$$P2 = 2 + 5 = 7$$

$$\text{rute baru: } 0 - 4 - 5 - 0$$

- Penggabungan (2, 3)

$$P1 = 5 + 4 = 9$$

$$P2 = 3 + 6 = 9$$

$$\text{rute baru: } 0 - 2 - 3 - 0$$

- Penggabungan (1, 2, 3)

$$P1 = 9 + 6 = 15$$

$$P2 = 9 + 4 = 14$$

$$\text{rute baru: } 0 - 1 - 2 - 3 - 0$$

Langkah 4: Rute Akhir

- Rute 1 : $0 - 4 - 5 - 0$

$$L_1 = 20 + 7 + 18 = 45$$

- Rute 2 : $0 - 1 - 2 - 3 - 0$

$$L_2 = 10 + 8 + 6 + 15 = 39$$

Jumlah kendaraan yang digunakan: 2

$$L_{\text{total}} = 45 + 39 = 84$$

Diperoleh total jarak akhir sebesar 84 km dari jarak awal sebesar 150 km maka penghematan yang diperoleh sebanyak 44%.

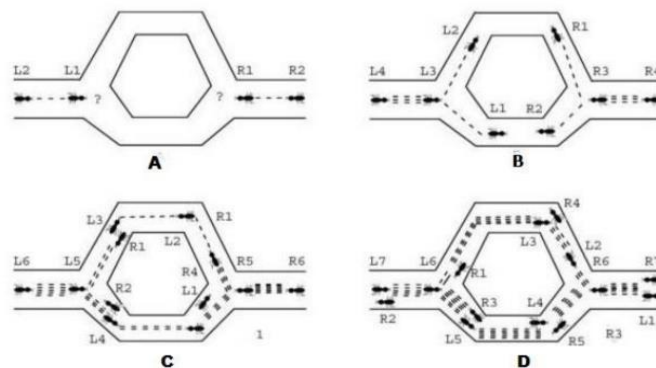
1.6.6 *Ant Colony Optimization (ACO)*

Algoritma *Ant Colony Optimization (ACO)* dikembangkan oleh Marco Dorigo pada tahun 1991 yang terinspirasi dari hasil percobaan mengenai tingkah laku koloni semut dalam mencari makanan. Semut mampu mencari makanan dengan cara

menemukan jarak terpendek antara sumber makanan dengan sarangnya tanpa menggunakan indra penglihatan (Pratama et al., 2022).

Semut berkomunikasi menggunakan zat kimia yang berasal dari kelenjar endokrin yang disebut feromon. Feromon ini akan menjadi penanda bahwa rute telah dilewati dan akan mempengaruhi individu lain dari spesies yang sama. Feromon menguap dari waktu ke waktu. Rute yang mengandung feromon yang cukup padat merupakan rute terpendek, karena waktu yang dihabiskan untuk pergi ke sumber makan dan kembali ke koloninya, semut selalu meninggalkan feromon.

Seiring waktu, jejak feromon akan menguap dan akan mengurangi kekuatan daya tariknya. Jejak akan hilang karena jarang dilalui. Sedangkan rute pendeknya akan memiliki padatan feromon yang tinggi dan akan membuat rute tersebut dapat dipilih dan rute yang jauh akan ditinggalkan. Proses pelepasan feromon ini disebut *stigmergy*, yaitu mekanisme di mana semut memodifikasi lingkungan sekitarnya (Karjono et al., 2016).



ACO memiliki empat karakteristik utama, diantaranya yaitu: aturan transisi status, *update pheromone trail*, *update local trail*, dan daftar calon rute dari kota yang dilalui (Agustiningsih & Wahyuningsih).

1. Aturan Transisi Status

Aturan transisi status yang digunakan disebut *probability*. P_{ij} merupakan probabilitas dari k semut pada titik i yang memilih untuk menuju titik j . Nilai probabilitas dihitung menggunakan rumus yang ditunjukkan pada persamaan (25).

$$P_{ij}^k = \begin{cases} \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{j \in J_i} [\tau_{ij}]^\alpha [\eta_{ij}]^\beta}, & \text{jika } j \in \text{dilalui} \\ 0, & \text{lainnya} \end{cases} \quad (25)$$

dengan:

P_{ij}^k = Probabilitas semut antar titik.

τ_{ij} = Intensitas feromon antara titik.

η_{ij} = Visibilitas antar titik.

α = Konstanta pengendali intensitas feromon, nilai $\alpha > 0$.

β = Konstanta pengendali visibilitas, nilai $\beta > 0$.

J_i = Pelanggan yang belum dikunjungi dan tidak melanggar kapasitas.

2. Update Pheromone Trail

Setelah semua semut menyelesaikan perjalanannya masing-masing maka feromon di-*update*. Pembaruan feromon bertujuan untuk menghasilkan jejak feromon baru yang akan dipakai pada iterasi berikutnya. Perhitungan pembaruan ini dapat dilakukan dengan menggunakan persamaan (26).

$$\tau_{ij}(t+1) = \rho \cdot \tau_{ij}(t) + \Delta\tau_{ij} \quad (26)$$

di mana:

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k}, & \text{jejak semut } k \text{ melewati sisi } (i, j) \\ 0, & \text{lainnya} \end{cases} \quad (27)$$

$$\Delta\tau_{ij} = \sum_{k=1}^m \Delta\tau_{ij}^k \quad (28)$$

dengan:

$\tau_{ij}(t+1)$ = Nilai jejak feromon pada iterasi berikutnya (baru).

ρ = Tingkat penguapan jejak feromon ($0 < \rho < 1$).

$\tau_{ij}(t)$ = Nilai jejak feromon pada iterasi sebelumnya (lama).

$\Delta\tau_{ij}$ = Perubahan total feromon semut yang ditambahkan pada antar titik dari semua semut.

$\Delta\tau_{ij}^k$ = Perubahan kontribusi feromon semut antar titik

Q = Konstanta tetapan siklus.

L_k = Panjang rute semut ke- k

3. *Update local trail*

Selama melakukan *tour* untuk mencari solusi VRP, semut mengunjungi *edges* dan mengubah tingkat feromon pada *edges* tersebut dengan menerapkan aturan pembaruan feromon lokal. Adanya penguapan feromon menyebabkan semut-semut tidak mengikuti jalur yang sama dengan semut sebelumnya.

4. Daftar calon rute

Salah satu kota yang lebih dekat dan memiliki probabilitas yang lebih tinggi dipilih sebagai kota berikutnya.

Adapun langkah-langkah penyelesaian masalah dengan menggunakan metode *Ant Colony Optimization* (ACO) sebagai berikut (Suliantoro et al.):

1. Penerapan parameter, seperti banyak titik distribusi, α, β, ρ , jumlah semut (m) dan konstanta siklus semut (Q).
2. Membuat matriks *visibility*

$$\eta_{ij} = \frac{1}{d_{ij}} \quad (29)$$

3. Menentukan matriks feromon awal.
4. Semut ditempatkan pada titik awal yang berbeda-beda, kemudian semut mulai membuat sebuah perjalanan menuju titik pertama secara acak.
5. Memilih titik berikutnya dengan aturan transisi dengan menggunakan persamaan (25). Pilih titik berikutnya dengan probabilitas terbesar.
6. Pembaruan jumlah feromon dengan menggunakan persamaan (30)

$$\tau_{ij} = (1 - \rho) \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k \quad (30)$$

7. Ulangi sampai setiap semut mengunjungi semua titik distribusi.
8. Sesuaikan permintaan masing-masing titik distribusi dengan kapasitas armada. Apabila kapasitas sudah penuh, armada harus kembali ke depot dan pendistribusian titik yang tersisa menggunakan armada lain.

Persamaan tersebut menunjukkan bahwa nilai feromon baru diperoleh dari hasil evaporasi feromon sebelumnya yang kemudian ditambahkan dengan akumulasi feromon dari seluruh semut yang melewati lintasan tersebut. Dengan demikian, lintasan yang menghasilkan solusi lebih baik akan memiliki intensitas feromon yang lebih tinggi sehingga peluangnya untuk dipilih pada iterasi berikutnya juga semakin besar.

Berikut contoh penerapan sederhana algoritma *Ant Colony Optimization* (ACO) pada *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP).

Contoh 3. Penerapan ACO pada CVRP

Berdasarkan Tabel 3 dan Tabel 4 akan dihitung jarak optimal dengan menggunakan algoritma ACO pada CVRP.

Langkah 1: Inisialisasi parameter

- $m = 5$
- $Q = 10$
- $\alpha = 1$
- $\beta = 2$
- $\rho = 0,5$

Langkah 2: Matriks *visibility*

$$\eta_{ij} = \begin{bmatrix} 0 & 0,1 & 0,0833 & 0,0667 & 0,05 & 0,0556 \\ 0,1 & 0 & 0,125 & 0,111 & 0,0714 & 0,0625 \\ 0,0833 & 0,125 & 0 & 0,1667 & 0,1 & 0,0909 \\ 0,0667 & 0,111 & 0,1667 & 0 & 0,125 & 0,111 \\ 0,05 & 0,0714 & 0,1 & 0,125 & 0 & 0,1428 \\ 0,0556 & 0,0625 & 0,0909 & 0,111 & 0,1428 & 0 \end{bmatrix}$$

Langkah 3: Matriks feromon awal

$$\tau_{ij} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Langkah 4: Probabilitas transisi**➤ Iterasi 1**

• Kendaraan 1

$$P_{01} = 0,3706$$

$$P_{03} = 0,1649$$

$$P_{05} = 0,1146$$

$$P_{02} = 0,2572$$

$$P_{04} = 0,0927$$

Dipilih probabilitas terbesar yaitu pelanggan 1 = 0,3706

• Status kendaraan

$$\text{Load kendaraan} = \text{demand}(1) = 10$$

$$\text{Sisa kapasitas} = 60 - 10 = 50$$

$$\text{Rute : } 0 - 1$$

Pilih pelanggan berikutnya dari pelanggan 1

$$P_{12} = 0,4229$$

$$P_{14} = 0,1380$$

$$P_{13} = 0,3334$$

$$P_{15} = 0,1057$$

Dipilih probabilitas terbesar yaitu pelanggan 2 = 0,4229

• Status kendaraan

$$\text{Load kendaraan} = 10 + 15 = 25$$

$$\text{Sisa kapasitas} = 50 - 15 = 35$$

$$\text{Rute : } 0 - 1 - 2$$

Pilih pelanggan berikutnya dari pelanggan 2

$$P_{23} = 0,6034$$

$$P_{24} = 0,2171$$

$$P_{25} = 0,1794$$

Dipilih probabilitas terbesar yaitu pelanggan 3 = 0,6034

• Status kendaraan

$$\text{Load kendaraan} = 25 + 20 = 45$$

$$\text{Sisa kapasitas} = 35 - 20 = 15$$

$$\text{Rute : } 0 - 1 - 2 - 3$$

Pelanggan yang tersisa yaitu pelanggan 4 dan pelanggan 5 dimana *demand* dari kedua pelanggan tidak cukup untuk sisa kapasitas yang tersedia pada kendaraan 1 (tidak *feasible*). Maka kendaraan 1 kembali ke depot dan dilanjutkan kendaraan 2. Ulangi Langkah 4 pada kendaraan 1 untuk kendaraan 2 pada pelanggan yang belum dilayani, lakukan hingga semua pelanggan telah dilayani.

Diperoleh rute lengkap untuk:

- Kendaraan 1: 0 – 1 – 2 – 3 – 0
Load kendaraan = 45
 $L_1 = 10 + 8 + 6 + 15 = 39$
 - Kendaraan 2: 0 – 5 – 4 – 0
Load kendaraan = 55
 $L_2 = 18 + 7 + 20 = 45$
- $$L_{\text{total}} = 39 + 45 = 84$$

Langkah 5: Pembaruan feromon

- Evaporasi feromon

$$\tau_{ij} = (1 - 0,5) 1 = 0,5$$

- Penambahan feromon

$$\Delta\tau_{ij}^k = \frac{10}{84} = 0,1190$$

Untuk rute yang dilintasi

$$\tau_{ij} = \tau_{ji} = 0,5 + 0,1190 = 0,6190$$

Untuk rute yang tidak dilintasi

$$\tau_{ij} = \tau_{ji} = 0,5$$

Matriks feromon baru

$$\tau_{ij} = \begin{bmatrix} 0,5 & 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,6190 \\ 0,6190 & 0,5 & 0,6190 & 0,6190 & 0,6190 & 0,6190 \\ 0,6190 & 0,6190 & 0,5 & 0,6190 & 0,6190 & 0,6190 \\ 0,6190 & 0,6190 & 0,6190 & 0,5 & 0,6190 & 0,6190 \\ 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,5 & 0,6190 \\ 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,5 \end{bmatrix}$$

Tahap 6: Iterasi maksimum

Diperoleh iterasi 1 dengan total jarak sebesar 84 km. Untuk mengetahui apakah iterasi 1 memenuhi kriteria atau tidak, lanjutkan iterasi 2 hingga konvergen.

Contoh 4. Penerapan ACO pada MCVRP

Berdasarkan Tabel 3 dan Tabel 6 akan dihitung jarak optimal dengan menggunakan algoritma ACO pada MCVRP.

Jumlah kompartemen kendaraan: 2

Kapasitas: Kompartemen 1 = 15

Kompartemen 2 = 15

Berdasarkan Langkah 1 hingga Langkah 3 pada Contoh 1.3 akan dilanjutkan Langkah 4.

Langkah 4: Probabilitas transisi

➤ Iterasi 1

- Kendaraan 1

$$P_{01} = 0,3706$$

$$P_{03} = 0,1649$$

$$P_{05} = 0,1146$$

$$P_{02} = 0,2572$$

$$P_{04} = 0,0927$$

Dipilih probabilitas terbesar yaitu pelanggan 1 = 0,3706

- Status kendaraan

$$\text{Load kendaraan P1} = 6$$

$$\text{Load kendaraan P2} = 4$$

$$\text{Sisa kapasitas} = 15 - 6 = 9$$

$$\text{Sisa kapasitas} = 15 - 4 = 11$$

$$\text{Rute : } 0 - 1$$

Pilih pelanggan berikutnya dari pelanggan 1

$$\begin{array}{ll} P_{12} = 0,4229 & P_{14} = 0,1380 \\ P_{13} = 0,3334 & P_{15} = 0,1057 \end{array}$$

Dipilih probabilitas terbesar yaitu pelanggan 2 = 0,4229

- Status kendaraan

$$\begin{array}{ll} \text{Load kendaraan P1} = 6 + 5 = 11 & \text{Load kendaraan P2} = 4 + 3 = 7 \\ \text{Sisa kapasitas} = 9 - 5 = 4 & \text{Sisa kapasitas} = 11 - 3 = 8 \end{array}$$

$$\text{Rute : } 0 - 1 - 2$$

Pilih pelanggan berikutnya dari pelanggan 2

$$P_{23} = 0,6034 \quad P_{24} = 0,2171 \quad P_{25} = 0,1794$$

Dipilih probabilitas terbesar yaitu pelanggan 3 = 0,6034

- Status kendaraan

$$\begin{array}{ll} \text{Load kendaraan P1} = 11 + 4 = 15 & \text{Load kendaraan P2} = 7 + 6 = 13 \\ \text{Sisa kapasitas} = 4 - 4 = 0 & \text{Sisa kapasitas} = 8 - 6 = 2 \end{array}$$

$$\text{Rute : } 0 - 1 - 2 - 3$$

Pelanggan yang tersisa yaitu pelanggan 4 dan pelanggan 5 dimana *demand* dari kedua pelanggan tidak cukup untuk sisa kapasitas yang tersedia pada kendaraan 1, baik pada kompartemen 1 maupun kompartemen 2 (tidak *feasible*). Maka kendaraan 1 kembali ke depot dan dilanjutkan kendaraan 2. Ulangi Langkah 4 pada kendaraan 1 untuk kendaraan 2 pada pelanggan yang belum dilayani, lakukan hingga semua pelanggan telah dilayani.

Diperoleh rute lengkap untuk:

- Kendaraan 1: 0 - 1 - 2 - 3 - 0
Load kendaraan P1 = 15
Load kendaraan P2 = 13
 $L_1 = 10 + 8 + 6 + 15 = 39$
- Kendaraan 2: 0 - 5 - 4 - 0
Load kendaraan P1 = 10
Load kendaraan P2 = 7
 $L_2 = 18 + 7 + 20 = 45$

$$L_{\text{total}} = 39 + 45 = 84$$

Langkah 5: Pembaruan feromon

- Evaporasi feromon

$$\tau_{ij} = (1 - 0,5) 1 = 0,5$$

- Penambahan feromon

$$\Delta\tau_{ij}^k = \frac{10}{84} = 0,1190$$

Untuk rute yang dilintasi

$$\tau_{ij} = \tau_{ji} = 0,5 + 0,1190 = 0,6190$$

Untuk rute yang tidak dilintasi

$$\tau_{ij} = \tau_{ji} = 0,5$$

Matriks feromon baru

$$\tau_{ij} = \begin{bmatrix} 0,5 & 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,6190 \\ 0,6190 & 0,5 & 0,6190 & 0,6190 & 0,6190 & 0,6190 \\ 0,6190 & 0,6190 & 0,5 & 0,6190 & 0,6190 & 0,6190 \\ 0,6190 & 0,6190 & 0,6190 & 0,5 & 0,6190 & 0,6190 \\ 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,5 & 0,6190 \\ 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,6190 & 0,5 \end{bmatrix}$$

Langkah 6: Iterasi maksimum

Diperoleh iterasi 1 dengan total jarak sebesar 84 km. Untuk mengetahui apakah iterasi 1 memenuhi kriteria atau tidak, lanjutkan iterasi 2 hingga konvergensi.

BAB II

METODE PENELITIAN

2.1 Jenis Penelitian

Jenis penelitian yang digunakan dalam penelitian ini adalah analisis komparatif yang bertujuan membandingkan kinerja dua algoritma yaitu, *Clarke and Wright* dan *Ant Colony Optimization* (ACO) dalam menyelesaikan *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP). Perbandingan dilakukan pada tiga skenario, yaitu CVRP dengan kapasitas kendaraan homogen, CVRP dengan kapasitas kendaraan heterogen, dan MCVRP dengan kendaraan dua kompartemen. Kinerja kedua algoritma dievaluasi berdasarkan indikator yang telah ditentukan, seperti total jarak tempuh, waktu komputasi, utilitas kendaraan atau kompartemen, serta persentase peningkatan kinerja.

2.2 Data Penelitian

Penelitian ini menggunakan data sekunder yang berasal dari benchmark *Capacitated Vehicle Routing Problem Library* (CVRPLIB) sebagai sumber utama. Data yang digunakan terdiri atas tiga *instance*, yaitu B-n31-k5, B-n43-k6, dan B-n78-k10, yang mempresentasikan permasalahan dengan ukuran kecil, sedang, dan besar. Penggunaan ketiga *instance* tersebut bertujuan untuk mengevaluasi kinerja algoritma *Clarke and Wright* dan *Ant Colony Optimization* (ACO) pada tingkat kompleksitas yang berbeda sehingga dapat dianalisis konsistensi performa kedua algoritma terhadap penambahan jumlah pelanggan.

2.3 Prapemrosesan Data

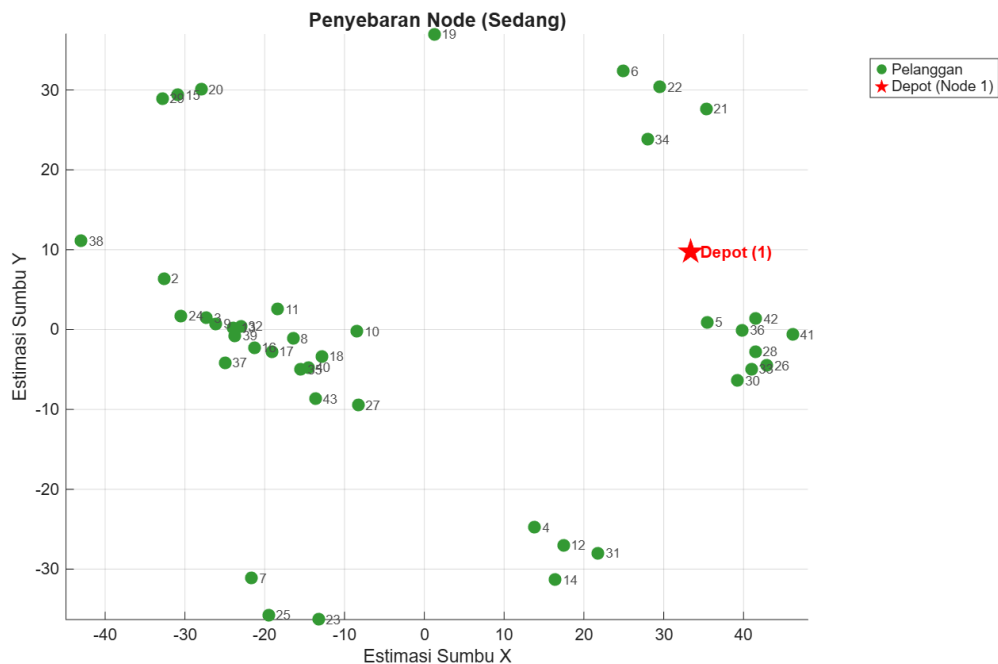
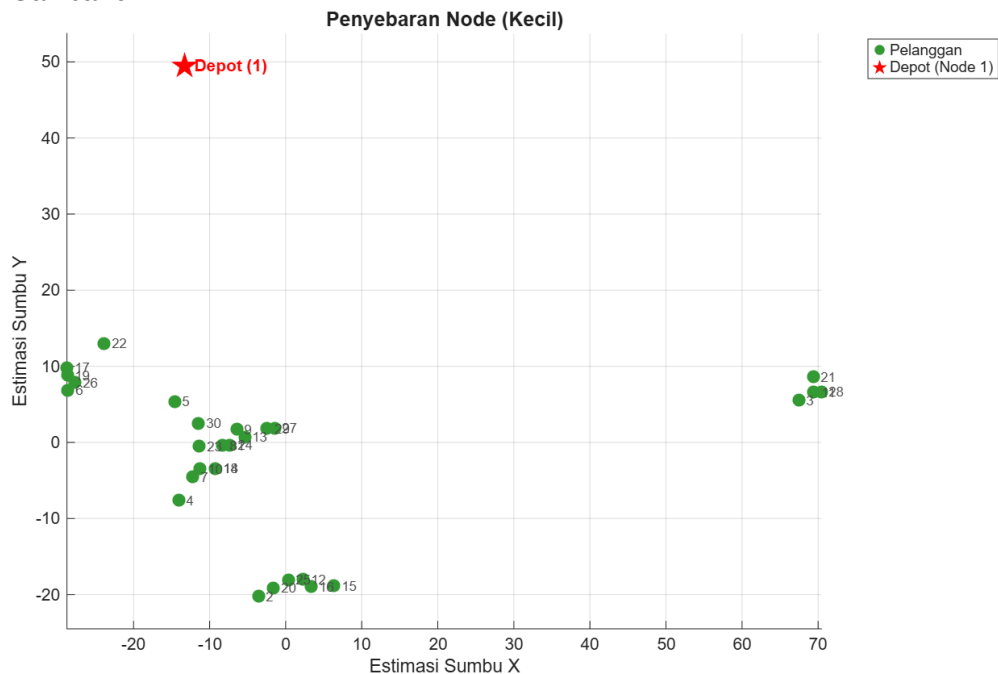
Dataset CVRPLIB yang memuat data dasar CVRP, seperti koordinat depot dan pelanggan, permintaan pelanggan (*demand*), kapasitas kendaraan, dan jumlah kendaraan, diprapemrosesan meliputi pembentukan matriks jarak, konversi nilai permintaan pelanggan ke satuan liter, serta pembentukan tiga skenario penelitian.

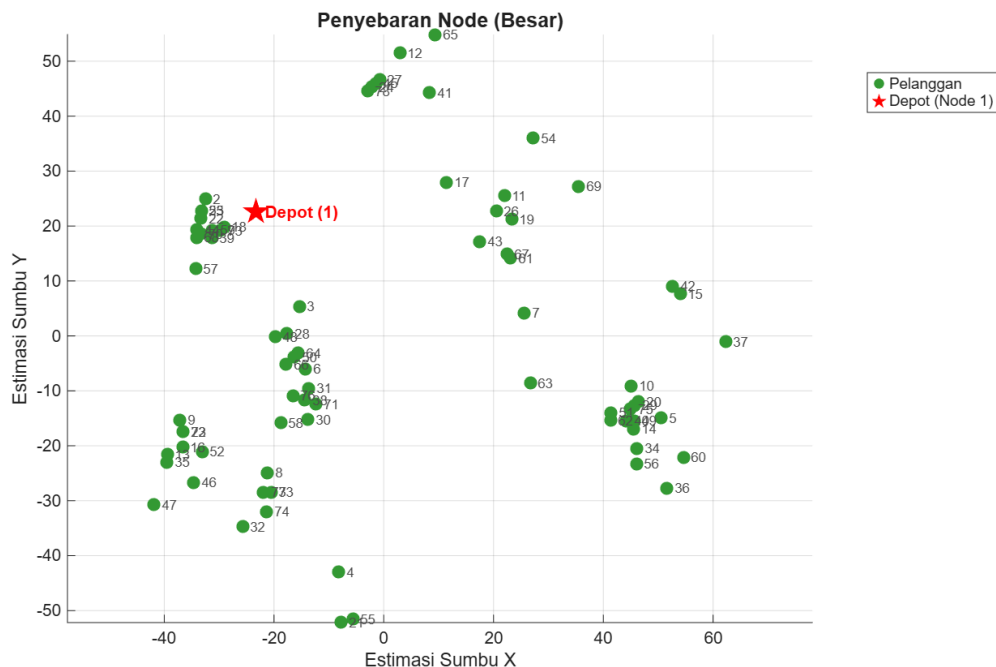
Matriks jarak dihitung menggunakan jarak Euclidean berdasarkan koordinat depot dan pelanggan. Jarak antar titik i dan j dihitung menggunakan persamaan berikut:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (31)$$

dengan d_{ij} merupakan jarak antar titik i dan j , sedangkan (x_i, y_i) dan (x_j, y_j) merupakan koordinat kedua titik. Matriks jarak yang diperoleh selanjutnya digunakan untuk merekonstruksi koordinat dua dimensi menggunakan metode *Classical Multidimensional Scaling* (CMDS). Hasil rekonstruksi tersebut divisualisasikan untuk memberikan gambaran penyebaran node pada setiap

instance penelitian, sebagaimana ditunjukkan pada Gambar 4 sampai dengan Gambar 6





Gambar 6. Visualisasi penyebaran node (besar)

Selanjutnya, nilai permintaan pada dataset CVRPLIB dikonversi ke satuan liter menggunakan faktor konversi yang diperoleh dari perbandingan kapasitas kendaraan distribusi BBM sebesar 32.000 liter terhadap kapasitas kendaraan pada dataset CVRPLIB. Faktor konversi tersebut dinyatakan sebagai berikut

$$\gamma = \frac{32000}{Q_{CVRPLIB}} \quad (32)$$

dengan γ adalah faktor konversi dan $Q_{CVRPLIB}$ adalah kapasitas kendaraan pada dataset CVRPLIB. Nilai permintaan hasil konversi kemudian dihitung menggunakan persamaan berikut

$$q'_i = \gamma \times q_i \quad (33)$$

dengan q'_i adalah permintaan pelanggan hasil konversi (liter) dan q_i adalah permintaan pelanggan pada dataset CVRPLIB.

Data hasil prapemrosesan ini, dataset CVRPLIB dapat digunakan sebagai dasar pembentukan skenario penelitian yang mempresentasikan distribusi BBM tanpa mengubah struktur dasar permasalahan CVRP.

2.4 Pembentukan Skenario Penelitian

Penelitian ini menggunakan tiga skenario untuk mengevaluasi kinerja algoritma *Clarke and Wright Savings* dan ACO pada permasalahan distribusi BBM dengan

karakteristik yang berbeda. Ketiga skenario dibentuk berdasarkan data hasil prapemrosesan sehingga memiliki koordinat pelanggan, matriks jarak, dan nilai permintaan yang sama. Perbedaan setiap skenario terletak pada karakteristik armada dan model distribusi yang digunakan.

1. Skenario 1

Skenario pertama dimodelkan sebagai CVRP dengan armada homogen. Pada skenario ini seluruh kendaraan memiliki kapasitas yang sama, yaitu 32.000 liter, sehingga setiap rute hanya dibatasi oleh kapasitas kendaraan tersebut.

2. Skenario 2

Skenario kedua juga menggunakan model CVRP, namun armada yang digunakan bersifat heterogen. Kendaraan memiliki beberapa variasi, yaitu 8.000 liter, 16.000 liter, 24.000 liter, dan 32.000 liter. Penggunaan armada heterogen bertujuan untuk mengevaluasi kemampuan algoritma dalam mengalokasikan kendaraan dengan kapasitas yang berbeda sesuai dengan kebutuhan distribusi.

3. Skenario 3

Skenario ketiga dimodelkan sebagai MCVRP. Pada skenario ini setiap kendaraan diasumsikan memiliki dua kompartemen dengan masing-masing 16.000 liter yang digunakan untuk mengangkut dua jenis BBM. Nilai permintaan pelanggan hasil konversi kemudian dibagi menjadi dua jenis BBM dengan tetap mempertahankan total permintaan setiap pelanggan. Dengan demikian, proses distribusi harus memenuhi kendala kapasitas masing-masing kompartemen kendaraan.

Ketiga skenario tersebut digunakan untuk membandingkan kinerja algoritma *Clarke and Wright Savings* dan ACO dalam menyelesaikan permasalahan distribusi dengan tingkat kompleksitas yang berbeda. Perbedaan karakteristik armada dan model distribusi pada setiap skenario diharapkan dapat memberikan gambaran mengenai kemampuan kedua algoritma dalam menghasilkan rute distribusi yang efisien.

2.5 Implementasi Algoritma Clarke and Wright Savings

2.5.1 Implementasi pada CVRP dengan Armada Homogen

Implementasi algoritma *Clarke and Wright Savings* pada skenario homogen menggunakan armada dengan kapasitas kendaraan yang sama. Setiap pelanggan pada awalnya diasumsikan membentuk satu rute yang menghubungkan depot dengan pelanggan dan kembali ke depot. Selanjutnya, dilakukan proses penggabungan rute distribusi berdasarkan urutan nilai *savings* dengan tetap memperhatikan kapasitas maksimum kendaraan. Setelah seluruh proses penggabungan selesai *running*, dihitung total jarak tempuh setiap rute dan total jarak distribusi yang diperoleh.

Pseudocode

Input: Matriks jarak D , demand pelanggan q , jumlah kendaraan m , kapasitas kendaraan Q

Output: Rute distribusi dan total jarak distribusi

Langkah-langkah:

1. Baca matriks jarak D .
 2. Inisialisasi jumlah kendaraan, kapasitas kendaraan, dan demand pelanggan.
 3. Hitung nilai saving untuk setiap pasangan pelanggan:

$$S_{ij} = d_{0i} + d_{0j} - d_{ij}.$$
 4. Urutkan seluruh nilai savings secara menurun.
 5. Bentuk rute awal:
 Setiap pelanggan membentuk rute
 Depot $\rightarrow$ Pelanggan $\rightarrow$ Depot.
 6. Untuk setiap pasangan pelanggan berdasarkan urutan saving:
 - a. Identifikasi rute yang memuat kedua pelanggan.
 - b. Jika kedua pelanggan berada pada rute yang berbeda, hitung total demand hasil penggabungan.
 - c. Jika total demand $\leq$ kapasitas kendaraan, gabungkan kedua rute sesuai posisi pelanggan (awal atau akhir rute).
 7. Ulangi langkah 6 hingga seluruh nilai *savings* selesai diproses.
 8. Hitung jarak setiap rute dan total jarak distribusi.
 9. Tampilkan rute hasil optimasi beserta visualisasi distribusi.
- Selesai.

2.5.2 Implementasi pada CVRP dengan Armada Heterogen

Implementasi algoritma *Clarke and Wright Savings* pada skenario heterogen menggunakan armada dengan kapasitas kendaraan yang berbeda. Proses implementasi diawali dengan pembentukan matriks *saving*, penggabungan rute berdasarkan nilai *saving* dengan memperhatikan kapasitas maksimum armada, kemudian dilanjutkan dengan proses alokasi kendaraan sesuai kapasitas yang tersedia.

Pseudocode

Input: Matriks jarak D , demand pelanggan q , kapasitas kendaraan heterogen Q , jumlah kendaraan m

Output: Rute distribusi, alokasi kendaraan, dan total jarak distribusi

Langkah-langkah:

1. Baca matriks jarak dan parameter kendaraan.
 2. Hitung nilai saving untuk setiap pasangan pelanggan.
 3. Urutkan nilai saving secara menurun.
 4. Bentuk rute awal untuk setiap pelanggan.
 5. Gabungkan rute berdasarkan urutan nilai saving dengan memperhatikan kapasitas maksimum armada.
 6. Hitung total muatan dan jarak setiap rute yang terbentuk.
 7. Urutkan rute berdasarkan total jarak secara menurun.
 8. Alokasikan kendaraan dengan memilih kapasitas terkecil yang masih mampu melayani setiap rute.
 9. Hitung total jarak distribusi dan jumlah kendaraan yang digunakan
 10. Tampilkan hasil optimasi dan visualisasi rute.
- Selesai.

2.5.3 Implementasi pada MCVRP dengan Dua Kompartemen

Implementasi algoritma *Clarke and Wright Savings* pada skenario MCVRP menggunakan kendaraan yang memiliki dua kompartemen dengan kapasitas masing-masing. Proses implementasi diawali dengan pembentukan rute menggunakan algoritma *Clarke and Wright Savings*, kemudian dilanjutkan dengan konsolidasi jumlah rute, proses *best-fit insertion* untuk pelanggan yang belum teralokasi, serta mekanisme *relocate* dan *swap repair* agar seluruh pelanggan dapat dilayani sesuai batas kapasitas setiap kompartemen.

Pseudocode

Input: Matriks jarak D , demand kompartemen 1 dan 2, kapasitas kompartemen Q_1 dan Q_2 , jumlah kendaraan m

Output: Rute distribusi dan total jarak distribusi

Langkah-langkah:

1. Baca matriks jarak dan parameter kendaraan.
2. Hitung nilai saving untuk setiap pasangan pelanggan.
3. Urutkan nilai saving secara menurun.
4. Bentuk rute awal untuk setiap pelanggan.
5. Gabungkan rute berdasarkan nilai saving dengan memperhatikan kapasitas kedua kompartemen.
6. Konsolidasikan jumlah rute agar sesuai dengan jumlah kendaraan yang tersedia.
7. Sisipkan pelanggan yang belum teralokasi ke dalam rute menggunakan metode *best-fit insertion*.
8. Jika masih terdapat pelanggan yang belum terlayani, lakukan proses *relocate* dan *swap repair*.

9. Hitung total jarak setiap rute dan total jarak distribusi.
 10. Tampilkan hasil optimasi dan visualisasi rute.
- Selesai.

2.6 Implementasi Algoritma Ant Colony Optimization (ACO)

2.6.1 Implementasi pada CVRP dengan Armada Homogen

Implementasi algoritma *Ant Colony Optimization* (ACO) pada skenario homogen menggunakan armada dengan kapasitas kendaraan yang sama. Proses implementasi diawali dengan inialisasi parameter ACO, pembentukan solusi oleh setiap semut berdasarkan kandidat pelanggan dan intensitas feromon, kemudian dilakukan perbaikan rute menggunakan metode *2-opt*. Selanjutnya feromon diperbarui melalui mekanisme penguapan dan penambahan feromon hingga diperoleh rute dengan total jarak minimum.

Pseudocode

Input: Matriks jarak D , demand pelanggan q , jumlah kendaraan m , kapasitas kendaraan Q , parameter ACO (α, β, ρ, Q , jumlah semut, iterasi)

Output: Rute distribusi terbaik dan total jarak minimum

Langkah-langkah:

1. Baca matriks jarak dan parameter kendaraan.
 2. Inialisasi parameter ACO, matriks feromon, dan informasi heuristik.
 3. Bentuk *candidate* list untuk setiap node.
 4. Tentukan nilai feromon awal menggunakan solusi awal *Nearest Neighbor*.
 5. Untuk setiap iterasi:
 - a. Bentuk rute setiap semut berdasarkan probabilitas transisi.
 - b. Perbaiki rute menggunakan metode *2-opt*.
 - c. Hitung total jarak setiap solusi.
 - d. Perbarui solusi terbaik.
 - e. Sesuaikan laju penguapan feromon secara adaptif.
 - f. Lakukan pembaruan feromon menggunakan mekanisme MMAS dan *Elitist Ant System*.
 6. Hitung total jarak solusi terbaik.
 7. Tampilkan hasil optimasi dan visualisasi rute.
- Selesai.

2.6.2 Implementasi pada CVRP dengan Armada Heterogen

Implementasi algoritma *Ant Colony Optimization* pada skenario heterogen menggunakan armada dengan kapasitas kendaraan yang berbeda. Proses implementasi diawali dengan inialisasi parameter ACO, pembentukan solusi oleh setiap semut dengan mempertimbangkan kapasitas kendaraan heterogen,

dilanjutkan dengan perbaikan rute menggunakan metode *2-opt*, serta pembaruan feromon hingga diperoleh rute distribusi dengan total jarak minimum.

Pseudocode

Input: Matriks jarak D , demand pelanggan q , kapasitas kendaraan heterogen Q , parameter ACO (α, β, ρ, Q , jumlah semut, iterasi)

Output: Rute distribusi terbaik, alokasi kendaraan dan total jarak minimum

Langkah-langkah:

1. Baca matriks jarak, demand pelanggan, dan data armada heterogen.
2. Inisialisasi parameter ACO, matriks feromon, informasi heuristik, dan *candidate list*.
3. Tentukan nilai feromon awal menggunakan solusi *Nearest Neighbor*.
4. Untuk setiap iterasi:
 - a. Bentuk solusi setiap semut dengan memperhatikan kapasitas kendaraan heterogen.
 - b. Perbaiki setiap rute menggunakan metode *2-opt*.
 - c. Hitung total jarak setiap solusi.
 - d. Perbarui solusi terbaik.
 - e. Sesuaikan laju penguapan feromon secara adaptif.
 - f. Perbarui feromon menggunakan mekanisme *Max-Min Ant Sistem* dan *Elitist Ant System*.
5. Hitung total jarak minimum dan kendaraan yang digunakan
6. Tampilkan hasil optimasi dan visualisasi rute.

Selesai.

2.6.3 Implementasi pada MCVRP dengan Dua Kompartemen

Implementasi algoritma *Ant Colony Optimization* pada skenario *Multi-Compartment Vehicle Routing Problem* (MCVRP) menggunakan kendaraan yang memiliki dua kompartemen dengan kapasitas masing-masing. Proses implementasi diawali dengan inisialisasi parameter ACO, kemudian setiap semut membentuk rute dengan mempertimbangkan kapasitas kedua kompartemen secara bersamaan. Selanjutnya dilakukan evaluasi solusi dan pembaruan feromon hingga diperoleh rute distribusi dengan total jarak minimum.

Pseudocode

Input: Matriks jarak D , demand kompartemen 1 (q_1), demand kompartemen 2 (q_2), jumlah kendaraan m , kapasitas kompartemen Q_1 dan Q_2 , parameter ACO

Output: Rute distribusi terbaik dan total jarak minimum

Langkah-langkah:

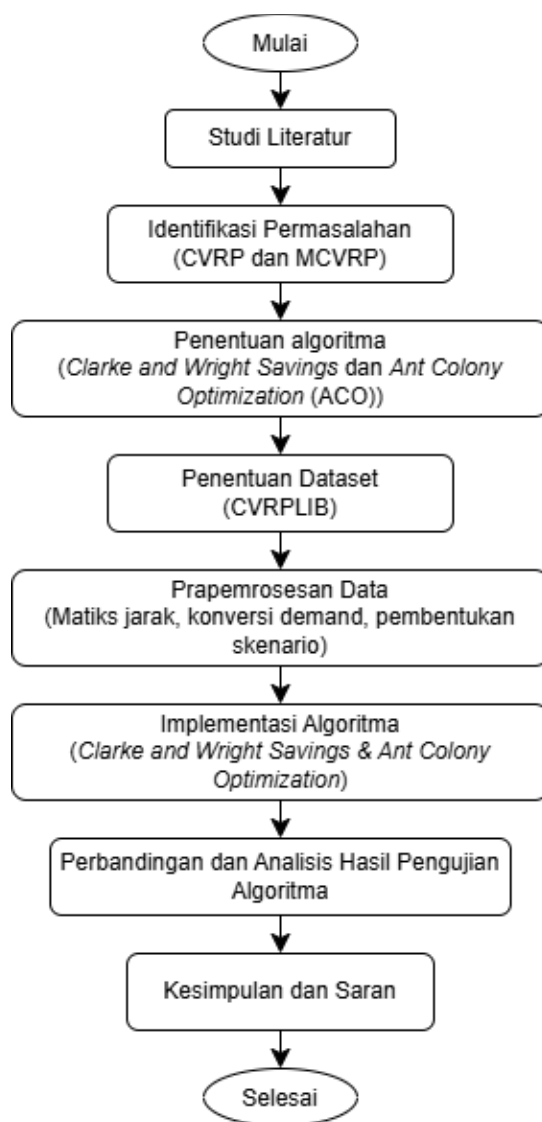
1. Baca matriks jarak, demand setiap kompartemen, dan parameter kendaraan.
 2. Inisialisasi parameter ACO.
 3. Untuk setiap iterasi:
 - a. Bentuk rute setiap semut dengan memperhatikan kapasitas kedua kompartemen.
 - b. Hitung total jarak setiap solusi.
 - c. Perbarui solusi terbaik.
 - d. Lakukan penguapan dan penambahan feromon.
 4. Hitung total jarak minimum dan kendaraan yang digunakan.
 5. Tampilkan hasil optimasi dan visualisasi rute.
- Selesai.

2.7 Prosedur Penelitian

Adapun prosedur penelitian ini yaitu:

1. Melakukan studi literatur untuk memahami *Capacitated Vehicle Routing Problem* (CVRP), *Multi-Compartment Vehicle Routing Problem* (MCVRP), *Clarke and Wright Savings*, dan *Ant Colony Optimization* (ACO).
2. Menetapkan permasalahan yang akan dijadikan fokus utama dalam penelitian ini yaitu *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP).
3. Menentukan algoritma yang digunakan yaitu algoritma *Clarke and Wright Savings*, dan *Ant Colony Optimization* (ACO).
4. Mengumpulkan data sekunder berupa dataset benchmark CVRPLIB yang meliputi koordinat depot dan pelanggan, *demand* pelanggan, jumlah kendaraan, dan kapasitas kendaraan.
5. Melakukan prapemrosesan data yang meliputi perhitungan matriks jarak, konversi *demand* ke satuan liter, serta pembentukan tiga skenario penelitian.
6. Mengimplementasikan algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* (ACO) pada setiap skenario penelitian untuk memperoleh solusi.
7. Membandingkan dan menganalisis hasil yang diperoleh dari algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* (ACO) berdasarkan indikator kinerja yang telah ditentukan.
8. Menyusun kesimpulan berdasarkan hasil pengimplementasian kedua algoritma dan memberikan saran untuk pengembangan lebih lanjut.

2.8 Diagram Alir Penelitian



Gambar 7. Diagram alir penelitian

BAB III

HASIL DAN PEMBAHASAN

3.1 Hasil

3.1.1 Deskripsi Data

Data yang digunakan pada penelitian ini merupakan data sekunder yang diperoleh dari *Capacitated Vehicle Routing Problem Library* (CVRPLIB). Data tersebut meliputi koordinat depot dan pelanggan, *demand* setiap pelanggan, kapasitas kendaraan, serta jumlah kendaraan yang digunakan. Penelitian ini menggunakan tiga dataset benchmark dengan ukuran yang berbeda, yaitu dataset berukuran kecil, sedang, dan besar. Dataset tersebut masing-masing terdiri atas 31 node, 43 node, dan 78 node, dengan satu titik depot serta 5, 6, dan 10 kendaraan. Penggunaan tiga ukuran dataset tersebut bertujuan untuk mengevaluasi kinerja algoritma pada tingkat kompleksitas permasalahan yang berbeda.

Selain data penelitian, parameter algoritma *Ant Colony Optimization* (ACO) juga ditentukan berdasarkan penelitian terdahulu yang telah menunjukkan kinerja parameter yang baik dalam menyelesaikan permasalahan VRP. Habibah et al. (2021) menggunakan parameter pengaruh feromon (α) bernilai 1, parameter pengaruh visibilitas (β) bernilai 3, laju evaporasi feromon (ρ) bernilai 0,1, dan konstanta feromon (Q) bernilai 5. Sementara itu, Siantu et al. (2025) menggunakan parameter pengaruh feromon (α) bernilai 0,25, parameter pengaruh visibilitas (β) nilai 2, laju evaporasi feromon (ρ) bernilai 0,3 dan konstanta feromon (Q) bernilai 6 sampai 7. Kombinasi parameter tersebut memperoleh solusi terbaik dalam algoritma *Ant Colony Optimization* (ACO).

Berdasarkan hasil penelitian terdahulu tersebut, parameter ACO yang digunakan dalam penelitian ini ditentukan dengan mengacu pada rentang nilai yang telah digunakan sebelumnya dan disesuaikan melalui beberapa percobaan awal (*trial and error*). Percobaan tersebut dilakukan untuk memperoleh kombinasi parameter yang mampu menghasilkan total jarak tempuh yang lebih kecil dan hasil relatif stabil pada kasus distribusi BBM yang diteliti. Selain itu, penelitian ini menggunakan 100 iterasi dengan 10 kali *running* untuk mengurangi pengaruh sifat stokastik ACO dan memperoleh solusi yang lebih representatif.

Hasil dari *running* tersebut kemudian akan dirata-ratakan untuk mendapatkan hasil akhir. Adapun beberapa parameter yang digunakan pada algoritma *Ant Colony Optimization* (ACO) dapat dilihat pada Tabel 7.

Tabel 7. Parameter *Ant Colony Optimization* (ACO)

Jumlah Semut	α	β	ρ	Q
30 dan 80	0,25 dan 1	2 dan 3	0,1 dan 0,3	5 – 7

3.1.2 Evaluasi Gap terhadap Best Known Solution (BKS)

Untuk mengevaluasi kualitas solusi dari sisi jarak, hasil metode dibandingkan dengan nilai *Best Known Solution* (BKS) pada CVRPLIB dengan menggunakan persamaan (34).

$$Gap = \frac{Z_{hasil} - Z_{BKS}}{Z_{BKS}} \times 100\% \quad (34)$$

dengan:

Z_{hasil} = total jarak hasil CWS atau ACO

Z_{BKS} = total jarak terbaik pada CVRPLIB

Tabel 8. Hasil validasi implementasi algoritma

Dataset	BKS	CWS	ACO (Mean)	S.Deviasi	GAP CWS (%)	GAP ACO (%)
Kecil	672	681,17	724,00	5,4417	1,36%	7,74%
Sedang	742	781,97	788,58	17,7899	5,39%	6,28%
Besar	1221	1264,99	1352,18	14,6399	3,60%	10,74%

Berdasarkan hasil dari Tabel 8, implementasi algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* mampu menghasilkan solusi yang mendekati nilai benchmark sehingga implementasi algoritma dinilai telah berjalan dengan baik. Oleh karena itu, kedua algoritma selanjutnya diterapkan pada data yang telah dikonversi ke dalam satuan liter dan digunakan pada tiga skenario penelitian.

3.2 Pembahasan

Pada penelitian ini dilakukan analisis komparatif terhadap algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* (ACO) untuk menyelesaikan permasalahan optimasi rute distribusi BBM menggunakan model *Capacitated Vehicle Routing Problem* (CVRP) dan *Multi-Compartment Vehicle Routing Problem* (MCVRP). Kedua algoritma diterapkan pada dataset benchmark CVRPLIB yang telah melalui tahap prapemrosesan sehingga sesuai dengan karakteristik distribusi BBM. Analisis dilakukan untuk mengevaluasi kemampuan masing-masing algoritma dalam menghasilkan rute distribusi dengan total jarak tempuh minimum pada setiap skenario penelitian.

Secara umum, algoritma ACO membutuhkan waktu komputasi yang lebih kompleks dibandingkan algoritma *Clarke and Wright Savings* karena membangun solusi secara iteratif melalui mekanisme probabilistik berdasarkan intensitas feromon dan informasi heuristik. Sebaliknya, *Clarke and Wright Savings* merupakan algoritma heuristik konstruktif yang membentuk solusi berdasarkan nilai *savings* sehingga proses pencarian solusi relatif lebih sederhana. Perbedaan mekanisme tersebut menyebabkan kedua algoritma memiliki karakteristik pencarian solusi yang berbeda.

Untuk mendapatkan pemahaman yang lebih baik tentang kinerja kedua algoritma, dilakukan evaluasi terhadap proses pembentukan rute distribusi berdasarkan matriks jarak antar lokasi serta data permintaan pelanggan pada dataset benchmark yang telah dikonversi ke dalam satuan liter. Evaluasi dilakukan pada tiga skenario penelitian, yaitu CVRP dengan kapasitas kendaraan homogen, CVRP dengan kapasitas kendaraan heterogen, dan MCVRP menggunakan kendaraan dua kompartemen.

Pada algoritma *Clarke and Wright Savings*, parameter utama yang digunakan meliputi matriks jarak, data permintaan (*demand*) setiap pelanggan, kapasitas kendaraan serta jumlah kendaraan yang telah ditetapkan. Hasil yang diperoleh kemudian dievaluasi berdasarkan total jarak tempuh yang dihasilkan. Hasil pengujian algoritma *Clarke and Wright Savings* telah disajikan dalam Tabel 9.

Tabel 9. Hasil Algoritma *Clarke and Wright Savings* Skenario 1

Dataset	Jumlah Node	Jumlah kendaraan	Total Jarak (km)
Kecil	31	5	681,17
Sedang	43	6	781,97
Besar	78	10	1264,99

Berdasarkan Tabel 9, algoritma *Clarke and Wright Savings* pada skenario CVRP dengan armada homogen mampu menghasilkan solusi untuk seluruh ukuran dataset. Total jarak tempuh meningkat dari 681,17 km pada dataset kecil menjadi 781,97 km pada dataset sedang dan 1264,99 km pada dataset besar. Peningkatan tersebut sejalan dengan bertambahnya jumlah pelanggan yang harus dilayani sehingga rute distribusi yang terbentuk menjadi lebih panjang.

Tabel 10. Hasil Algoritma *Clarke and Wright Savings* Skenario 2

Dataset	Jumlah Node	Jumlah kendaraan	Total Jarak (km)
Kecil	31	5	681,17
Sedang	43	6	781,97
Besar	78	10	1264,99

Berdasarkan Tabel 10, algoritma *Clarke and Wright Savings* pada CVRP dengan armada heterogen menghasilkan total jarak tempuh yang sama dengan skenario CVRP dengan armada homogen untuk seluruh ukuran dataset, yaitu sebesar 681,17 km untuk dataset kecil, 781,97 km untuk dataset sedang, dan 1264,99 km untuk dataset besar. Hasil tersebut menunjukkan bahwa penggunaan armada dengan kapasitas yang berbeda tidak memengaruhi proses pembentukan rute pada algoritma *Clarke and Wright Savings*. Hal ini disebabkan karena pembentukan rute terlebih dahulu dilakukan berdasarkan nilai *savings* dengan memperhatikan total permintaan pelanggan, sedangkan pemilihan kapasitas

kendaraan baru dilakukan setelah rute terbentuk melalui alokasi kendaraan yang sesuai. Oleh karena itu, meskipun kapasitas kendaraan bervariasi, susunan rute dan total jarak tempuh yang dihasilkan tetap sama dengan skenario armada homogen.

Tabel 11. Hasil Algoritma *Clarke and Wright Savings* Skenario 3

Dataset	Jumlah Node	Jumlah kendaraan	Total Jarak (km)
Kecil	31	5	689,57
Sedang	43	6	808,09
Besar	78	10	1575,14

Berdasarkan Tabel 11, algoritma *Clarke and Wright Savings* berhasil menghasilkan solusi rute pada skenario MCVRP untuk seluruh ukuran dataset yang diuji. Pada dataset kecil diperoleh total jarak sebesar 689,57 km. Pada dataset sedang diperoleh total jarak sebesar 808,09 km, sedangkan pada dataset besar diperoleh total jarak sebesar 1575,14 km. pada skenario ini pembentukan rute tidak hanya dipengaruhi oleh kapasitas kendaraan, tetapi juga oleh kapasitas masing-masing kompartemen sehingga proses penggabungan pelanggan menjadi lebih terbatas dibandingkan kedua skenario sebelumnya.

Berdasarkan ketiga skenario yang telah diuji, terlihat bahwa karakteristik model distribusi memengaruhi hasil yang diperoleh algoritma *Clarke and Wright Savings*. Pada skenario homogen, proses penggabungan rute hanya dibatasi oleh satu kapasitas kendaraan sehingga algoritma memiliki fleksibilitas yang lebih besar dalam membentuk rute. Pada skenario heterogen, proses pembentukan rute tetap menggunakan mekanisme *savings*, namun kendaraan dialokasikan sesuai kapasitas yang paling sesuai sehingga pemanfaatan armada menjadi lebih efisien. Sementara itu, pada skenario MCVRP pembentukan rute menjadi lebih kompleks karena selain mempertimbangkan kapasitas kendaraan, algoritma juga harus memenuhi kapasitas masing-masing kompartemen. Kendala tambahan tersebut membatasi kemungkinan penggabungan pelanggan dalam satu rute sehingga memengaruhi total jarak tempuh yang dihasilkan.

Setelah diperoleh hasil penerapan algoritma *Clarke and Wright Savings* pada ketiga skenario penelitian, selanjutnya dilakukan pengujian kombinasi parameter pada algoritma ACO. Pengujian ini bertujuan untuk menentukan kombinasi parameter yang menghasilkan total jarak tempuh paling kecil pada setiap skenario penelitian. Parameter yang diuji meliputi pengaruh feromon (α) sebesar 0,25 dan 1,0, pengaruh visibilitas (β) sebesar 2 dan 3, laju evaporasi feromon (ρ) sebesar 0,1 dan 0,3, dan konstanta feromon (Q) sebesar 5, 6, dan 7. Setiap kombinasi parameter dijalankan sebanyak 10 kali (*running*) untuk memperoleh nilai rata-rata total jarak tempuh. Kombinasi parameter yang menghasilkan rata-rata total jarak tempuh terkecil kemudian digunakan pada pengujian utama. Hasil parameter yang

menghasilkan rata-rata total jarak tempuh terkecil pada setiap skenario disajikan pada Tabel 12.

Tabel 12. Hasil pengujian parameter algoritma ACO

Skenario	Dataset	Q	α	β	ρ	Mean Total Jarak (km)
CVRP Homogen	Kecil	5	0,25	3	0,3	724,00
	Sedang	7	1	2	0,3	788,58
	Besar	6	1	3	0,3	1352,18
CVRP Heterogen	Kecil	5	0,25	3	0,3	688,32
	Sedang	7	1	2	0,3	788,88
	Besar	6	1	2	0,3	1351,26
MCVRP	Kecil	6	1	3	0,1	705,20
	Sedang	7	1	3	0,3	794,52
	Besar	6	1	3	0,1	1589,33

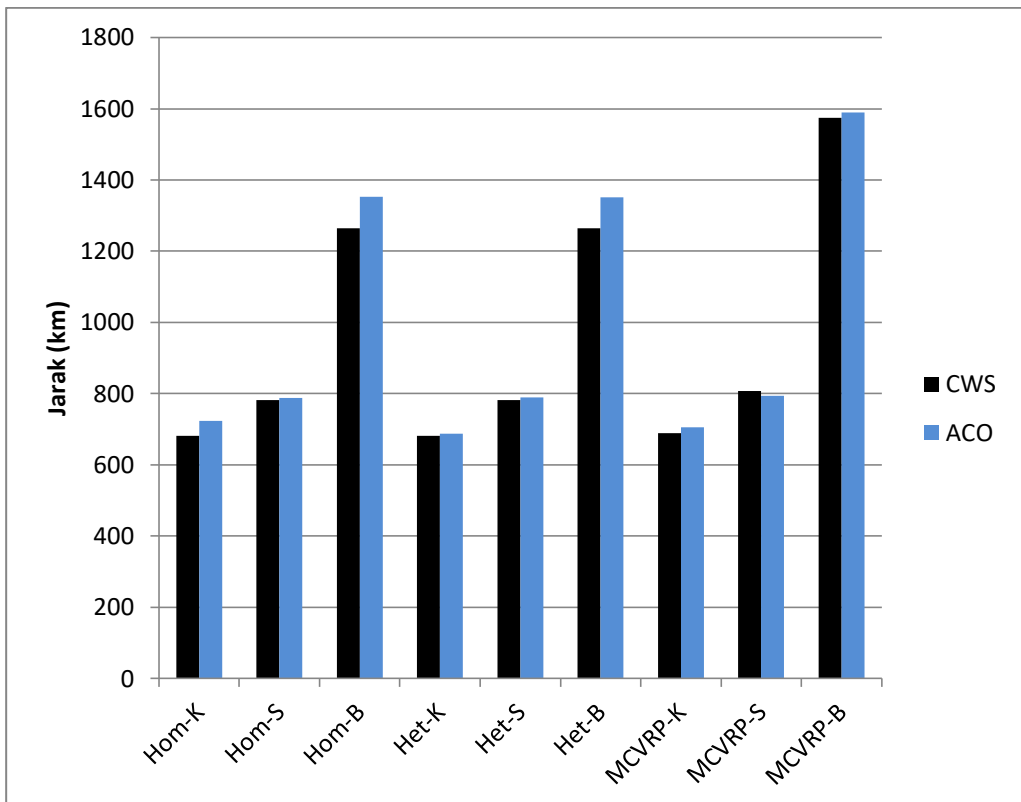
Berdasarkan Tabel 12, setiap skenario dan ukuran dataset menghasilkan kombinasi parameter ACO yang berbeda. Pada skenario CVRP homogen diperoleh kombinasi parameter yang menghasilkan rata-rata total jarak tempuh sebesar 724 km, 788,58 km, dan 1352,18 km untuk dataset kecil, sedang, dan besar. Pada skenario CVRP heterogen diperoleh rata-rata total jarak tempuh sebesar 688,32 km, 788,88 km, dan 1351,26 km. Sementara itu, pada skenario MCVRP diperoleh rata-rata total jarak tempuh sebesar 705,20 km, 794,52 km, dan 1589,33 km untuk dataset kecil, sedang, dan besar. Perbedaan kombinasi parameter pada setiap skenario menunjukkan bahwa karakteristik model distribusi dan ukuran dataset memengaruhi hasil yang diperoleh algoritma ACO. Oleh karena itu, kombinasi parameter yang menghasilkan rata-rata total jarak tempuh paling kecil pada masing-masing skenario digunakan pada tahap pengujian selanjutnya.

Tabel 13. Ringkasan statistik hasil pengujian algoritma ACO

Skenario	Dataset	Mean Total Jarak (km)	Standar Deviasi	CV (%)
CVRP Homogen	Kecil	724,00	5,4417	0,75
	Sedang	788,58	17,7899	2,26
	Besar	1352,18	14,6399	1,08
CVRP Heterogen	Kecil	688,32	4,8642	0,71
	Sedang	788,88	10,4250	1,32
	Besar	1351,26	20,4954	1,52
MCVRP	Kecil	705,20	4,2842	0,61
	Sedang	794,52	5,5091	0,69
	Besar	1589,33	25,8048	1,62

Berdasarkan tabel 13, hasil pengujian ACO pada seluruh skenario menunjukkan adanya variasi total jarak tempuh antar *running*, yang ditunjukkan oleh nilai standar deviasi. Namun, nilai koefisien variasi (CV) pada seluruh pengujian berada pada

rentang yang relatif kecil, yaitu antara 0,61% hingga 2,26%. Hal tersebut menunjukkan bahwa variasi hasil terhadap nilai rata-rata total jarak tempuh tidak terlalu besar sehingga hasil yang diperoleh dari 10 kali *running* cenderung konsisten. Dengan demikian, nilai rata-rata total jarak tempuh pada setiap skenario dapat digunakan sebagai representasi hasil pengujian ACO untuk tahap perbandingan dengan algoritma *Clarke and Wright Savings*. Setelah diperoleh hasil pengujian dari kedua algoritma, selanjutnya dilakukan perbandingan kinerja kedua algoritma berdasarkan total jarak tempuh yang dihasilkan pada setiap skenario penelitian yang dapat dilihat pada Gambar 8.



Gambar 8. Perbandingan total jarak tempuh kedua algoritma

Berdasarkan Gambar 8, terdapat perbedaan total jarak tempuh yang dihasilkan oleh algoritma *Clarke and Wright Savings* dan ACO pada setiap skenario penelitian. Pada skenario CVRP homogen dan CVRP heterogen, algoritma *Clarke and Wright Savings* menghasilkan total jarak tempuh yang lebih kecil dibandingkan algoritma ACO pada seluruh ukuran dataset. Sementara itu, pada skenario MCVRP, algoritma ACO menghasilkan total jarak tempuh yang lebih kecil pada dataset sedang, sedangkan pada dataset kecil dan besar, algoritma *Clarke and Wright Savings* menghasilkan total jarak tempuh yang lebih kecil. Perbedaan hasil tersebut menunjukkan bahwa kedua algoritma memiliki karakteristik pembentukan

rute yang berbeda. *Clarke and Wright Savings* membentuk rute secara langsung berdasarkan nilai *savings*, sedangkan ACO membangun solusi melalui proses pencarian berbasis feromon yang bersifat stokastik. Akibatnya, total jarak tempuh yang dihasilkan kedua algoritma dapat berbeda meskipun diterapkan pada data dan skenario yang sama.

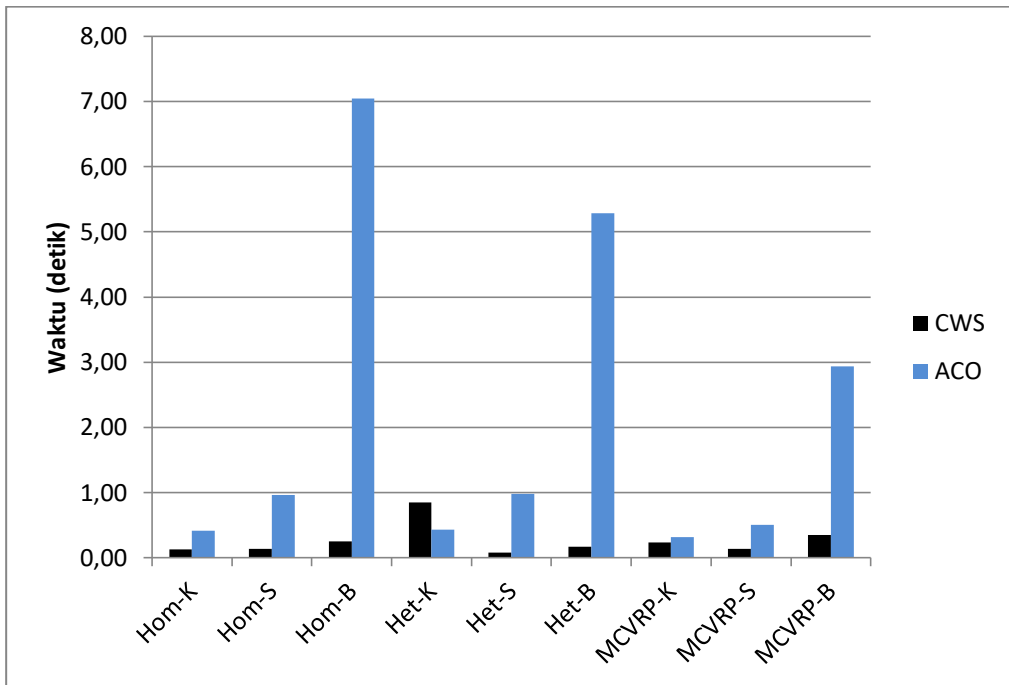
Untuk mengetahui besarnya perbedaan kinerja antara *Clarke and Wright Savings* dan ACO, dilakukan perhitungan persentase perubahan berdasarkan total jarak tempuh yang dihasilkan oleh masing-masing algoritma. Hasil perhitungan tersebut disajikan pada Tabel 14.

Tabel 14. Persentase perubahan total jarak tempuh

Skenario	Dataset	CWS (km)	ACO (km)	Perubahan (%)
CVRP Homogen	Kecil	681,17	724,00	-6,29
	Sedang	781,97	788,58	-0,85
	Besar	1264,99	1352,18	-6,89
CVRP Heterogen	Kecil	681,17	688,32	-1,05
	Sedang	781,97	788,88	-0,88
	Besar	1264,99	1351,26	-6,82
MCVRP	Kecil	689,57	705,20	-2,27
	Sedang	808,09	794,52	1,68
	Besar	1575,14	1589,33	-0,90

Berdasarkan Tabel 14, persentase perubahan total jarak menunjukkan bahwa algoritma ACO menghasilkan total jarak tempuh yang lebih besar dibandingkan algoritma *Clarke and Wright Savings* pada sebagian besar skenario penelitian. Hal tersebut ditunjukkan oleh nilai persentase perubahan yang bernilai negatif pada sebagian besar kasus pengujian. Sementara itu, nilai persentase positif hanya diperoleh pada skenario MCVRP dengan dataset sedang, yang menunjukkan bahwa ACO menghasilkan total jarak tempuh yang lebih rendah dibandingkan *Clarke and Wright Savings* pada skenario tersebut.

Perbandingan waktu komputasi dari algoritma *Clarke and Wright Savings* dan ACO pada CVRP dan MCVRP dapat dilihat pada Gambar 9.



Gambar 9. Perbandingan waktu komputasi kedua algoritma

Berdasarkan Gambar 9, algoritma *Clarke and Wright Savings* menghasilkan waktu komputasi yang singkat dibandingkan ACO pada sebagian besar skenario penelitian. Perbedaan waktu komputasi semakin terlihat pada dataset berukuran besar, di mana ACO memerlukan waktu komputasi yang lebih lama dibandingkan *Clarke and Wright Savings*. Hal ini disebabkan karena *Clarke and Wright Savings* merupakan algoritma konstruktif yang membentuk solusi berdasarkan nilai *savings* dan penggabungan rute, sedangkan ACO melakukan pencarian solusi secara iteratif dengan melibatkan sejumlah semut, pembaruan feromon, serta evaluasi solusi pada setiap iterasi. Akibatnya, semakin besar ukuran dataset, semakin banyak solusi yang harus dieksplorasi oleh ACO sehingga waktu komputasinya meningkat lebih signifikan dibandingkan *Clarke and Wright Savings*.

Perbedaan waktu komputasi tersebut sejalan dengan kompleksitas kedua algoritma. Secara teoritis, *Clarke and Wright Savings* memiliki kompleksitas yang mendekati $O(n^2)$ karena melibatkan perhitungan nilai *savings* untuk setiap pasangan pelanggan. Sementara itu, kompleksitas ACO dipengaruhi oleh jumlah semut dan iterasi yang digunakan, yaitu $O(m \times n \times iter)$, dengan m menyatakan jumlah semut, n jumlah pelanggan, dan $iter$ jumlah iterasi. Oleh karena itu, waktu komputasi ACO cenderung meningkat seiring bertambahnya ukuran dan kompleksitas permasalahan, sebagaimana terlihat pada hasil pengujian dataset berukuran besar.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil pengujian dan analisis yang telah dilakukan terhadap penerapan algoritma *Clarke and Wright Savings* dan algoritma *Ant Colony Optimization* (ACO) pada permasalahan optimasi rute distribusi BBM, maka dapat disimpulkan beberapa hal sebagai berikut:

1. Algoritma *Clarke and Wright Savings* dan *Ant Colony Optimization* (ACO) berhasil diimplementasikan untuk menyelesaikan permasalahan optimasi rute distribusi menggunakan model CVRP dan MCVRP pada tiga skenario penelitian. Kedua algoritma mampu menghasilkan solusi yang memenuhi kendala kapasitas kendaraan pada seluruh dataset yang digunakan.
2. Berdasarkan hasil perbandingan, algoritma *Clarke and Wright Savings* menghasilkan total jarak tempuh yang lebih rendah pada sebagian besar kasus pengujian serta umumnya memerlukan waktu komputasi yang lebih singkat dibandingkan algoritma ACO. Dengan demikian, pada dataset benchmark CVRPLIB yang digunakan dalam penelitian ini, algoritma *Clarke and Wright Savings* menghasilkan total jarak tempuh yang lebih rendah dengan waktu komputasi yang relative lebih singkat dibandingkan algoritma ACO.

4.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, beberapa saran yang dapat diberikan sebagai berikut:

1. Pengujian algoritma *Ant Colony Optimization* (ACO) dapat dilakukan dengan variasi parameter, strategi pembaruan feromon, atau jumlah iterasi yang lebih beragam sehingga pengaruh parameter terhadap kualitas solusi dapat dianalisis secara lebih mendalam.
2. Perbandingan dapat diperluas dengan melibatkan algoritma optimasi lainnya, seperti *Genetic Algorithm*, *Particle Swarm Optimization*, atau *Tabu Search*, sehingga diperoleh gambaran yang lebih komprehensif mengenai kinerja berbagai algoritma dalam menyelesaikan permasalahan CVRP maupun MCVRP.
3. Penggunaan dataset dengan karakteristik yang lebih beragam atau data distribusi aktual dapat dipertimbangkan untuk mengevaluasi performa algoritma pada kondisi yang lebih mendekati permasalahan nyata.

DAFTAR PUSTAKA

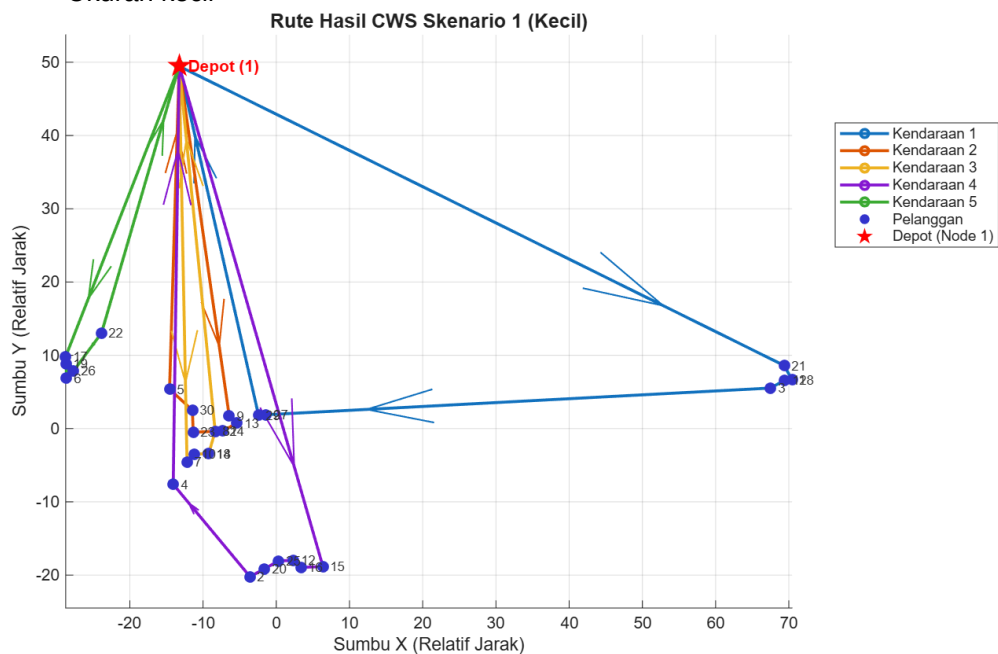
- Anggraeni, M. S., et al. (2023). Optimization of Fuel Distribution Routes for Green Logistics in Multi Compartment Vehicle Routing Problem (MCVRP) using Branch and Bound Algorithm (Case Study: Boyolali Fuel Terminal). *IEOM Society International*. 1132-1144. <https://doi.org/10.46254/AN13.20230333>
- Anshoriy, W. F. (2023). *Penerapan Metode Algoritma Clarke and Wright Savings pada Penentuan Rute Terpendek (Studi Kasus di Kantor Pos Pemeriksa Kabupaten Blitar)*. Universitas Islam Negeri Maulana Malik Ibrahim, Malang.
- Bouleft, Y., dan Alaoui, A. E. (2023). Dynamic Multi-Compartment Vehicle Routing Problem for Smart Waste Collection. *Journal of Appllied System Innovation*. 6(1). 30. <https://doi.org/10.3390/asi6010030>
- Ending, S. & Juniwati, I. (2022). *Ant Colony Optimization* untuk Menyelesaikan Perutean Distribusi Snack dengan *Vehicle Routing Problem*. *Jurnal Teknologi Informasi dan Terapan*. 9(2). 2580-2291. <https://doi.org/10/25047/jtit.v9i2.296>
- Fadhilah, M. R. (2019). *Optimalisasi Rute Distribusi Ayam Potong Menggunakan Algoritma Clarke and Wright Savings dengan Prosedur Pengurutan (Studi Kasus: Miami Chicken, Malang)*. Universitas Brawijaya, Malang.
- Febriandini, I. F., Yuniaristanto, & Sutopo, W. (2020). Multi-Compartment Vehicle Routing Problem to Find the Alternative Distribution Route of Petroleum Product Deliver. *Materials Science and Engineering*. <https://doi.org/10.1088/1757-899X/943/1/012039>
- Gustova, F. (2025). Kasus Praktik Pengoplosan BBM oleh PT. Pertamina Patra Niaga. Universitas Sebelas Maret.
- Hulu, N., et al. (2026). Efisiensi Distribusi Melalui Biaya dan Jarak dengan Pendekatan Model Transportasi. *Jurnal Ekonomi, Bisnis dan Manajemen*. 1(1). 1-5. <https://doi.org/10.56495/econobis.v1i1.937>
- Karjono, Moedjiono, & Kurniawan, D. (2016). Ant Colony Optimization. *Jurnal TICOM*. 4(3). 119-125. <https://media.neliti.com>
- Kristina, S., Sianturi, R. D., & Husnadi, R. (2020). Penerapan Model *Capacitated Vehicle Routing Problem* (CVRP) Menggunakan *Google OR-Tools* untuk Penentuan Rute Pengantaran Obat pada Perusahaan Pedagang Besar Farmasi (PBF). *Jurnal Telematika*. 15(2). 2579-3772. <https://doi.org/10.61769/telematika.v15i2.359>

- Kusuma, A.S & Sumiati. (2020). Penerapan Metode *Clarke and Wright Savings Heuristic* dalam Menentukan Rute Pendistribusian Produk di Bagian Distributor Koperasi ABC Bojonegoro. *Jurnal Manajemen Industri dan Teknologi*. 01(04), 1-11. <http://juminten.upnjatim.ac.id/index.php/juminten>
- Maulana, S. H. (2026). Optimasi dan Analisis Distribusi BBM Menggunakan Metode *Savings Matrix*, *Nearest Insert*, dan *Nearest Neighbor* pada *Multi Compartment Vehicle Routing Problem* (MCVRP). *Jurnal Ilmiah Research Student*. 3(1). 484-500. <https://doi.org/10.61722/jirs.v3i1.8695>
- Ostermeier, M., Hengke, T., Hubner, A., Wascher, G. (2021). Multi-Compartment Vehicle Routing Problems: State-of-the-art, Modeling Framework and Future Directions. *Jurnal Riset Operasional Eropa*. 292(3). 799-817. <https://doi.org/10.1016/j.ejor.2020.11.009>
- Patmawati, H. & Nugroho, Y. A. (2022). Optimasi Rute Distribusi Matras pada Penyelesaian *Capacitated Vehicle Routing Problem* dengan Metode Algoritma Genetika. *Jurnal Cakrawala Ilmiah*. 1(11). 2745-2756. <http://bajangjournal.com/index.php/JCI>
- Pratama, R. A., Utomo. P. H., & Wibowo, S. (2022). Perbandingan Solusi CVRP pada Distribusi Buku Aqila di Surakarta Menggunakan Algoritme Tabu Search dan Algoritme ACO. *Jurnal Riset dan Aplikasi Matematika*. 6(1). 13-22. <http://journal.unesa.ac.id/index.php/jram>
- Soimun, A., Putrisari, N.M. M. & Sadri, P. D. A. (2025). Optimization of the *Capacitated Vehicle Routing Problem* (CVRP) and Distribution Costs in a Drinking Water Company. *Jurnal Penelitian Sekolah Tinggi Transportasi Darat*. 16(1). 80-96. <https://doi.org/10.55511/jpsttd.v16i1.722>
- Suliantoro, H., Susanty, A., dan F. B. Silaban. (2016). Perancangan Rute Distribusi Beras Sejahtera Menggunakan Algoritma Ant Colony Optimization (Studi Kasus di BULOG Kabupaten Semarang). *Jurnal Teknik Industri*. 5(1). 1-7. <https://ejournal3.undip.ac.id>
- Vania, F. R. & Rochmoeljati, R. (2025). Optimasi Penentuan Rute Distribusi Beras Nasi Uduk Menggunakan Metode Ant Colony Optimization. *Jurnal Fakultas Teknik Universitas Islam Lamongan*. 17(2), 2085-0859. <http://www.jurnalteknik.unisla.ac.id/index.php/teknika/index>
- Zahra, S. (2021). *Optimasi Vehicle Routing Problem (VRP) dengan Menggunakan Metode Nearest Neighbour pada Pendistribusian Paku di PT. Putra Bandar Wiretama*. Universitas Medan.

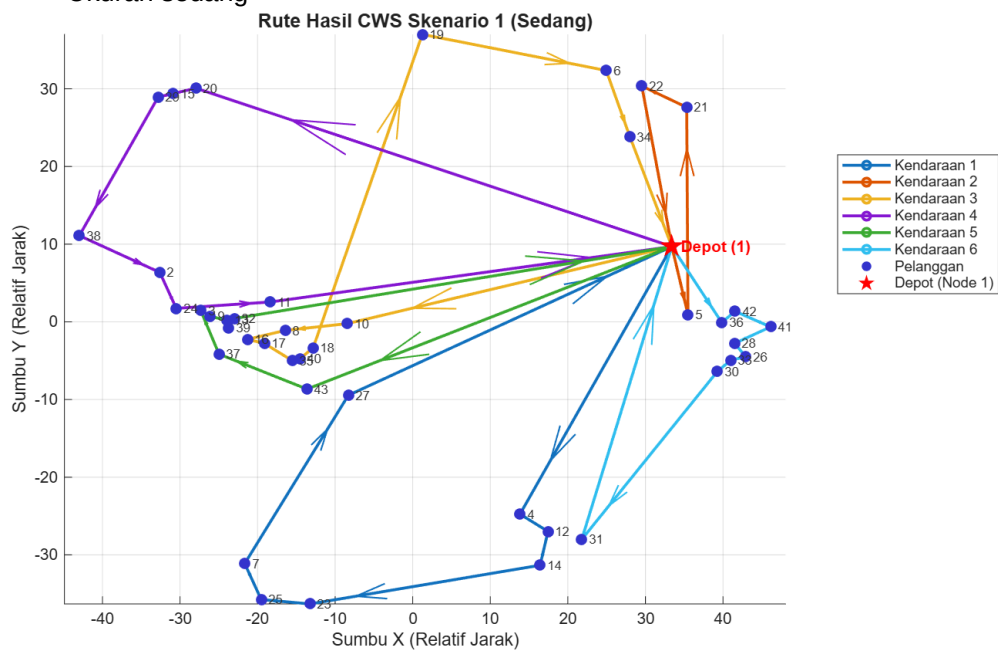
LAMPIRAN

Lampiran 1. Rute hasil *Clarke and Wright Savings* skenario 1

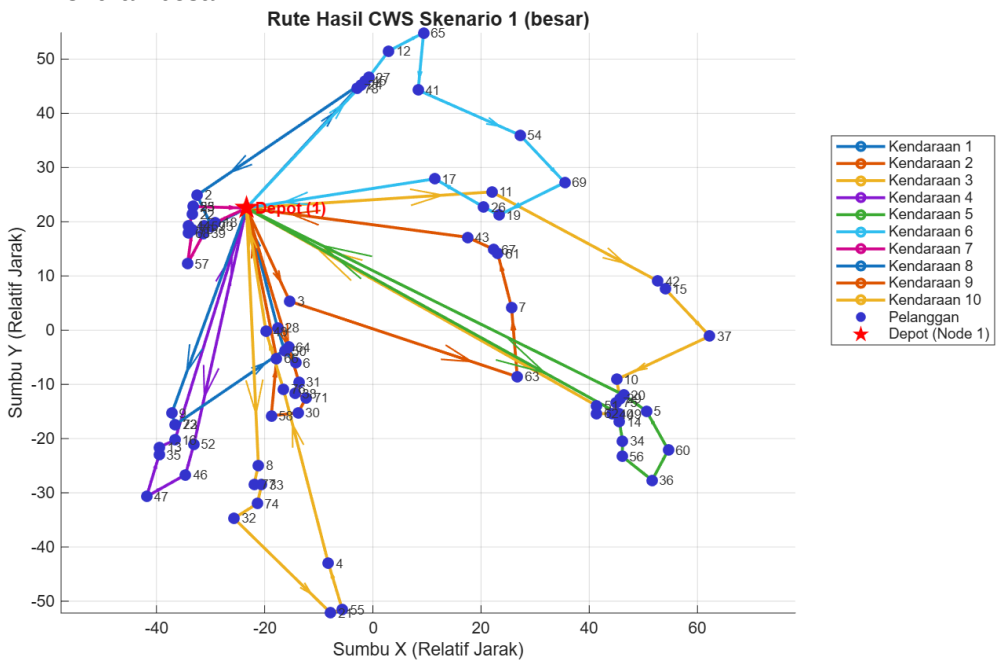
- Ukuran kecil



- Ukuran sedang

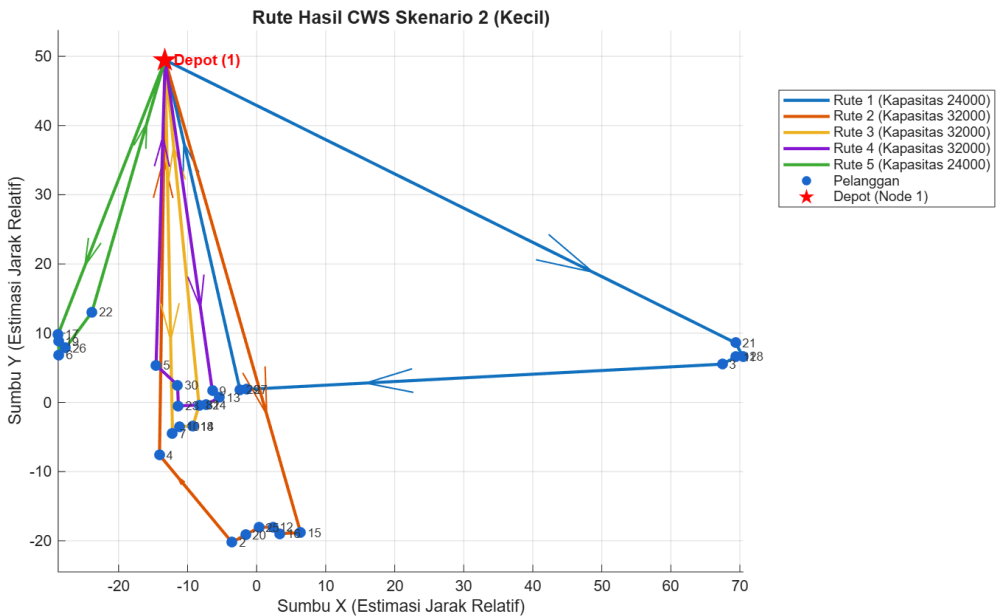


- Ukuran besar

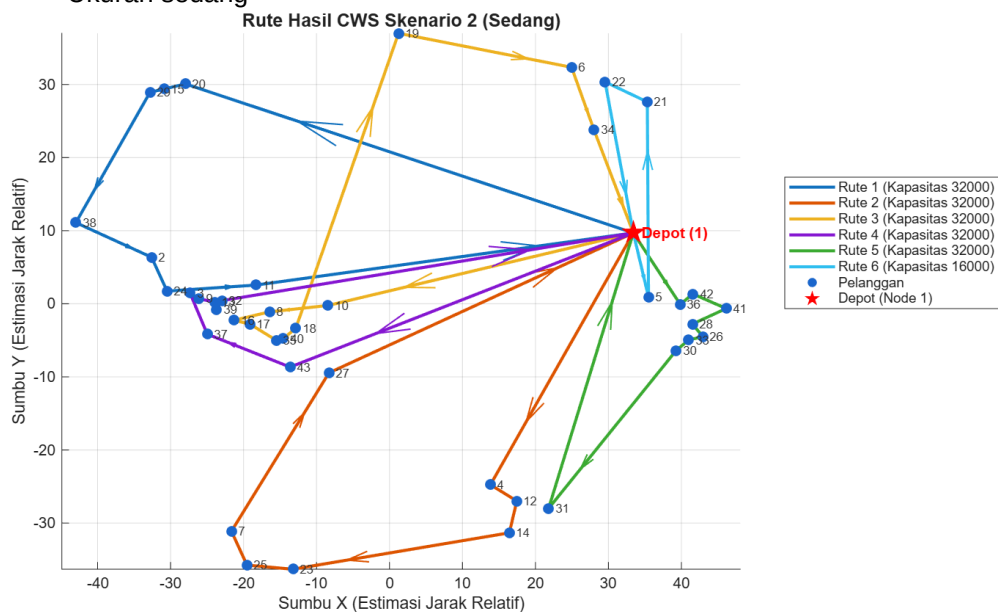


Lampiran 2. Rute hasil *Clarke and Wright Savings* skenario 2

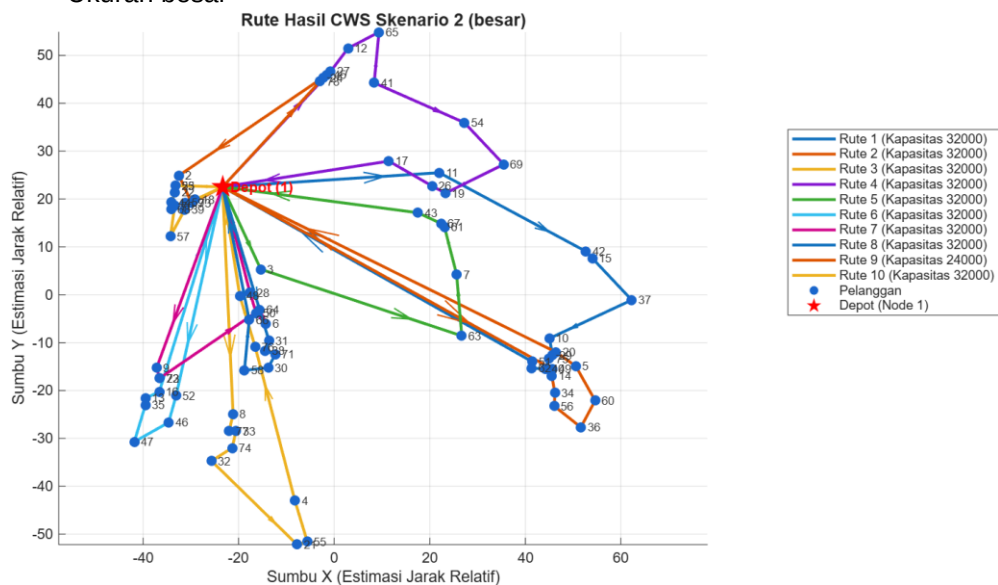
- Ukuran kecil



- Ukuran sedang

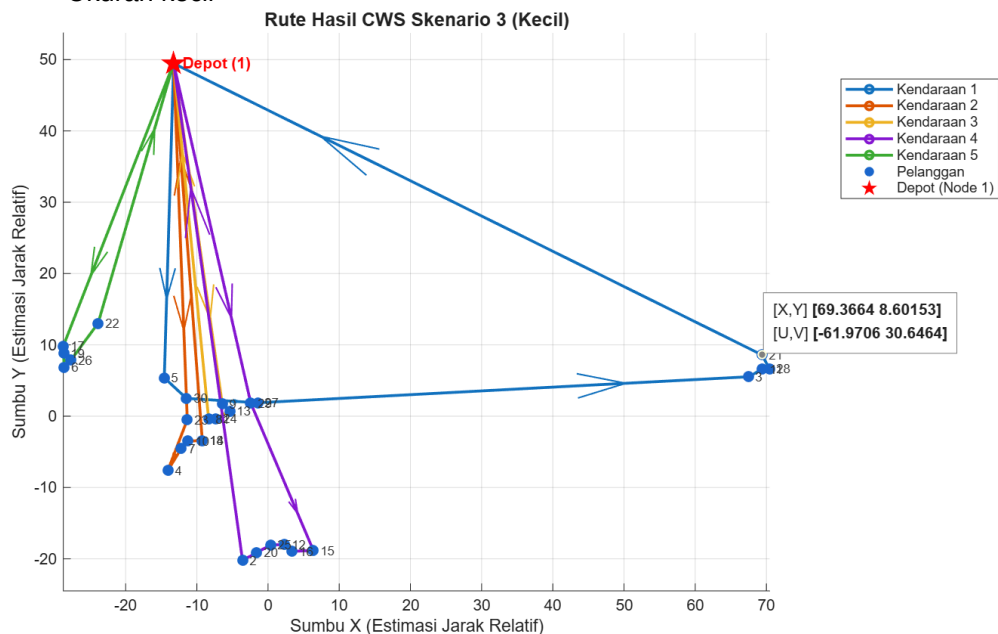


- Ukuran besar

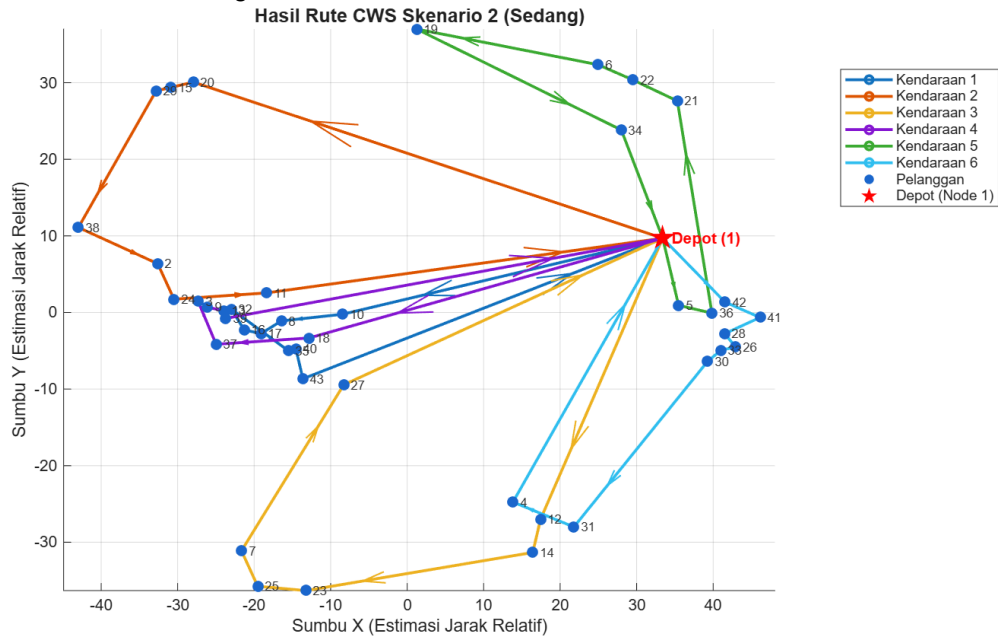


Lampiran 3. Rute hasil *Clarke and Wright Savings* skenario 3

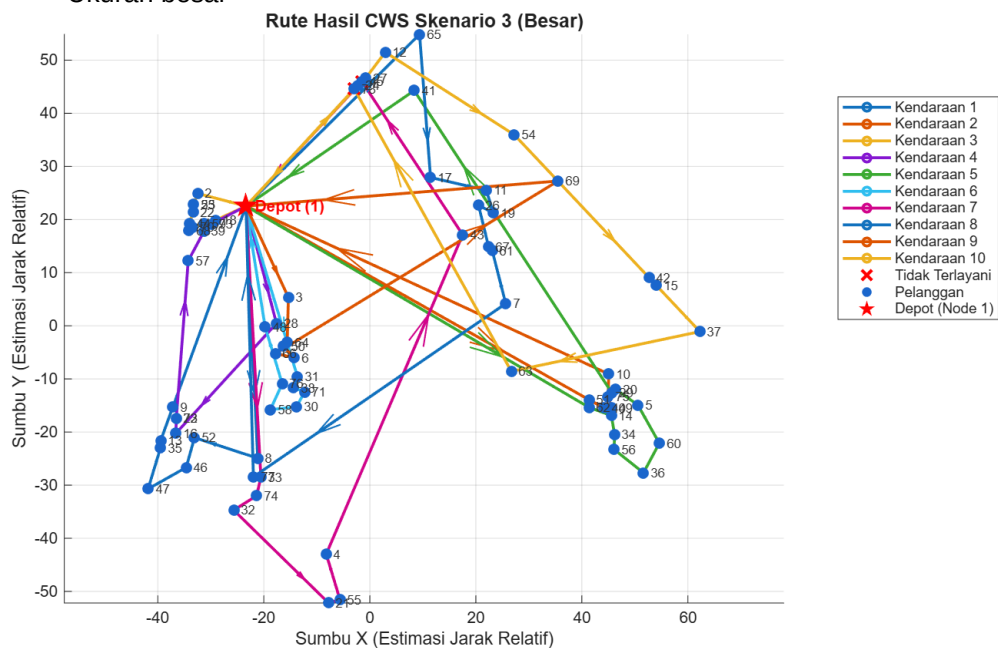
- Ukuran kecil



- Ukuran sedang

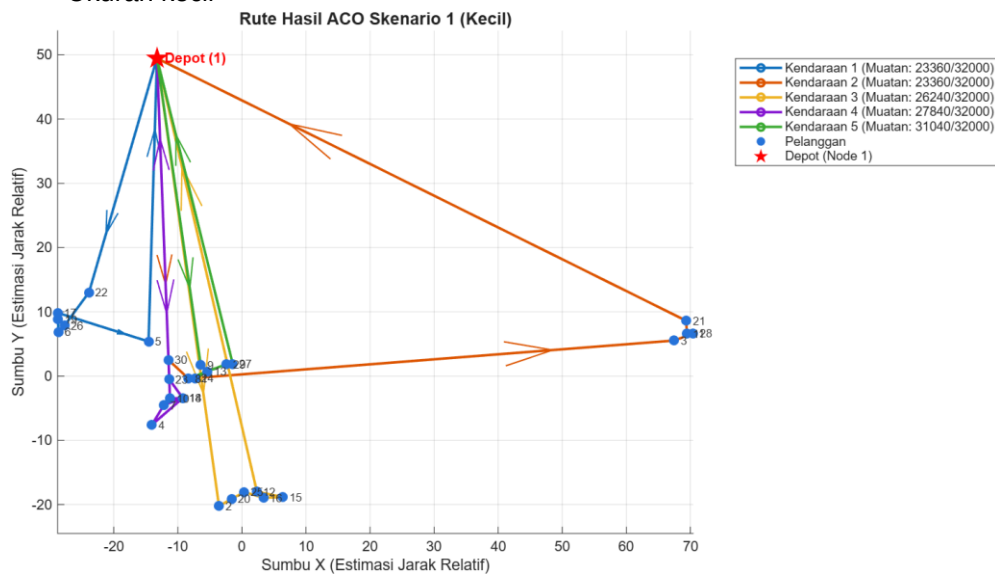


- Ukuran besar

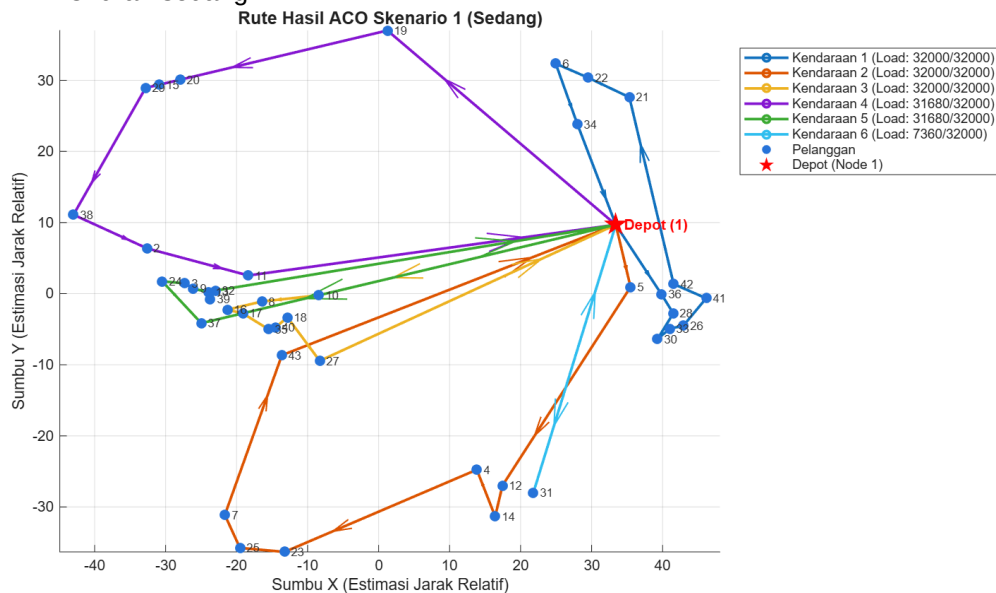


Lampiran 4. Rute hasil *Ant Colony Optimization* skenario 1

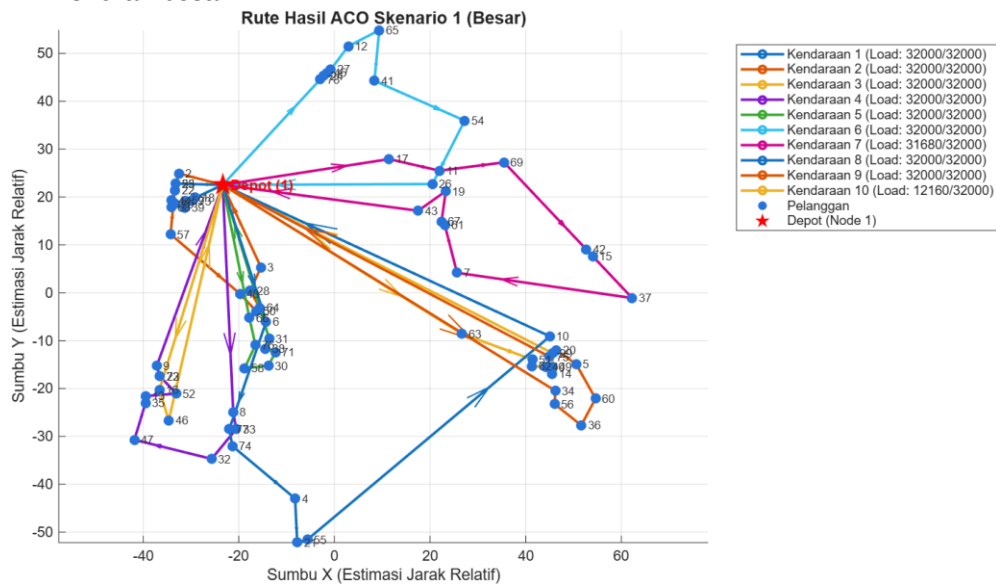
- Ukuran kecil



- Ukuran sedang

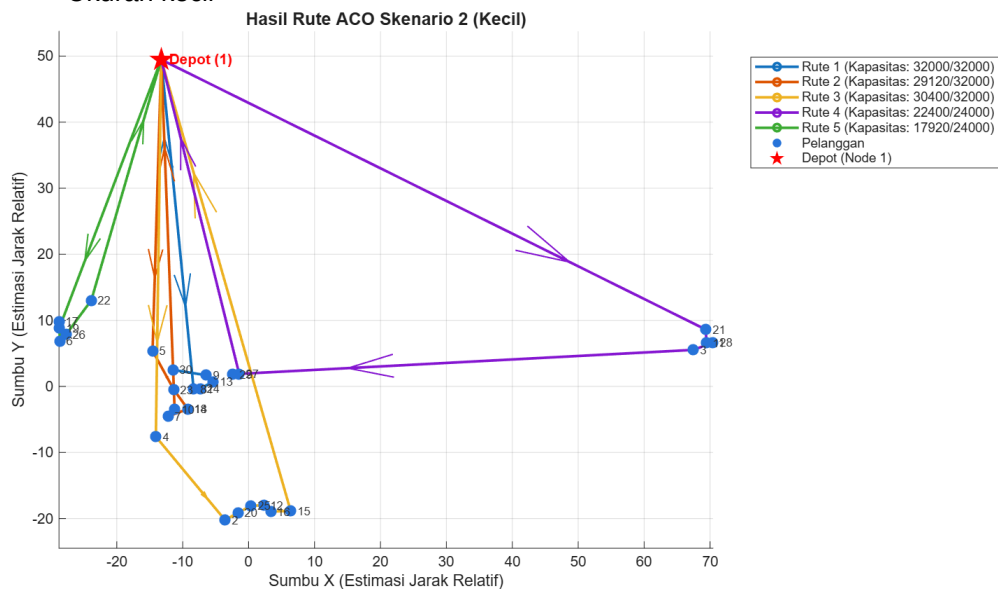


- Ukuran besar

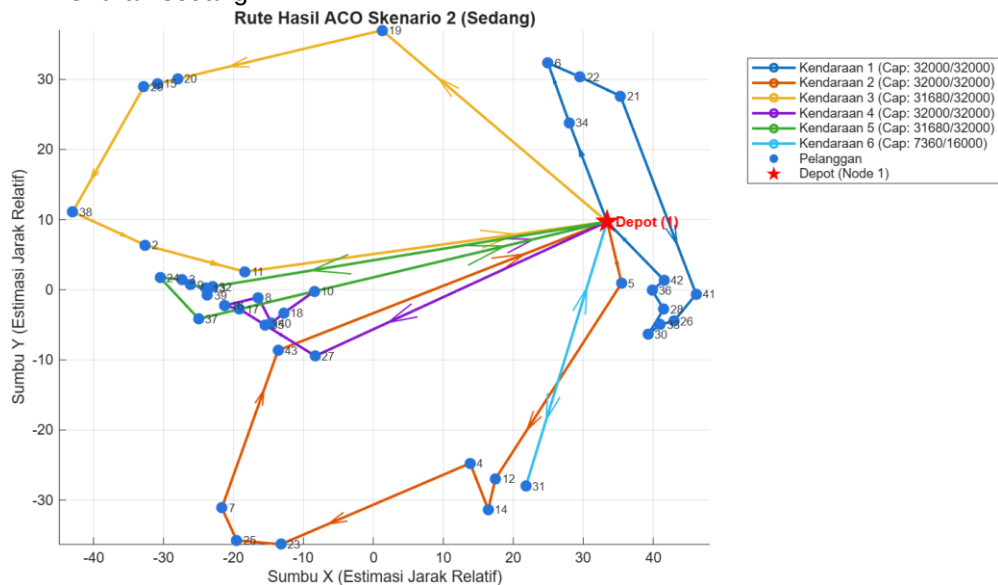


Lampiran 5. Rute hasil *Ant Colony Optimization* skenario 2

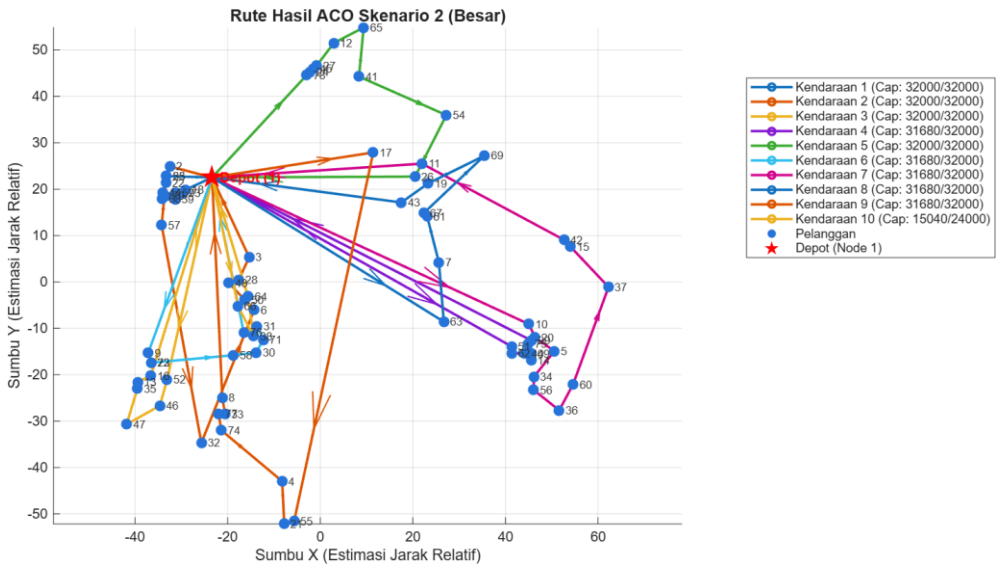
• Ukuran kecil



• Ukuran sedang

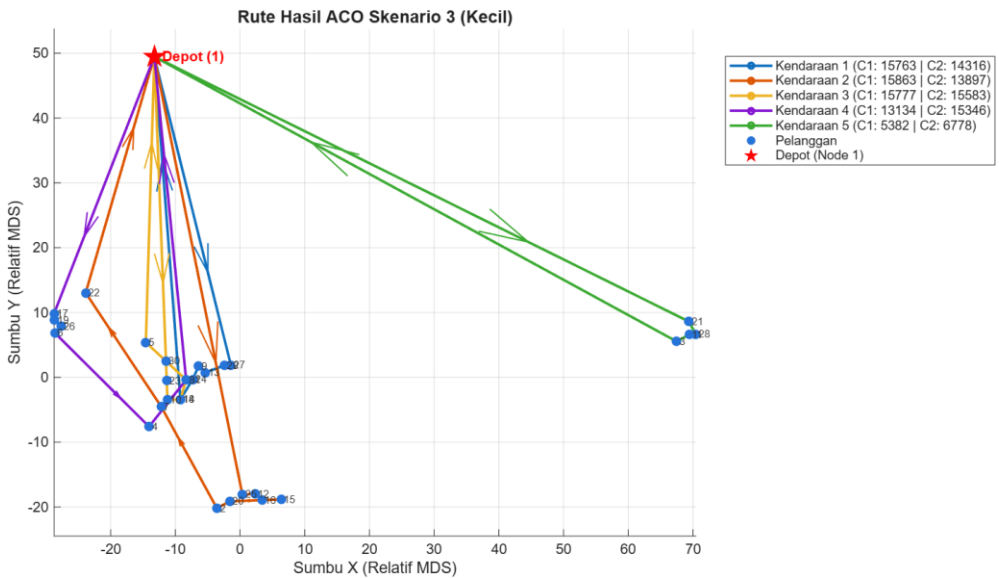


- Ukuran besar

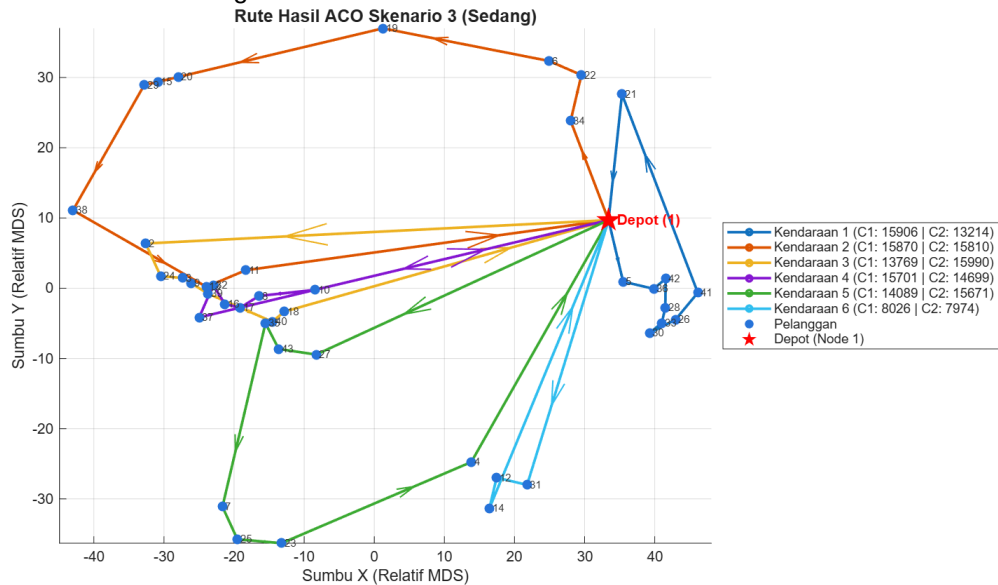


Lampiran 6. Rute hasil *Ant Colony Optimization* skenario 3

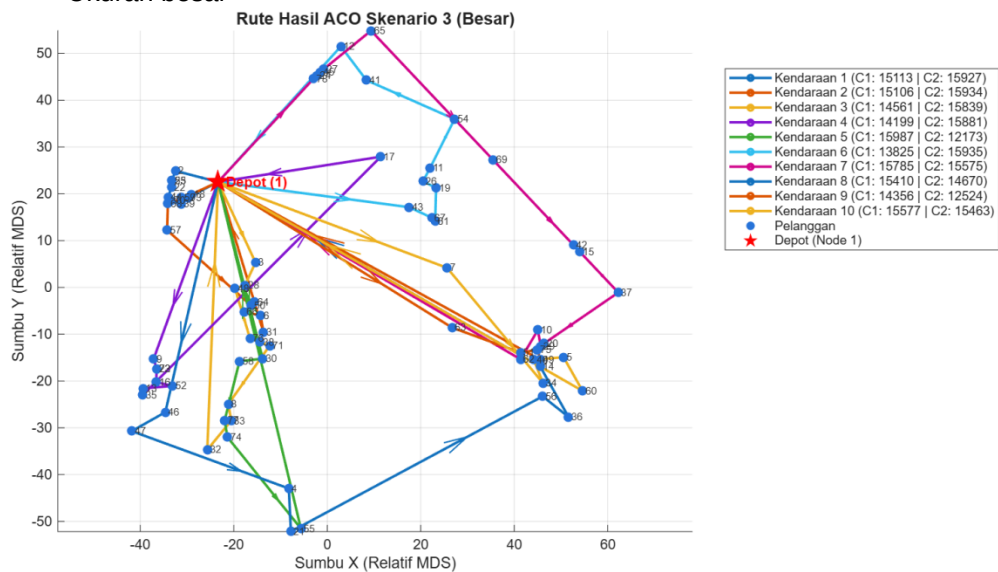
- Ukuran kecil



- Ukuran sedang



- Ukuran besar



Lampiran 7. Program Matlab pembagian *demand* pada MCVRP

```
clear; clc; close all;
```

```
% DEMAND
```

```
demand = [0, 8000, 960, 4160, 5440, 5120, 2880, 7040, 3200, 5120,  
2560, 960, 5120, 5120, 3200, 7680, 5120, 4800, 4480, 1600, 3840,
```

```

640, 5760, 6400, 4800, 2560, 7040, 4800, 3200, 4160, 6080];

n = length(demand);
BBM_A = zeros(n,1);
BBM_B = zeros(n,1);
targetA = sum(demand)/2;

%% Langkah 1
% Proporsi acak setiap pelanggan
rng(1);
for i=1:n

    % proporsi acak antara 30% - 70%
    lambda = 0.30 + (0.70-0.30)*rand;
    BBM_A(i)=lambda*demand(i);
    BBM_B(i)=demand(i)-BBM_A(i);
end

%% Langkah 2
% Koreksi agar total tepat 50%-50%
selisih = targetA - sum(BBM_A);
iter = 0;
while abs(selisih) > 1e-5
    iter = iter + 1;
    if iter > 10000
        break;
    end

    idx = randi(n);
    if selisih > 0
        ruang = min(BBM_B(idx),selisih);
        BBM_A(idx)=BBM_A(idx)+ruang;
        BBM_B(idx)=BBM_B(idx)-ruang;
    else
        ruang=min(BBM_A(idx),abs(selisih));
        BBM_A(idx)=BBM_A(idx)-ruang;
        BBM_B(idx)=BBM_B(idx)+ruang;
    end
    selisih = targetA-sum(BBM_A);
end

%% Hasil
T = table((1:n)',demand',BBM_A,BBM_B,
'VariableNames',{ 'Customer', 'Demand', 'BBM_A', 'BBM_B' });
disp(T)
fprintf('\n');
fprintf('Total Demand = %.2f\n',sum(demand));
fprintf('Total BBM A = %.2f\n',sum(BBM_A));
fprintf('Total BBM B = %.2f\n',sum(BBM_B));

```

Lampiran 8. Program Matlab visualisasi penyebaran node

```

clear; clc; close all;

%% 1. READ MATRIKS JARAK
filename = 'Matriks_Jarakkecil.xlsx';
try
    distance_matrix = readmatrix(filename);
catch
    error('File %s tidak ditemukan.', filename);
end
n_nodes = size(distance_matrix, 1);

%% 2. REKONSTRUKSI KOORDINAT DARI MATRIKS JARAK
try
    % Rekonstruksi koordinat 2D dari matriks jarak
    [coords, ~] = cmdscale(distance_matrix, 2);
    x = coords(:, 1);
    y = coords(:, 2);
catch
    % Fallback: Jika cmdscale gagal/tidak ada toolbox, susun
    melingkar
    theta = linspace(0, 2*pi, n_nodes);
    x = cos(theta);
    y = sin(theta);
end

%% 3. PLOT VISUALISASI NODE
figure('Name', 'Visualisasi Penyebaran Node dari Matriks Jarak',
'Color', 'w');
hold on; grid on; axis equal;

% Plot Pelanggan
scatter(x(2:end), y(2:end), 60, [0.2 0.6 0.2], 'filled',
'DisplayName', 'Pelanggan');

% Plot Depot
plot(x(1), y(1), 'rp', 'MarkerSize', 15, 'MarkerFaceColor', 'r',
'DisplayName', 'Depot (Node 1)');

% Tambahkan Nomor Node
for i = 1:n_nodes
    if i == 1
        text(x(i), y(i), ' Depot (1)', 'FontWeight', 'bold',
'Color', 'r');
    else
        text(x(i), y(i), sprintf(' %d', i), 'FontSize', 8,
'Color', [0.3 0.3 0.3]);
    end
end

```

```

end

title('Penyebaran Node Hasil Rekonstruksi Matriks Jarak',
'FontSize', 12, 'FontWeight', 'bold');
xlabel('Estimasi Sumbu X');
ylabel('Estimasi Sumbu Y');
legend('Location', 'northeastoutside');
hold off;

```

Lampiran 9. Program Matlab *Clarke and Wright Savings* skenario 1

```

clear; clc; close all;
tic

%% 1. INPUT FILE EXCEL
filename = 'Matriks_Jarakkecil.xlsx';

try
    distance_matrix = readmatrix(filename);
catch
    error('File "%s" tidak ditemukan. Pastikan file Excel berada
di folder yang sama dengan script MATLAB Anda.', filename);
end

num_nodes = size(distance_matrix, 1);

%% 2. PARAMETER KENDARAAN DAN DEMAND
num_vehicles = 5;           % Jumlah kendaraan maksimal
vehicle_capacity = 32000;   % Kapasitas maksimal kendaraan

% Input Demand Pelanggan Langsung via Array (Indeks 1 = Depot = 0)
demand = [0, 8000, 960, 4160, 5440, 5120, 2880, 7040, 3200, 5120,
2560, 960, 5120, 5120, 3200, 7680, 5120, 4800, 4480, 1600, 3840,
640, 5760, 6400, 4800, 2560, 7040, 4800, 3200, 4160, 6080];
node_demands = demand(:)';

% Validasi ukuran matriks Excel dengan jumlah data demand
if length(node_demands) ~= num_nodes
    error('Jumlah data demand (%d elemen) tidak sesuai dengan
ukuran matriks Excel (%dx%d)!', length(node_demands), num_nodes,
num_nodes);
end

num_customers = num_nodes - 1;
depot = 1;

%% 3. HITUNG MATRIKS SAVINGS
savings = [];

```

```

for i = 2:num_nodes-1
    for j = i+1:num_nodes
        s_value = distance_matrix(depot, i) +
distance_matrix(depot, j) - distance_matrix(i, j);
        savings = [savings; s_value, i, j];
    end
end

% Urutkan savings dari yang terbesar secara menurun
savings = sortrows(savings, -1);

%% 4. PEMBENTUKAN RUTE (MERGING ROUTES)
% Inisialisasi awal: Setiap customer memiliki rute tersendiri
[Depot -> i -> Depot]
routes = cell(num_customers, 1);
for i = 1:num_customers
    routes{i} = [depot, i+1, depot];
end

% Iterasi matriks savings untuk menggabungkan rute
for k = 1:size(savings, 1)
    node_i = savings(k, 2);
    node_j = savings(k, 3);

    route_idx_i = 0;
    route_idx_j = 0;

    for r = 1:length(routes)
        if ismember(node_i, routes{r})
            route_idx_i = r;
        end
        if ismember(node_j, routes{r})
            route_idx_j = r;
        end
    end

    % Syarat penggabungan rute
    if route_idx_i ~= route_idx_j && route_idx_i > 0 &&
route_idx_j > 0
        r_i = routes{route_idx_i};
        r_j = routes{route_idx_j};

        % Hitung total demand jika digabungkan
        total_demand = sum(node_demands(r_i)) +
sum(node_demands(r_j));

        if total_demand <= vehicle_capacity
            is_i_end = (r_i(end-1) == node_i);

```

```

        is_i_start = (r_i(2) == node_i);
        is_j_end = (r_j(end-1) == node_j);
        is_j_start = (r_j(2) == node_j);

        merged_route = [];
        if is_i_end && is_j_start
            merged_route = [r_i(1:end-1), r_j(2:end)];
        elseif is_i_start && is_j_end
            merged_route = [r_j(1:end-1), r_i(2:end)];
        elseif is_i_end && is_j_end
            merged_route = [r_i(1:end-1), fliplr(r_j(2:end-
1))), depot];
        elseif is_i_start && is_j_start
            merged_route = [depot, fliplr(r_i(2:end-1)),
r_j(2:end)];
        end

        if ~isempty(merged_route)
            routes{route_idx_i} = merged_route;
            routes(route_idx_j) = [];
        end
    end
end

%% 5. MENGHITUNG TOTAL JARAK DAN MENAMPILKAN HASIL DI COMMAND
WINDOW
total_distance = 0;

fprintf('          HASIL OPTIMASI CVRP (CLARKE & WRIGHT)          \n');
fprintf('===== \n \n');

% Cek apakah rute melebihi armada tersedia
if length(routes) > num_vehicles
    fprintf('❗ PERINGATAN: Rute optimal membutuhkan %d kendaraan,
melebihi batas armada (%d kendaraan). \n \n', length(routes),
num_vehicles);
end

for r = 1:length(routes)
    current_route = routes{r};
    route_dist = 0;
    route_demand = 0;

    for n = 1:(length(current_route) - 1)
        route_dist = route_dist +
distance_matrix(current_route(n), current_route(n+1));
        route_demand = route_demand +
node_demands(current_route(n));
    end
end

```

```

    end
    route_demand = route_demand +
node_demands(current_route(end));
    total_distance = total_distance + route_dist;

    % Format cetak panah rute
    rute_str = sprintf('%d', current_route(1));
    for n = 2:length(current_route)
        rute_str = [rute_str, sprintf(' -> %d',
current_route(n))];
    end

    fprintf('Kendaraan %d:\n', r);
    fprintf('  Rute   : %s\n', rute_str);
    fprintf('  Jarak  : %.2f\n', route_dist);
    fprintf('  Muatan  : %d / %d\n\n', route_demand,
vehicle_capacity);
end

fprintf('Total Kendaraan Digunakan : %d dari maks %d\n',
length(routes), num_vehicles);
fprintf('Total Jarak Seluruh Rute   : %.2f\n', total_distance);
toc
%% 6. VISUALISASI PENYEBARAN RUTE
figure('Name', 'CWS1sbesar', 'NumberTitle', 'off', 'Color', 'w');
hold on; grid on; axis equal;

% 1. Rekonstruksi Koordinat 2D dari Matriks Jarak
(Multidimensional Scaling / MDS)
try
    [coords, ~] = cmdscale(distance_matrix, 2);
    x = coords(:, 1);
    y = coords(:, 2);
catch
    % Fallback jika cmdscale tidak tersedia: Plot node dalam tata
letak melingkar
    theta = linspace(0, 2*pi, num_nodes);
    x = [0, cos(theta(1:end-1)) * 10];
    y = [0, sin(theta(1:end-1)) * 10];
end

% 2. Siapkan Palet Warna Unik untuk Setiap Kendaraan
colors = lines(length(routes));

% 3. Plot Jalur Rute Masing-Masing Kendaraan
for r = 1:length(routes)
    route = routes{r};
    route_x = x(route);
    route_y = y(route);

```

```

    % Gambar garis rute
    plot(route_x, route_y, '-o', 'LineWidth', 1.8, 'MarkerSize',
5, 'Color', colors(r, :), 'DisplayName', sprintf('Kendaraan %d',
r));

    % Tambahkan panah penunjuk arah perjalanan
    for k = 1:(length(route) - 1)
        u = x(route(k+1)) - x(route(k));
        v = y(route(k+1)) - y(route(k));
        quiver(x(route(k)), y(route(k)), u*0.8, v*0.8, 0, 'Color',
colors(r, :), 'MaxHeadSize', 0.5, 'LineWidth', 1,
'HandleVisibility', 'off');
    end
end

% 4. Plot Simbol Titik Pelanggan dan Depot
scatter(x(2:end), y(2:end), 50, [0.2 0.2 0.8], 'filled',
'DisplayName', 'Pelanggan');
plot(x(1), y(1), 'rp', 'MarkerSize', 14, 'MarkerFaceColor', 'r',
'DisplayName', 'Depot (Node 1)');

% 5. Tampilkan Label Nomor Node pada Grafik
for i = 1:num_nodes
    if i == 1
        text(x(i), y(i), ' Depot (1)', 'FontWeight', 'bold',
'Color', 'r', 'FontSize', 10);
    else
        text(x(i), y(i), sprintf(' %d', i), 'FontSize', 8,
'Color', [0.2 0.2 0.2]);
    end
end

title('Rute Hasil CWS Skenario 1 (besar)', 'FontSize', 12,
'FontWeight', 'bold');
xlabel('Sumbu X (Relatif Jarak)');
ylabel('Sumbu Y (Relatif Jarak)');
legend('Location', 'eastoutside');
hold off;

```

Lampiran 10. Program Matlab *Clarke and Wright Savings* skenario 2

```

clear; clc; close all;
tic

%% 1. INPUT PARAMETERS
excel_file = 'Matriks_Jarakkecil.xlsx';
sheet_name = 'Sheet1';

```

```

if exist(excel_file, 'file')
    distance_matrix = readmatrix(excel_file, 'Sheet', sheet_name);
else
    error('File Excel %s tidak ditemukan. Silakan pastikan file
berada di folder yang sama.', excel_file);
end

n_nodes = size(distance_matrix, 1);

% demand
demand = [0, 8000, 960, 4160, 5440, 5120, 2880, 7040, 3200, 5120,
2560, 960, 5120, 5120, 3200, 7680, 5120, 4800, 4480, 1600, 3840,
640, 5760, 6400, 4800, 2560, 7040, 4800, 3200, 4160, 6080];

% kendaraan
capacity = [8000 8000 8000 16000 16000 16000 24000 24000 24000
32000 32000 32000];
num_vehicles = 5; % Jumlah kendaraan

if num_vehicles > length(capacity)
    error('Jumlah kendaraan yang ditentukan (%d) melebihi total
kapasitas armada yang tersedia (%d)!', num_vehicles,
length(capacity));
end

% urutkan dari terkecil ke terbesar untuk efisiensi alokasi rute.
capacity = sort(capacity, 'descend');
capacity = capacity(1:num_vehicles);
capacity = sort(capacity);

%% 2. INISIALISASI & HITUNG MATRIKS SAVINGS
depot = 1;
n_customers = n_nodes - 1;

savings = [];
for i = 2:n_nodes
    for j = (i+1):n_nodes
        s_val = distance_matrix(depot, i) + distance_matrix(depot,
j) - distance_matrix(i, j);
        savings = [savings; i, j, s_val]; %#ok<AGROW>
    end
end

savings = sortrows(savings, -3);

%% 3. PROSES PENGABUNGAN RUTE
routes = cell(n_customers, 1);
for i = 2:n_nodes

```

```

    routes{i-1} = [depot, i, depot];
end

max_cap_available = max(capacity);

for k = 1:size(savings, 1)
    i = savings(k, 1);
    j = savings(k, 2);

    route_i_idx = []; route_j_idx = [];
    for r = 1:length(routes)
        if ismember(i, routes{r}), route_i_idx = r; end
        if ismember(j, routes{r}), route_j_idx = r; end
    end

    if ~isempty(route_i_idx) && ~isempty(route_j_idx) &&
(route_i_idx ~= route_j_idx)
        r_i = routes{route_i_idx};
        r_j = routes{route_j_idx};

        is_i_end = (r_i(2) == i) || (r_i(end-1) == i);
        is_j_end = (r_j(2) == j) || (r_j(end-1) == j);

        if is_i_end && is_j_end
            if r_i(end-1) == i && r_j(2) == j
                new_route = [r_i(1:end-1), r_j(2:end)];
            elseif r_i(2) == i && r_j(end-1) == j
                new_route = [r_j(1:end-1), r_i(2:end)];
            elseif r_i(end-1) == i && r_j(end-1) == j
                new_route = [r_i(1:end-1), fliplr(r_j(2:end-1)),
depot];
            else
                new_route = [depot, fliplr(r_i(2:end-1)),
r_j(2:end)];
            end

            total_load = sum(demand(new_route));

            if total_load <= max_cap_available
                routes{route_i_idx} = new_route;
                routes{route_j_idx} = [];
            end
        end
    end
end

end

%% 4. PRE-EVALUASI RUTE SEBELUM ALOKASI KENDARAAN
temp_routes_info = cell(length(routes), 1);
for r = 1:length(routes)

```

```

curr_route = routes{r};
route_load = sum(demand(curr_route));

route_dist = 0;
for node_idx = 1:(length(curr_route)-1)
    route_dist = route_dist +
distance_matrix(curr_route(node_idx), curr_route(node_idx+1));
end

temp_routes_info{r} = struct('nodes', curr_route, 'distance',
route_dist, 'load', route_load);
end

% Ekstrak jarak untuk keperluan sorting rute utama
all_distances = zeros(1, length(temp_routes_info));
for r = 1:length(temp_routes_info)
    all_distances(r) = temp_routes_info{r}.distance;
end

% Urutkan rute dari jarak terpanjang ke terpendek agar rute-rute
gabungan besar
[~, sort_route_idx] = sort(all_distances, 'descend');
temp_routes_info = temp_routes_info(sort_route_idx);

%% 5. ALOKASI KENDARAAN
vehicle_used_mask = false(1, num_vehicles);
final_routes_info = cell(length(temp_routes_info), 1);
total_distance_all = 0;
vehicles_used_count = 0;

for r = 1:length(temp_routes_info)
    curr_info = temp_routes_info{r};

    % Cari kendaraan terkecil yang muat dari kuota num_vehicles
    assigned_cap = NaN;
    for v = 1:num_vehicles
        if ~vehicle_used_mask(v) && curr_info.load <= capacity(v)
            assigned_cap = capacity(v);
            vehicle_used_mask(v) = true;
            vehicles_used_count = vehicles_used_count + 1;
            break;
        end
    end

    % Simpan dengan struktur field 'distance' yang mutlak ada di
    kedua kondisi
    if ~isnan(assigned_cap)
        final_routes_info{r} = struct('nodes', curr_info.nodes,
'distance', curr_info.distance, 'load', curr_info.load, 'vehicle',

```

```

sprintf('Kapasitas %d', assigned_cap));
    total_distance_all = total_distance_all +
curr_info.distance;
    else
        % Kondisi jika rute melebihi kuota 10 kendaraan yang
        ditentukan
        final_routes_info{r} = struct('nodes', curr_info.nodes,
'distance', curr_info.distance, 'load', curr_info.load, 'vehicle',
'TIDAK TERLAYANI (Kuota 10 Kendaraan Habis)');
    end
end

%% 6. DISPLAY OUTPUT HASIL OPTIMASI
fprintf('          HASIL OPTIMASI HETEROGENEOUS CVRP          \n');
fprintf('===== \n');

for r = 1:length(final_routes_info)
    info = final_routes_info{r};
    fprintf('Rute Hasil Analisis #d : %s\n', r,
num2str(info.nodes));
    fprintf(' - Status/Armada : %s\n', info.vehicle);
    fprintf(' - Jarak Rute : %.2f\n', info.distance);
    fprintf(' - Muatan Rute : %d\n\n', info.load);
end

fprintf('===== \n');
fprintf('TOTAL RINGKASAN OPTIMASI UTAMA: \n');
fprintf('Total Kendaraan Yang Digunakan : %d kendaraan\n',
vehicles_used_count);
fprintf('Total Jarak Rute Terlayani : %.2f\n',
total_distance_all);
toc

%% 7. VISUALISASI PENYEBARAN RUTE (HETEROGENEOUS CVRP)
figure('Name', 'CWS2besar', 'NumberTitle', 'off', 'Color', 'w');
hold on; grid on; axis equal;

% 1. Rekonstruksi Koordinat 2D dari Matriks Jarak
(Multidimensional Scaling / MDS)
try
    [coords, ~] = cmdscale(distance_matrix, 2);
    x = coords(:, 1);
    y = coords(:, 2);
catch
    % Fallback jika cmdscale tidak tersedia: Plot node dalam
    bentuk melingkar
    theta = linspace(0, 2*pi, n_nodes);
    x = [0, cos(theta(1:end-1)) * 10];
    y = [0, sin(theta(1:end-1)) * 10];
end

```

```

% 2. Siapkan Palet Warna Unik untuk Setiap Rute
colors = lines(length(final_routes_info));

% 3. Plot Jalur Rute Masing-Masing Kendaraan
for r = 1:length(final_routes_info)
    info = final_routes_info{r};
    route = info.nodes;
    route_x = x(route);
    route_y = y(route);

    % Cek status alokasi kendaraan
    is_served = ~contains(info.vehicle, 'TIDAK TERLAYANI');

    if is_served
        line_style = '-';
        line_width = 2.0;
        display_label = sprintf('Rute %d (%s)', r, info.vehicle);
        line_color = colors(r, :);
    else
        % Rute tidak terlayani ditampilkan dengan garis putus-
        putus merah
        line_style = '--';
        line_width = 1.2;
        display_label = sprintf('Rute %d (TIDAK TERLAYANI)', r);
        line_color = [0.8 0.1 0.1];
    end

    % Gambar garis rute
    plot(route_x, route_y, line_style, 'LineWidth', line_width,
        'MarkerSize', 5, 'Color', line_color, 'DisplayName',
        display_label);

    % Tambahkan panah penunjuk arah perjalanan (hanya jika
    terlayani)
    if is_served
        for k = 1:(length(route) - 1)
            u = x(route(k+1)) - x(route(k));
            v = y(route(k+1)) - y(route(k));
            quiver(x(route(k)), y(route(k)), u*0.75, v*0.75, 0,
                'Color', line_color, 'MaxHeadSize', 0.4, 'LineWidth', 1,
                'HandleVisibility', 'off');
        end
    end
end

% 4. Plot Simbol Titik Pelanggan dan Depot
scatter(x(2:end), y(2:end), 50, [0.1 0.4 0.8], 'filled',
    'DisplayName', 'Pelanggan');

```

```

plot(x(1), y(1), 'rp', 'MarkerSize', 15, 'MarkerFaceColor', 'r',
'DisplayName', 'Depot (Node 1)');

% 5. Tampilkan Label Nomor Node pada Grafik
for i = 1:n_nodes
    if i == 1
        text(x(i), y(i), ' Depot (1)', 'FontWeight', 'bold',
'Color', 'r', 'FontSize', 10);
    else
        text(x(i), y(i), sprintf(' %d', i), 'FontSize', 8,
'Color', [0.2 0.2 0.2]);
    end
end

title('Rute Hasil CWS Skenario 2 (besar)', 'FontSize', 12,
'FontWeight', 'bold');
xlabel('Sumbu X (Estimasi Jarak Relatif)');
ylabel('Sumbu Y (Estimasi Jarak Relatif)');
legend('Location', 'eastoutside');
hold off;

```

Lampiran 11. Program Matlab *Clarke and Wright Savings* skenario 3

```

clear; clc; close all;
tic

%% 1. INPUT PARAMETERS
excel_file = 'Matriks_Jarakkecil.xlsx';
sheet_name = 'Sheet1';

if exist(excel_file, 'file')
    distance_matrix = readmatrix(excel_file, 'Sheet', sheet_name);
else
    error('File Excel %s tidak ditemukan.', excel_file);
end

n_nodes = size(distance_matrix, 1);

% INPUT KENDARAAN
num_vehicles = 5;           % Jumlah kendaraan
cap_comp1 = 16000;         % Kapasitas Kompartemen 1
cap_comp2 = 16000;         % Kapasitas Kompartemen 2

% INPUT DEMAND 2 KOMPARTEMEN
demand = [
    0 4705 288.04 1751.1 1951.3 1725.1 1078.6 3085.1 1467.9 2639.5
  1197.3 551.12 1954.7 5120 995.06 4363.7 3293.4 2512.7 1595.6
  606.78 2381.9 439.87 2450.1 3692.3 3122.7 1684.1 2351.1 1515

```

```

1177.4 4160 2063.2;    % Kompartemen 1
    0 3295 671.96 2408.9 3488.7 3394.9 1801.4 3954.9 1732.1 2480.5
1362.7 408.88 3165.3 0 2204.9 3316.3 1826.6 2287.3 2884.4 993.22
1458.1 200.13 3309.9 2707.7 1677.3 875.92 4688.5 3285 2022.6 0
4016.8    % Kompartemen 2
];

if size(demand, 2) ~= n_nodes
    error('Jumlah kolom demand (%d) harus sama dengan jumlah baris
matriks jarak (%d)!', size(demand,2), n_nodes);
end

%% 2. HITUNG MATRIKS SAVINGS
depot = 1;
savings = [];
for i = 2:n_nodes
    for j = (i+1):n_nodes
        s_val = distance_matrix(depot, i) + distance_matrix(depot,
j) - distance_matrix(i, j);
        savings = [savings; i, j, s_val]; %#ok<AGROW>
    end
end
savings = sortrows(savings, -3);

%% 3. PROSES PENGGABUNGAN RUTE AWAL (SAVINGS LOOP)
routes = cell(n_nodes - 1, 1);
for i = 2:n_nodes
    routes{i-1} = [depot, i, depot];
end

for k = 1:size(savings, 1)
    i = savings(k, 1);
    j = savings(k, 2);

    route_i_idx = []; route_j_idx = [];
    for r = 1:length(routes)
        if ismember(i, routes{r}), route_i_idx = r; end
        if ismember(j, routes{r}), route_j_idx = r; end
    end

    if ~isempty(route_i_idx) && ~isempty(route_j_idx) &&
(route_i_idx ~= route_j_idx)
        r_i = routes{route_i_idx};
        r_j = routes{route_j_idx};

        is_i_end = (r_i(2) == i) || (r_i(end-1) == i);
        is_j_end = (r_j(2) == j) || (r_j(end-1) == j);

        if is_i_end && is_j_end

```

```

        if r_i(end-1) == i && r_j(2) == j
            new_route = [r_i(1:end-1), r_j(2:end)];
        elseif r_i(2) == i && r_j(end-1) == j
            new_route = [r_j(1:end-1), r_i(2:end)];
        elseif r_i(end-1) == i && r_j(end-1) == j
            new_route = [r_i(1:end-1), fliplr(r_j(2:end-1)),
depot];
        else
            new_route = [depot, fliplr(r_i(2:end-1)),
r_j(2:end)];
        end

        % Cek Kapasitas Ganda (Strict - Tidak boleh overload
sejak awal)
        if sum(demand(1, new_route)) <= cap_comp1 &&
sum(demand(2, new_route)) <= cap_comp2
            routes{route_i_idx} = new_route;
            routes(route_j_idx) = [];
        end
    end
end

%% 4. STRATEGI REDUKSI RUTE TANPA OVERLOAD

if length(routes) > num_vehicles
    % Urutkan rute berdasarkan efisiensi muatan totalnya
    total_loads_temp = zeros(1, length(routes));
    for r = 1:length(routes)
        total_loads_temp(r) = sum(demand(1, routes{r})) +
sum(demand(2, routes{r}));
    end
    [~, sort_idx] = sort(total_loads_temp, 'descend');
    routes = routes(sort_idx);

    % Ambil 5 rute pertama sebagai rute utama
    main_routes = routes(1:num_vehicles);
    overflow_routes = routes(num_vehicles+1:end);

    % Ekstrak pelanggan dari rute sisa yang harus didistribusikan
ulang
    leftover_customers = [];
    for r = 1:length(overflow_routes)
        r_nodes = overflow_routes{r};
        leftover_customers = [leftover_customers, r_nodes(r_nodes
~= depot)]; %#ok<AGROW>
    end

    % Masukkan pelanggan sisa ke rute utama secara selektif (Hanya

```

```

jika MUAT)
    unassigned_customers = [];
    for c_idx = 1:length(leftover_customers)
        cust = leftover_customers(c_idx);
        inserted = false;

        for r = 1:num_vehicles
            % Uji coba penyisipan pelanggan di posisi terbaik
            (sebelum depot akhir)
            test_route = [main_routes{r}(1:end-1), cust, depot];

            % Validasi Kapasitas Ganda secara ketat
            if sum(demand(1, test_route)) <= cap_comp1 &&
sum(demand(2, test_route)) <= cap_comp2
                main_routes{r} = test_route;
                inserted = true;
                break;
            end
        end

        % Jika tidak muat di rute mana pun karena sekat sudah
        penuh
        if ~inserted
            unassigned_customers = [unassigned_customers, cust];
        end
    end
    routes = main_routes;

    % Tampilkan peringatan jika ada pelanggan tertinggal akibat
    keterbatasan kapasitas + armada
    if ~isempty(unassigned_customers)
        warning('Beberapa pelanggan (Node %s) tidak dapat
        terlayani karena batasan ketat jumlah armada dan kapasitas
        kompartemen!', num2str(unassigned_customers));
    end
end

%% 5. PENYUSUNAN OUTPUT & HITUNG REAL DISTANCE
total_distance_all = 0;
vehicles_used_count = 0;

fprintf('    HASIL OPTIMASI MCVRP 2 KOMPARTEMEN - SAVINGS    \n');
fprintf('===== \n');

for r = 1:length(routes)
    curr_route = routes{r};
    load_c1 = sum(demand(1, curr_route));
    load_c2 = sum(demand(2, curr_route));

```

```

    % Jika rute kosong (hanya depot [1 1]), lewati pengosongan
    armada
    if length(curr_route) <= 2 && all(curr_route == depot)
        continue;
    end

    route_dist = 0;
    for node_idx = 1:(length(curr_route)-1)
        route_dist = route_dist +
distance_matrix(curr_route(node_idx), curr_route(node_idx+1));
    end

    vehicles_used_count = vehicles_used_count + 1;
    total_distance_all = total_distance_all + route_dist;

    fprintf('Rute Kendaraan #%d : %s\n', vehicles_used_count,
num2str(curr_route));
    fprintf(' - Jarak Rute      : %.2f\n', route_dist);
    fprintf(' - Muatan Komp. 1 : %d / %d (Aman)\n', load_c1,
cap_comp1);
    fprintf(' - Muatan Komp. 2 : %d / %d (Aman)\n\n', load_c2,
cap_comp2);
end

fprintf('=====');
fprintf('TOTAL RINGKASAN OPTIMASI UTAMA:\n');
fprintf('Total Kendaraan Yang Digunakan : %d kendaraan\n',
vehicles_used_count);
fprintf('Total Jarak Seluruh Rute      : %.2f\n',
total_distance_all);
toc

%% 6. VISUALISASI PENYEBARAN RUTE (MCVRP 2 KOMPARTEMEN)
figure('Name', 'Visualisasi Penyebaran Rute MCVRP 2 Kompartemen',
'NumberTitle', 'off', 'Color', 'w');
hold on; grid on; axis equal;

% 1. Rekonstruksi Koordinat 2D dari Matriks Jarak
(Multidimensional Scaling / MDS)
try
    [coords, ~] = cmdscale(distance_matrix, 2);
    x = coords(:, 1);
    y = coords(:, 2);
catch
    % Fallback jika cmdscale tidak tersedia: Plot node dalam tata
    letak melingkar
    theta = linspace(0, 2*pi, n_nodes);
    x = [0, cos(theta(1:end-1)) * 10];

```

```

    y = [0, sin(theta(1:end-1)) * 10];
end

% 2. Siapkan Warna Unik untuk Tiap Kendaraan
colors = lines(length(routes));

% 3. Plot Jalur Rute Kendaraan Terlayani
for r = 1:length(routes)
    curr_route = routes{r};

    % Abaikan rute kosong (hanya depot -> depot)
    if length(curr_route) <= 2 && all(curr_route == depot)
        continue;
    end

    route_x = x(curr_route);
    route_y = y(curr_route);

    % Gambar garis rute
    plot(route_x, route_y, '-o', 'LineWidth', 1.8, 'MarkerSize',
5, 'Color', colors(r, :), 'DisplayName', sprintf('Kendaraan %d',
r));

    % Tambahkan panah penunjuk arah perjalanan
    for k = 1:(length(curr_route) - 1)
        u = x(curr_route(k+1)) - x(curr_route(k));
        v = y(curr_route(k+1)) - y(curr_route(k));
        quiver(x(curr_route(k)), y(curr_route(k)), u*0.75, v*0.75,
0, 'Color', colors(r, :), 'MaxHeadSize', 0.4, 'LineWidth', 1,
'HandleVisibility', 'off');
    end
end

% 4. Tandai Pelanggan yang Tidak Terlayani (Jika Ada)
if exist('unassigned_customers', 'var') &&
~isempty(unassigned_customers)
    plot(x(unassigned_customers), y(unassigned_customers), 'rx',
'MarkerSize', 10, 'LineWidth', 2, 'DisplayName', 'Tidak
Terlayani');
end

% 5. Plot Simbol Titik Pelanggan dan Depot
scatter(x(2:end), y(2:end), 50, [0.1 0.4 0.8], 'filled',
'DisplayName', 'Pelanggan');
plot(x(1), y(1), 'rp', 'MarkerSize', 15, 'MarkerFaceColor', 'r',
'DisplayName', 'Depot (Node 1)');

% 6. Tampilkan Label Nomor Node pada Grafik
for i = 1:n_nodes

```

```

        if i == 1
            text(x(i), y(i), ' Depot (1)', 'FontWeight', 'bold',
'Color', 'r', 'FontSize', 10);
        else
            text(x(i), y(i), sprintf(' %d', i), 'FontSize', 8,
'Color', [0.2 0.2 0.2]);
        end
    end
end

title('Visualisasi Penyebaran Rute MCVRP (2 Kompartemen)',
'FontSize', 12, 'FontWeight', 'bold');
xlabel('Sumbu X (Estimasi Jarak Relatif)');
ylabel('Sumbu Y (Estimasi Jarak Relatif)');
legend('Location', 'eastoutside');
hold off;

```

Lampiran 12. Program Matlab Ant Colony Optimization skenario 1

```

clear; clc; close all;
tic

%% 1. INPUT PARAMETERS
excel_file = 'Matriks_Jarak.xlsx';
sheet_name = 'Sheet1';

if exist(excel_file, 'file')
    dist_matrix = readmatrix(excel_file, 'Sheet', sheet_name);
else
    error('File Excel %s tidak ditemukan! Pastikan nama file dan
folder kerja sudah benar.', excel_file);
end

n_nodes = size(dist_matrix, 1);
depot = 1; % Node 1 dipastikan sebagai Depot

% B. PARAMETER KENDARAAN & DEMAND
num_vehicles = 6;
capacity = 32000;

% Demand Pelanggan
demand = [0, 8000, 6080, 2560, 3200, 320, 4480, 3520, 6400, 7360,
960, 1280, 1920, 7360, 2240, 4800, 4160, 960, 3520, 7680, 2560,
4480, 2560, 2880, 7680, 3200, 6080, 3200, 1600, 2240, 7360, 3520,
3840, 1280, 3520, 5120, 7040, 7680, 3840, 1600, 3520, 2240, 2880];

% Validasi Ukuran Data
if length(demand) ~= n_nodes
    error('Ukuran array demand (%d) tidak sesuai dengan jumlah

```

```

node pada matriks jarak (%d)!', length(demand), n_nodes);
end

% C. PARAMETER ALGORITMA ACO
n_ants = 30;
max_iter = 100;
alpha = 1.0;           % Bobot Intensitas Feromon
beta = 2.0;            % Bobot Informasi Heuristik (Visibilitas)
Q_val = 7.0;           % KONSTANTA DEPOSIT FEROMON (Q)
rho_init = 0.3;        % Laju Penguapan Awal (Adaptive)
rho = rho_init;
e_weight = 5;          % Bobot Tambahan Elitist Ant

% Parameter Candidate List (Minimal 1, Maksimal 8)
k_candidate = max(1, min(8, n_nodes - 1));

% Inisialisasi Candidate List yang Aman
candidate_list = cell(n_nodes, 1);
for i = 1:n_nodes
    [~, sorted_idx] = sort(dist_matrix(i, :));
    sorted_idx(sorted_idx == i) = []; % Hapus node itu sendiri

    if isempty(sorted_idx)
        candidate_list{i} = [];
    else
        k_actual = min(k_candidate, length(sorted_idx));
        candidate_list{i} = sorted_idx(1:k_actual);
    end
end

%% 2. INISIALISASI MAX-MIN PHEROMONE (MMAS) & HEURISTIK
eta = 1 ./ (dist_matrix + eye(n_nodes) * 1e-6);

% Initial Cost menggunakan Nearest Neighbor untuk menetapkan nilai
Tau_Max
init_cost = run_nearest_neighbor(dist_matrix, demand, capacity,
depot);
tau_max = Q_val / (rho * init_cost); % Menyesuaikan variabel Q_val
tau_min = tau_max / (2 * n_nodes);

% Matriks Feromon awal
pheromone = ones(n_nodes, n_nodes) * tau_max;

% Pelacak Solusi Global Terbaik
best_global_cost = Inf;
best_global_routes = {};
stagnation_counter = 0;

%% 3. MAIN ACO ITERATION LOOP

```

```

fprintf('  MEMULAI OPTIMASI ADVANCED ACO - CVRP          \n');
fprintf('===== \n');

for iter = 1:max_iter
    all_ant_routes = cell(n_ants, 1);
    all_ant_costs = inf(n_ants, 1);

    for ant = 1:n_ants
        % BENTUK RUTE DENGAN CANDIDATE LIST
        routes = construct_ant_routes(dist_matrix, demand,
capacity, depot, num_vehicles, pheromone, eta, candidate_list,
alpha, beta);

        % Validasi jika semut berhasil melayani seluruh pelanggan
        if ~isempty(routes)
            % LOCAL SEARCH 2-OPT
            for r = 1:length(routes)
                routes{r} = apply_2opt(routes{r}, dist_matrix);
            end

            cost = calculate_total_cost(routes, dist_matrix);
            all_ant_routes{ant} = routes;
            all_ant_costs(ant) = cost;
        end
    end

    % Cari Semut Terbaik pada Iterasi Ini
    [iter_best_cost, iter_best_idx] = min(all_ant_costs);

    if iter_best_cost < Inf
        iter_best_routes = all_ant_routes{iter_best_idx};

        % Update Global Best
        if iter_best_cost < best_global_cost
            best_global_cost = iter_best_cost;
            best_global_routes = iter_best_routes;
            stagnation_counter = 0;
        else
            stagnation_counter = stagnation_counter + 1;
        end
    else
        stagnation_counter = stagnation_counter + 1;
    end

    % ADAPTIVE PARAMETER CONTROL (SAFE CLAMPING)
    if stagnation_counter > 10
        rho = min(0.50, rho * 1.10); % Tingkatkan penguapan jika
stagnan
    else

```

```

        rho = max(0.10, rho * 0.98); % Normalisasi laju penguapan
    end

    % Proteksi Mutlak nilai Rho (0.10 - 0.50)
    rho = max(0.10, min(0.50, rho));

    % UPDATE FEROMON (EVAPORATION + ELITIST + MAX-MIN)
    % 1. Penguapan Feromon (Evaporation)
    pheromone = (1 - rho) * pheromone;

    % 2. Deposito Feromon Iteration Best (Menggunakan Q_val)
    if iter_best_cost < Inf
        pheromone = deposit_pheromone(pheromone, iter_best_routes,
Q_val / iter_best_cost);
    end

    % 3. Fitur Elitist Ant System (EAS) & Max-Min Limits (MMAS)
    if best_global_cost < Inf
        % Deposit Elitist menggunakan Q_val * e_weight
        pheromone = deposit_pheromone(pheromone,
best_global_routes, (Q_val * e_weight) / best_global_cost);

        tau_max = Q_val / (rho * best_global_cost);
        tau_min = tau_max / (2 * n_nodes);
    end

    % Clamping MMAS
    pheromone(pheromone > tau_max) = tau_max;
    pheromone(pheromone < tau_min) = tau_min;

    % Logging Progress
    if mod(iter, 10) == 0 || iter == 1
        if best_global_cost < Inf
            fprintf('Iterasi %3d | Cost Terbaik: %.2f | Rho
(Adaptive): %.3f\n', iter, best_global_cost, rho);
        else
            fprintf('Iterasi %3d | Mencari Solusi Valid...\n',
iter);
        end
    end
end

%% 4. HASIL AKHIR & OUTPUT
fprintf('\n=====');
fprintf('OPTIMASI SELESAI!\n');
if best_global_cost < Inf

    for r = 1:length(best_global_routes)
        r_path = best_global_routes{r};
    end
end

```

```

        r_load = sum(demand(r_path));
        fprintf('Rute Kendaraan #%d : %s (Load: %d/%d)\n', r,
num2str(r_path), r_load, capacity);
    end
else
    warning('Tidak ditemukan solusi valid! Coba tambahkan
`num_vehicles` atau `capacity`.');
end

%% HELPER FUNCTIONS

function routes = construct_ant_routes(dist, demand, capacity,
depot, max_veh, phero, eta, cand_list, alpha, beta)
    n_nodes = length(demand);
    visited = false(1, n_nodes);

    % Pastikan depot berbentuk skalar tunggal
    if ~isscalar(depot)
        depot = depot(1);
    end
    visited(depot) = true;
    routes = {};

    while sum(visited) < n_nodes
        if length(routes) >= max_veh
            routes = {}; % Batas jumlah kendaraan terlampaui
            return;
        end

        curr_route = depot;
        curr_load = 0;
        curr_node = depot;

        while true
            if ~isscalar(curr_node)
                curr_node = curr_node(1);
            end

            % Pengecekan Aman Candidate List
            if (curr_node <= length(cand_list)) &&
~isempty(cand_list{curr_node})
                cands = cand_list{curr_node};
                cands = cands(:)'; % Konversi ke vektor baris
                feasible = cands(~visited(cands) & (curr_load +
demand(cands) <= capacity));
            else
                feasible = [];
            end
        end
    end
end

```

```

        % Fallback: Jika Candidate List kosong/habis, cari
dari SELURUH node tersisa
        if isempty(feasible)
            unvisited_all = find(~visited);
            feasible = unvisited_all(curr_load +
demand(unvisited_all) <= capacity);
        end

        % Jika tidak ada lagi node yang muat, rute kendaraan
ini selesai
        if isempty(feasible)
            break;
        end

        % SELECTION (ROULETTE WHEEL) AMAN
        probs = (phero(curr_node, feasible) .^ alpha) .*
(eta(curr_node, feasible) .^ beta);
        prob_sum = sum(probs);

        if prob_sum == 0 || isnan(prob_sum)
            next_node = feasible(randi(length(feasible)));
        else
            probs = probs / prob_sum;
            cum_probs = cumsum(probs);

            r = rand();
            idx = find(r <= cum_probs, 1, 'first');

            if isempty(idx)
                idx = length(feasible); % Proteksi jika rand()
mendekati 1.0
            end

            next_node = feasible(idx);
        end

        if isempty(next_node)
            break;
        end

        next_node = next_node(1); % Skalar

        curr_route = [curr_route, next_node]; %#ok<AGROW>
        curr_load = curr_load + demand(next_node);
        visited(next_node) = true;
        curr_node = next_node;
    end

    curr_route = [curr_route, depot]; %#ok<AGROW>

```

```

        routes{end+1} = curr_route; %#ok<AGROW>
    end
end

function route = apply_2opt(route, dist)
    best_route = route;
    n = length(route);
    improved = true;

    while improved
        improved = false;
        for i = 2:(n - 2)
            for j = (i + 1):(n - 1)
                d_old = dist(best_route(i-1), best_route(i)) +
dist(best_route(j), best_route(j+1));
                d_new = dist(best_route(i-1), best_route(j)) +
dist(best_route(i), best_route(j+1));

                if d_new < d_old
                    best_route(i:j) = fliplr(best_route(i:j));
                    improved = true;
                end
            end
        end
        route = best_route;
    end

function phero = deposit_pheromone(phero, routes, delta)
    for r = 1:length(routes)
        path = routes{r};
        for i = 1:(length(path) - 1)
            u = path(i);
            v = path(i+1);
            phero(u, v) = phero(u, v) + delta;
            phero(v, u) = phero(v, u) + delta;
        end
    end

function total_cost = calculate_total_cost(routes, dist)
    total_cost = 0;
    for r = 1:length(routes)
        path = routes{r};
        for i = 1:(length(path) - 1)
            total_cost = total_cost + dist(path(i), path(i+1));
        end
    end
end
end

```

```

function cost = run_nearest_neighbor(dist, demand, capacity,
depot)
    n = length(demand);
    visited = false(1, n);
    visited(depot) = true;
    cost = 0;

    curr = depot;
    load = 0;
    while sum(visited) < n
        unvisited = find(~visited);
        feasible = unvisited(load + demand(unvisited) <=
capacity);

        if isempty(feasible)
            cost = cost + dist(curr, depot);
            curr = depot;
            load = 0;
        else
            [min_d, idx] = min(dist(curr, feasible));
            next_node = feasible(idx);
            cost = cost + min_d;
            load = load + demand(next_node);
            visited(next_node) = true;
            curr = next_node;
        end
    end
    cost = cost + dist(curr, depot);
end
fprintf('\n===== \n');
fprintf('Total Jarak Terpendek : %.2f \n', best_global_cost);
fprintf('Armada Terpakai      : %d / %d Kendaraan \n',
length(best_global_routes), num_vehicles);
toc

%% 5. VISUALISASI PENYEBARAN RUTE ADVANCED ACO
if best_global_cost < Inf && ~isempty(best_global_routes)
    figure('Name', 'AC01Sedang', 'Color', 'w');
    hold on; grid on; axis equal;

    % 1. Rekonstruksi Koordinat 2D dari Matriks Jarak
    (Multidimensional Scaling / MDS)
    try
        [coords, ~] = cmdscale(dist_matrix, 2);
        x = coords(:, 1);
        y = coords(:, 2);
    catch
        % Fallback jika cmdscale gagal/tidak ada toolbox: Susun

```

```

node secara melingkar
    theta = linspace(0, 2*pi, n_nodes);
    x = [0, cos(theta(1:end-1)) * 10];
    y = [0, sin(theta(1:end-1)) * 10];
end

% 2. Siapkan Warna Unik untuk Setiap Kendaraan
colors = lines(length(best_global_routes));

% 3. Plot Jalur Perjalanan Setiap Kendaraan
for r = 1:length(best_global_routes)
    r_path = best_global_routes{r};
    r_load = sum(demand(r_path));

    route_x = x(r_path);
    route_y = y(r_path);

    % Plot Garis Rute
    plot(route_x, route_y, '-o', 'LineWidth', 1.8,
'MarkerSize', 5, 'Color', colors(r, :), 'DisplayName',
sprintf('Kendaraan %d (Load: %d/%d)', r, r_load, capacity));

    % Tambahkan Panah Penunjuk Arah Perjalanan
    for k = 1:(length(r_path) - 1)
        u = x(r_path(k+1)) - x(r_path(k));
        v = y(r_path(k+1)) - y(r_path(k));
        quiver(x(r_path(k)), y(r_path(k)), u*0.75, v*0.75, 0,
'Color', colors(r, :), 'MaxHeadSize', 0.4, 'LineWidth', 1,
'HandleVisibility', 'off');
    end
end

% 4. Plot Simbol Pelanggan & Depot
scatter(x(2:end), y(2:end), 50, [0.15 0.45 0.85], 'filled',
'DisplayName', 'Pelanggan');
plot(x(1), y(1), 'rp', 'MarkerSize', 15, 'MarkerFaceColor',
'r', 'DisplayName', 'Depot (Node 1)');

% 5. Tampilkan Nomor Label Node
for i = 1:n_nodes
    if i == 1
        text(x(i), y(i), 'Depot (1)', 'FontWeight', 'bold',
'Color', 'r', 'FontSize', 10);
    else
        text(x(i), y(i), sprintf(' %d', i), 'FontSize', 8,
'Color', [0.2 0.2 0.2]);
    end
end
end

```

```

    title('Rute Hasil ACO Skenario 1 (Sedang)', 'FontSize', 12,
'FontWeight', 'bold');
    xlabel('Sumbu X (Estimasi Jarak Relatif)');
    ylabel('Sumbu Y (Estimasi Jarak Relatif)');
    legend('Location', 'eastoutside');
    hold off;
end

```

Lampiran 13. Program Matlab Ant Colony Optimization skenario 2

```

clear; clc; close all;
tic

%% 1. INPUT PARAMETERS
% A. BACA FILE EXCEL
excel_file = 'Matriks_Jarakbesar.xlsx';
sheet_name = 'Sheet1';
if exist(excel_file, 'file')
    dist_matrix = readmatrix(excel_file, 'Sheet', sheet_name);
else
    error('File Excel %s tidak ditemukan! Pastikan nama file dan
folder kerja sudah benar.', excel_file);
end
n_nodes = size(dist_matrix, 1);
depot = 1; % Node 1 dipastikan sebagai Depot

% B. INPUT DEMAND PELANGGAN
demand = [0, 4480, 5440, 5440, 5120, 6080, 5440, 1600, 3840, 1280,
640, 640, 8320, 640, 2240, 5760, 1920, 1920, 5760, 640, 4480,
1600, 2880, 1280, 960, 4800, 1280, 7360, 2240, 6720, 1280, 320,
1920, 5120, 1280, 6400, 1600, 4480, 4480, 8320, 1600, 640, 4480,
3520, 6720, 6400, 5760, 640, 6080, 3840, 7040, 4480, 7360, 8000,
2560, 960, 2880, 6720, 960, 7040, 1920, 640, 7040, 6400, 1600,
4160, 1920, 4480, 5120, 3840, 7360, 1600, 3840, 4800, 6720, 1280,
7360, 6080];
if length(demand) ~= n_nodes
    error('Ukuran array demand (%d) tidak sesuai dengan jumlah
node pada matriks jarak (%d)!', length(demand), n_nodes);
end

% C. INPUT KENDARAAN HETEROGEN
types_capacity = [24000, 32000]; % Kapasitas tiap jenis kendaraan
types_count    = [1, 9];       % Jumlah unit yang tersedia

% Konstruksi Otomatis Array Armada Kendaraan
vec_capacities = [];
for t = 1:length(types_capacity)
    vec_capacities = [vec_capacities, repmat(types_capacity(t), 1,

```

```

types_count(t))];
end
num_vehicles = length(vec_capacities);

% Urutkan armada dari kapasitas terbesar ke terkecil agar
pengalokasian efisien
vec_capacities = sort(vec_capacities, 'descend');

% Validasi Kelayakan Total Kapasitas
if sum(vec_capacities) < sum(demand)
    error('Total kapasitas armada (%d) KURANG dari total demand
pelanggan (%d)! Tambahkan jumlah kendaraan.', sum(vec_capacities),
sum(demand));
end

% D. PARAMETER ALGORITMA AC
n_ants = 80;
max_iter = 100;
alpha = 1.0;           % Bobot Intensitas Feromon
beta = 2.0;            % Bobot Informasi Heuristik (Visibilitas)
Q_val = 6.0;           % Konstanta Deposit Feromon (Q)
rho_init = 0.3;        % Laju Penguapan Awal (Adaptive)
rho = rho_init;
e_weight = 5;          % Bobot Tambahan Elitist Ant

% Parameter Candidate List (Minimal 1, Maksimal 8)
k_candidate = max(1, min(8, n_nodes - 1));
candidate_list = cell(n_nodes, 1);
for i = 1:n_nodes
    [~, sorted_idx] = sort(dist_matrix(i, :));
    sorted_idx(sorted_idx == i) = []; % Hapus node itu sendiri

    if isempty(sorted_idx)
        candidate_list{i} = [];
    else
        k_actual = min(k_candidate, length(sorted_idx));
        candidate_list{i} = sorted_idx(1:k_actual);
    end
end

%% 2. INISIALISASI MAX-MIN PHEROMONE (MMAS) & HEURISTIK
% Visibilitas (Eta = 1 / Jarak)
eta = 1 ./ (dist_matrix + eye(n_nodes) * 1e-6);

% Initial Cost menggunakan Heuristik Sederhana
init_cost = run_hetero_nearest_neighbor(dist_matrix, demand,
vec_capacities, depot);
tau_max = Q_val / (rho * init_cost);
tau_min = tau_max / (2 * n_nodes);

```

```

pheromone = ones(n_nodes, n_nodes) * tau_max;

best_global_cost = Inf;
best_global_routes = {};
best_global_caps = [];
stagnation_counter = 0;

%% 3. MAIN ACO ITERATION LOOP
fprintf(' MEMULAI OPTIMASI ADVANCED ACO - HETEROGENEOUS CVRP \n');
fprintf('===== \n');
fprintf('Armada Tersedia: %d Kendaraan (Total Kapasitas: %d)\n',
num_vehicles, sum(vec_capacities));
for t = 1:length(types_capacity)
    fprintf(' - Jenis Kapasitas %d : %d Unit\n',
types_capacity(t), types_count(t));
end
fprintf('----- \n');

for iter = 1:max_iter
    all_ant_routes = cell(n_ants, 1);
    all_ant_costs = inf(n_ants, 1);
    all_ant_caps = cell(n_ants, 1);

    for ant = 1:n_ants
        % BENTUK RUTE DENGAN ARMADA HETEROGEN
        [routes, used_caps] =
construct_hetero_ant_routes(dist_matrix, demand, vec_capacities,
depot, pheromone, eta, candidate_list, alpha, beta);

        if ~isempty(routes)
            % Local Search 2-opt
            for r = 1:length(routes)
                routes{r} = apply_2opt(routes{r}, dist_matrix);
            end

            cost = calculate_total_cost(routes, dist_matrix);
            all_ant_routes{ant} = routes;
            all_ant_costs(ant) = cost;
            all_ant_caps{ant} = used_caps;
        end
    end

    % Cari Semut Terbaik pada Iterasi Ini
    [iter_best_cost, iter_best_idx] = min(all_ant_costs);

    if iter_best_cost < Inf
        iter_best_routes = all_ant_routes{iter_best_idx};

        if iter_best_cost < best_global_cost

```

```

        best_global_cost = iter_best_cost;
        best_global_routes = iter_best_routes;
        best_global_caps = all_ant_caps{iter_best_idx};
        stagnation_counter = 0;
    else
        stagnation_counter = stagnation_counter + 1;
    end
else
    stagnation_counter = stagnation_counter + 1;
end

% ADAPTIVE PARAMETER CONTROL
if stagnation_counter > 10
    rho = min(0.50, rho * 1.10);
else
    rho = max(0.10, rho * 0.98);
end
rho = max(0.10, min(0.50, rho));

% UPDATE FEROMON
pheromone = (1 - rho) * pheromone;

if iter_best_cost < Inf
    pheromone = deposit_pheromone(pheromone, iter_best_routes,
Q_val / iter_best_cost);
end

if best_global_cost < Inf
    pheromone = deposit_pheromone(pheromone,
best_global_routes, (Q_val * e_weight) / best_global_cost);
    tau_max = Q_val / (rho * best_global_cost);
    tau_min = tau_max / (2 * n_nodes);
end

pheromone(pheromone > tau_max) = tau_max;
pheromone(pheromone < tau_min) = tau_min;

% Logging Progress
if mod(iter, 10) == 0 || iter == 1
    if best_global_cost < Inf
        fprintf('Iterasi %3d | Cost Terbaik: %.2f | Rho
(Adaptive): %.3f\n', iter, best_global_cost, rho);
    else
        fprintf('Iterasi %3d | Mencari Solusi Valid...\n',
iter);
    end
end
end
end

```

```

%% 4. HASIL AKHIR & OUTPUT
fprintf('\n=====');
fprintf('OPTIMASI HETEROGEN SELESAI!\n');
if best_global_cost < Inf
    for r = 1:length(best_global_routes)
        r_path = best_global_routes{r};
        r_load = sum(demand(r_path));
        r_cap = best_global_caps(r);
        fprintf('Kendaraan #%d (Kapasitas Maks %d) : %s | Load: %d\n',
            r, r_cap, num2str(r_path), r_load, (r_load/r_cap)*100);
    end
else
    warning('Tidak ditemukan solusi valid! Coba tambahkan variasi/jumlah kendaraan.');
```

fprintf('\nTotal Jarak Terpendek : %.2f\n', best_global_cost);
fprintf('Armada Terpakai : %d / %d Kendaraan\n',
length(best_global_routes), num_vehicles);

```

%% 5. VISUALISASI PENYEBARAN RUTE 2D
if best_global_cost < Inf && ~isempty(best_global_routes)
    fig = figure('Name', 'ACO2Sedang', 'Color', 'w');
    hold on; grid on; axis equal;

    % 1. Rekonstruksi Koordinat 2D dari Matriks Jarak
    (Multidimensional Scaling / MDS)
    try
        [coords, ~] = cmdscale(dist_matrix, 2);
        x = coords(:, 1);
        y = coords(:, 2);
    catch
        % Fallback jika cmdscale tidak tersedia: Susun node secara
        melingkar
        theta = linspace(0, 2*pi, n_nodes);
        x = [0, cos(theta(1:end-1)) * 10];
        y = [0, sin(theta(1:end-1)) * 10];
    end

    % 2. Siapkan Warna Unik untuk Setiap Kendaraan
    colors = lines(length(best_global_routes));

    % 3. Plot Jalur Perjalanan Setiap Kendaraan
    for r = 1:length(best_global_routes)
        r_path = best_global_routes{r};
        r_load = sum(demand(r_path));
        r_cap = best_global_caps(r);

```

```

route_x = x(r_path);
route_y = y(r_path);

% Hitung total jarak rute ini
r_dist = 0;
for n_idx = 1:(length(r_path)-1)
    r_dist = r_dist + dist_matrix(r_path(n_idx),
r_path(n_idx+1));
end

% Plot Garis Rute
plot(route_x, route_y, '-o', 'LineWidth', 1.8,
'MarkerSize', 5, 'Color', colors(r, :), 'DisplayName',
sprintf('Kendaraan %d (Cap: %d | Load: %d | Jarak: %.1f)', r,
r_cap, r_load, r_dist));

% Tambahkan Panah Arah Perjalanan
for k = 1:(length(r_path) - 1)
    u = x(r_path(k+1)) - x(r_path(k));
    v = y(r_path(k+1)) - y(r_path(k));
    quiver(x(r_path(k)), y(r_path(k)), u*0.75, v*0.75, 0,
'Color', colors(r, :), 'MaxHeadSize', 0.4, 'LineWidth', 1,
'HandleVisibility', 'off');
end
end

% 4. Plot Simbol Pelanggan & Depot
scatter(x(2:end), y(2:end), 60, [0.15 0.45 0.85], 'filled',
'DisplayName', 'Pelanggan');
plot(x(1), y(1), 'rp', 'MarkerSize', 16, 'MarkerFaceColor',
'r', 'DisplayName', 'Depot (Node 1)');

% 5. Tampilkan Nomor Label Node
for i = 1:n_nodes
    if i == 1
        text(x(i), y(i), ' Depot (1)', 'FontWeight', 'bold',
'Color', 'r', 'FontSize', 10);
    else
        text(x(i), y(i), sprintf(' %d', i), 'FontSize', 8,
'Color', [0.2 0.2 0.2]);
    end
end

title('Rute Hasil ACO Skenario 2 (Sedang)', 'FontSize', 12,
'FontWeight', 'bold');
xlabel('Sumbu X (Estimasi Jarak Relatif)');
ylabel('Sumbu Y (Estimasi Jarak Relatif)');
legend('Location', 'eastoutside');
hold off;

```

```

    % Paksa Jendela Gambar Muncul ke Paling Depan
    figure(fig);
    drawnow;
else
    warning('Visualisasi tidak dapat ditampilkan karena belum ada
solusi valid yang ditemukan.');
```

```
end

toc

%% HELPER FUNCTIONS
function [routes, used_caps] = construct_hetero_ant_routes(dist,
demand, vec_caps, depot, phero, eta, cand_list, alpha, beta)
    n_nodes = length(demand);
    visited = false(1, n_nodes);

    if ~isscalar(depot)
        depot = depot(1);
    end
    visited(depot) = true;
    routes = {};
    used_caps = [];

    for v = 1:length(vec_caps)
        if sum(visited) >= n_nodes
            break;
        end

        curr_cap = vec_caps(v);
        curr_route = depot;
        curr_load = 0;
        curr_node = depot;

        while true
            if ~isscalar(curr_node)
                curr_node = curr_node(1);
            end

            if (curr_node <= length(cand_list)) &&
~isempty(cand_list{curr_node})
                cands = cand_list{curr_node};
                cands = cands(:)';
                feasible = cands(~visited(cands) & (curr_load +
demand(cands) <= curr_cap));
            else
                feasible = [];
            end
        end
    end
end

```

```

        if isempty(feasible)
            unvisited_all = find(~visited);
            feasible = unvisited_all(curr_load +
demand(unvisited_all) <= curr_cap);
        end

        if isempty(feasible)
            break;
        end

        probs = (phero(curr_node, feasible) .^ alpha) .*
(eta(curr_node, feasible) .^ beta);
        prob_sum = sum(probs);

        if prob_sum == 0 || isnan(prob_sum)
            next_node = feasible(randi(length(feasible)));
        else
            probs = probs / prob_sum;
            cum_probs = cumsum(probs);

            r = rand();
            idx = find(r <= cum_probs, 1, 'first');
            if isempty(idx)
                idx = length(feasible);
            end
            next_node = feasible(idx);
        end

        if isempty(next_node)
            break;
        end

        next_node = next_node(1);

        curr_route = [curr_route, next_node]; %#ok<AGROW>
        curr_load = curr_load + demand(next_node);
        visited(next_node) = true;
        curr_node = next_node;
    end

    if length(curr_route) > 1
        curr_route = [curr_route, depot]; %#ok<AGROW>
        routes{end+1} = curr_route; %#ok<AGROW>
        used_caps(end+1) = curr_cap; %#ok<AGROW>
    end
end

if sum(visited) < n_nodes
    routes = {};
end

```

```

        used_caps = [];
    end
end

function route = apply_2opt(route, dist)
    best_route = route;
    n = length(route);
    improved = true;

    while improved
        improved = false;
        for i = 2:(n - 2)
            for j = (i + 1):(n - 1)
                d_old = dist(best_route(i-1), best_route(i)) +
dist(best_route(j), best_route(j+1));
                d_new = dist(best_route(i-1), best_route(j)) +
dist(best_route(i), best_route(j+1));

                if d_new < d_old
                    best_route(i:j) = fliplr(best_route(i:j));
                    improved = true;
                end
            end
        end
        route = best_route;
    end

function phero = deposit_pheromone(phero, routes, delta)
    for r = 1:length(routes)
        path = routes{r};
        for i = 1:(length(path) - 1)
            u = path(i);
            v = path(i+1);
            phero(u, v) = phero(u, v) + delta;
            phero(v, u) = phero(v, u) + delta;
        end
    end

function total_cost = calculate_total_cost(routes, dist)
    total_cost = 0;
    for r = 1:length(routes)
        path = routes{r};
        for i = 1:(length(path) - 1)
            total_cost = total_cost + dist(path(i), path(i+1));
        end
    end
end
end

```

```

function cost = run_hetero_nearest_neighbor(dist, demand,
vec_caps, depot)
    n = length(demand);
    visited = false(1, n);
    visited(depot) = true;
    cost = 0;

    for v = 1:length(vec_caps)
        if sum(visited) >= n, break; end
        curr_cap = vec_caps(v);
        curr = depot;
        load = 0;

        while sum(visited) < n
            unvisited = find(~visited);
            feasible = unvisited(load + demand(unvisited) <=
curr_cap);

            if isempty(feasible)
                cost = cost + dist(curr, depot);
                break;
            else
                [min_d, idx] = min(dist(curr, feasible));
                next_node = feasible(idx);
                cost = cost + min_d;
                load = load + demand(next_node);
                visited(next_node) = true;
                curr = next_node;
            end
        end
        if curr ~= depot
            cost = cost + dist(curr, depot);
        end
    end
end

```

Lampiran 14. Program Matlab Ant Colony Optimization skenario 3

```

clear; clc; close all;
tic

%% 1. INPUT PARAMETERS
excel_file = 'Matriks_Jarakkecil.xlsx';
sheet_name = 'Sheet1';
if exist(excel_file, 'file')
    distance_matrix = readmatrix(excel_file, 'Sheet', sheet_name);
else

```

```

    error('File Excel %s tidak ditemukan.', excel_file);
end
n_nodes = size(distance_matrix, 1);
depot = 1;

% INPUT KENDARAAN
num_vehicles = 5;
cap_comp1 = 16000;
cap_comp2 = 16000;

% INPUT DEMAND 2 KOMPARTEMEN
demand = [
    0 4705 288.04 1751.1 1951.3 1725.1 1078.6 3085.1 1467.9 2639.5
    1197.3 551.12 1954.7 5120 995.06 4363.7 3293.4 2512.7 1595.6
    606.78 2381.9 439.87 2450.1 3692.3 3122.7 1684.1 2351.1 1515
    1177.4 4160 2063.2; % Kompartemen 1
    0 3295 671.96 2408.9 3488.7 3394.9 1801.4 3954.9 1732.1 2480.5
    1362.7 408.88 3165.3 0 2204.9 3316.3 1826.6 2287.3 2884.4 993.22
    1458.1 200.13 3309.9 2707.7 1677.3 875.92 4688.5 3285 2022.6 0
    4016.8 % Kompartemen 2
];

% Validasi awal total demand vs total kapasitas armada
if sum(demand(1,:)) > (num_vehicles * cap_comp1) ||
sum(demand(2,:)) > (num_vehicles * cap_comp2)
    warning('PERINGATAN: Total demand melebihi kapasitas maksimum
    armada!');
end

%% 2. PARAMETER ACO
n_ants = 30;           % Jumlah semut per iterasi
max_iter = 100;        % Jumlah iterasi
alpha = 1;             % Bobot pengaruh jejak feromon
beta = 3;              % Bobot pengaruh visibilitas jarak
rho = 0.1;             % Laju penguapan feromon
Q = 6;                 % Konstanta intensitas feromon

% Inisialisasi Matriks Feromon
pheromone = ones(n_nodes, n_nodes);

% Inisialisasi Nilai Terbaik Global
best_distance = Inf;
best_routes = {};

%% 3. MAIN ACO ITERATION LOOP
fprintf('Memulai komputasi pencarian rute terbaik dengan
ACO...\n');
for iter = 1:max_iter
    all_ant_routes = cell(n_ants, 1);

```

```

all_ant_distances = zeros(n_ants, 1);
all_ant_valid = false(n_ants, 1);

for ant = 1:n_ants
    visited = false(1, n_nodes);
    visited(depot) = true;

    ant_routes = cell(num_vehicles, 1);
    current_vehicle = 1;

    for v = 1:num_vehicles
        ant_routes{v} = depot;
    end

    load_c1 = 0; load_c2 = 0;
    curr_node = depot;

    while sum(visited) < n_nodes
        unvisited = find(~visited);

        feasible = [];
        for idx = 1:length(unvisited)
            cand = unvisited(idx);
            if (load_c1 + demand(1, cand) <= cap_comp1) &&
(load_c2 + demand(2, cand) <= cap_comp2)
                feasible = [feasible, cand]; %#ok<AGROW>
            end
        end

        if isempty(feasible)
            if current_vehicle < num_vehicles
                ant_routes{current_vehicle} =
[ant_routes{current_vehicle}, depot];
                current_vehicle = current_vehicle + 1;
                curr_node = depot;
                load_c1 = 0;
                load_c2 = 0;
                continue;
            else
                break; % Armada habis sebelum semua pelanggan
terkunjungi
            end
        end

        probabilities = zeros(size(feasible));
        for idx = 1:length(feasible)
            next_cand = feasible(idx);
            dist_val = distance_matrix(curr_node, next_cand);
            if dist_val == 0, dist_val = 0.1; end
        end
    end
end

```

```

        eta = 1 / dist_val;
        probabilities(idx) = (pheromone(curr_node,
next_cand)^alpha) * (eta^beta);
    end

    if sum(probabilities) == 0
        next_node = feasible(randi(length(feasible)));
    else
        probabilities = probabilities /
sum(probabilities);
        cum_prob = cumsum(probabilities);
        r_val = rand();
        next_node = feasible(find(cum_prob >= r_val, 1,
'first'));
    end

    ant_routes{current_vehicle} =
[ant_routes{current_vehicle}, next_node];
    visited(next_node) = true;
    load_c1 = load_c1 + demand(1, next_node);
    load_c2 = load_c2 + demand(2, next_node);
    curr_node = next_node;
end

    ant_routes{current_vehicle} =
[ant_routes{current_vehicle}, depot];

    for v = 1:num_vehicles
        if length(ant_routes{v}) == 1
            ant_routes{v} = [depot, depot];
        end
    end

    if sum(visited) == n_nodes
        total_dist = 0;
        for v = 1:num_vehicles
            r_path = ant_routes{v};
            for n_idx = 1:(length(r_path)-1)
                total_dist = total_dist +
distance_matrix(r_path(n_idx), r_path(n_idx+1));
            end
        end

        all_ant_routes{ant} = ant_routes;
        all_ant_distances(ant) = total_dist;
        all_ant_valid(ant) = true;
    end
end
end

```

```

    % Update Solusi Terbaik Global
    valid_ants = find(all_ant_valid);
    if ~isempty(valid_ants)
        [min_dist, best_ant_idx] =
min(all_ant_distances(valid_ants));
        actual_best_ant = valid_ants(best_ant_idx);

        if min_dist < best_distance
            best_distance = min_dist;
            best_routes = all_ant_routes{actual_best_ant};
        end
    end

    % Evaporasi & Deposit Feromon
    pheromone = (1 - rho) * pheromone;
    if ~isempty(valid_ants)
        for ant = valid_ants'
            dist_eff = all_ant_distances(ant);
            for v = 1:num_vehicles
                r_path = all_ant_routes{ant}{v};
                for n_idx = 1:(length(r_path)-1)
                    n_from = r_path(n_idx);
                    n_to = r_path(n_idx+1);
                    pheromone(n_from, n_to) = pheromone(n_from,
n_to) + (Q / dist_eff);
                end
            end
        end
    end

    if mod(iter, 10) == 0 || iter == 1
        if best_distance < Inf
            fprintf('Iterasi %3d | Total Jarak Terbaik: %.2f\n',
iter, best_distance);
        else
            fprintf('Iterasi %3d | Mencari Solusi Valid...\n',
iter);
        end
    end

    %% 4. PENAMPILAN HASIL AKHIR RINGKASAN
    if isempty(best_routes)
        error('ACO gagal menemukan kombinasi rute yang valid. Coba
atur ulang parameter atau kapasitas.');
```

```

    end

    fprintf('\n=====');

```

```

fprintf('  HASIL OPTIMASI MCVRP 2 KOMPARTEMEN  \n');
fprintf('===== \n');
active_vehicles = 0;
for r = 1:num_vehicles
    curr_route = best_routes{r};
    if length(curr_route) > 2 % Mengabaikan kendaraan yang tidak
    terpakai [1 1]
        active_vehicles = active_vehicles + 1;
        load_c1 = sum(demand(1, curr_route));
        load_c2 = sum(demand(2, curr_route));

        route_dist = 0;
        for node_idx = 1:(length(curr_route)-1)
            route_dist = route_dist +
distance_matrix(curr_route(node_idx), curr_route(node_idx+1));
        end

        fprintf('Rute Kendaraan #%d : %s\n', r,
num2str(curr_route));
        fprintf(' - Jarak Rute      : %.2f\n', route_dist);
        fprintf(' - Muatan Komp. 1 : %.1f / %d (Aman)\n',
load_c1, cap_comp1);
        fprintf(' - Muatan Komp. 2 : %.1f / %d (Aman)\n\n',
load_c2, cap_comp2);
    end
end
fprintf('===== \n');
fprintf('TOTAL RINGKASAN OPTIMASI UTAMA: \n');
fprintf('Total Kendaraan Terpakai : %d dari %d kendaraan\n',
active_vehicles, num_vehicles);
fprintf('Total Jarak Terpendek      : %.2f\n', best_distance);
fprintf('Keterangan                  : Semua pelanggan sukses
dikunjungi! \n');
toc

%% 5. VISUALISASI PENYEBARAN RUTE 2D (MCVRP)
if best_distance < Inf && ~isempty(best_routes)
    fig = figure('Name', 'ACO3Kecil', 'Color', 'w');
    hold on; grid on; axis equal;

    % 1. Rekonstruksi Koordinat 2D menggunakan Multidimensional
Scaling (MDS)
    try
        [coords, ~] = cmdscale(distance_matrix, 2);
        x = coords(:, 1);
        y = coords(:, 2);
    catch
        % Fallback jika cmdscale tidak tersedia: Susun node secara
melingkar

```

```

        theta = linspace(0, 2*pi, n_nodes);
        x = [0, cos(theta(1:end-1)) * 10];
        y = [0, sin(theta(1:end-1)) * 10];
    end

    % 2. Warna Unik untuk Setiap Kendaraan
    colors = lines(num_vehicles);

    % 3. Plot Garis Rute Kendaraan Aktif
    for r = 1:num_vehicles
        r_path = best_routes{r};
        if length(r_path) > 2
            load_c1 = sum(demand(1, r_path));
            load_c2 = sum(demand(2, r_path));

            route_x = x(r_path);
            route_y = y(r_path);

            r_dist = 0;
            for n_idx = 1:(length(r_path)-1)
                r_dist = r_dist + distance_matrix(r_path(n_idx),
r_path(n_idx+1));
            end

            % Plot Garis Lintasan
            plot(route_x, route_y, '-o', 'LineWidth', 1.8,
'MarkerSize', 4, 'Color', colors(r, :), 'DisplayName',
sprintf('Kendaraan %d (C1: %.0f | C2: %.0f)', r, load_c1, load_c2,
r_dist));

            % Plot Panah Arah Perjalanan
            for k = 1:(length(r_path) - 1)
                u = x(r_path(k+1)) - x(r_path(k));
                v = y(r_path(k+1)) - y(r_path(k));
                quiver(x(r_path(k)), y(r_path(k)), u*0.7, v*0.7,
0, 'Color', colors(r, :), 'MaxHeadSize', 0.4, 'LineWidth', 1,
'HandleVisibility', 'off');
            end
        end
    end

    % 4. Plot Simbol Pelanggan & Depot
    scatter(x(2:end), y(2:end), 45, [0.15 0.45 0.85], 'filled',
'DisplayName', 'Pelanggan');
    plot(x(1), y(1), 'rp', 'MarkerSize', 16, 'MarkerFaceColor',
'r', 'DisplayName', 'Depot (Node 1)');

    % 5. Tampilkan Nomor Label Node
    for i = 1:n_nodes

```

```

        if i == 1
            text(x(i), y(i), ' Depot (1)', 'FontWeight', 'bold',
'Color', 'r', 'FontSize', 10);
        else
            text(x(i), y(i), sprintf(' %d', i), 'FontSize', 7,
'Color', [0.2 0.2 0.2]);
        end
    end

    title('Rute Hasil ACO Skenario 3 (Kecil)', 'FontSize', 12,
'FontWeight', 'bold');
    xlabel('Sumbu X (Relatif MDS)');
    ylabel('Sumbu Y (Relatif MDS)');
    legend('Location', 'eastoutside');
    hold off;

    figure(fig);
    drawnow;
end

```